

Cuestiones de diseño e implementación

A menudo, se invoca con frecuencia al método *getInstancia* del Singleton. En aplicaciones con varios hilos de ejecución, la etapa de creación de la lógica de **inicialización perezosa** (*lazy*) es una sección crítica que requiere el control de concurrencia del hilo. Por tanto, asumiendo que la instancia se inicializa de manera perezosa, es habitual envolver el método con control de concurrencia. En Java, por ejemplo:

```
public static synchronized FactoriaDeServicios getInstancia()
{
    if ( instancia == null )
    {
        //sección crítica si es una aplicación con varios hilos
        instancia = new FactoriaDeServicios();
    }
    return instancia;
}
```

A propósito de la inicialización perezosa, ¿no es preferible una **inicialización impaciente** (*eager*), como en este ejemplo?

```
public class FactoriaDeServicios
{
    //inicialización impaciente
    private static FactoriaDeServicios instancia =
        new FactoriaDeServicios();

    public static FactoriaDeServicios getInstancia()
    {
        return instancia;
    }

    //otros métodos...
}
```

Es preferible el primer enfoque de inicialización perezosa al menos por estas razones:

- Se evita el trabajo de creación (y quizás retener recursos “caros”) si nunca se accede a la instancia.
- La inicialización perezosa de *getInstancia* algunas veces contiene lógica de creación compleja y condicional.

Otra pregunta típica de la implementación del Singleton es: ¿por qué no hacer que todos los métodos de los servicios sean métodos *estáticos* de la propia clase, en lugar de utilizar un objeto instancia con métodos de instancia? Por ejemplo, qué pasa si añadimos un método *estático* denominado *getAdaptadorContabilidad* a la *FactoriaDeServicios*. Pero normalmente es preferible utilizar una instancia y los métodos de instancia por estos motivos:

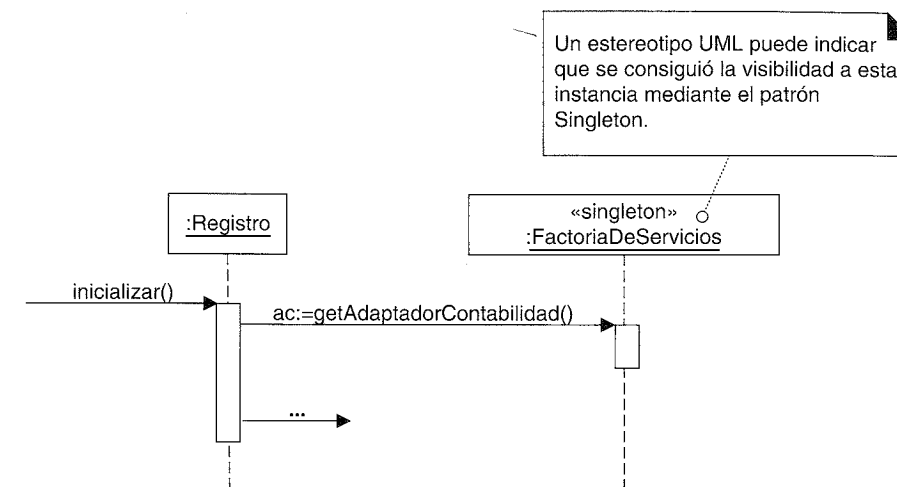


Figura 23.6. Mensaje implícito del patrón Singleton *getInstancia* que se indica en UML con un estereotipo.

- Los métodos de instancia permiten que se definan subclases y se refinan la clase singleton; los métodos estáticos no son polimórficos (virtual) y en la mayoría de los lenguajes (excepto Smalltalk) no se pueden redefinir en las subclases.
- La mayoría de los mecanismos de comunicación remota orientada a objetos (por ejemplo el RMI de Java) sólo soportan la activación remota de los métodos de instancia, no de los métodos estáticos. Se podría activar de manera remota una instancia singleton, aunque es verdad que raramente se hace.
- Una clase no siempre es un singleton en todos los contextos de aplicación. En la aplicación X, podría ser un singleton, pero podría ser una “multi-tonelada” en la aplicación Y. También es habitual comenzar un diseño pensando que será un singleton y luego descubrir que se necesitan múltiples instancias en el mismo proceso. Por tanto, la solución basada en la instancia es más flexible.

Patrones Relacionados

El patrón Singleton se utiliza a menudo para los objetos Factoría y Fachada —otro patrón GoF que se presentará—.

23.5. Conclusiones del problema de los servicios externos con diversas interfaces

Se ha utilizado una combinación de los patrones Adaptador, Factoría y Singleton para proporcionar Variaciones Protegidas contra las diversas interfaces de los calculadores de impuestos externos, sistemas de contabilidad, etcétera. La Figura 23.7 ilustra un contexto más amplio de la utilización de los patrones en la realización del caso de uso.

Este diseño podría no ser ideal, y siempre se puede mejorar. Pero uno de los objetivos que intenta conseguir este caso de estudio es demostrar que se puede construir por lo menos un diseño a partir de un conjunto de principios o patrones que son los “componentes básicos”, y que existe un enfoque metódico para llevar a cabo y explicar un di-

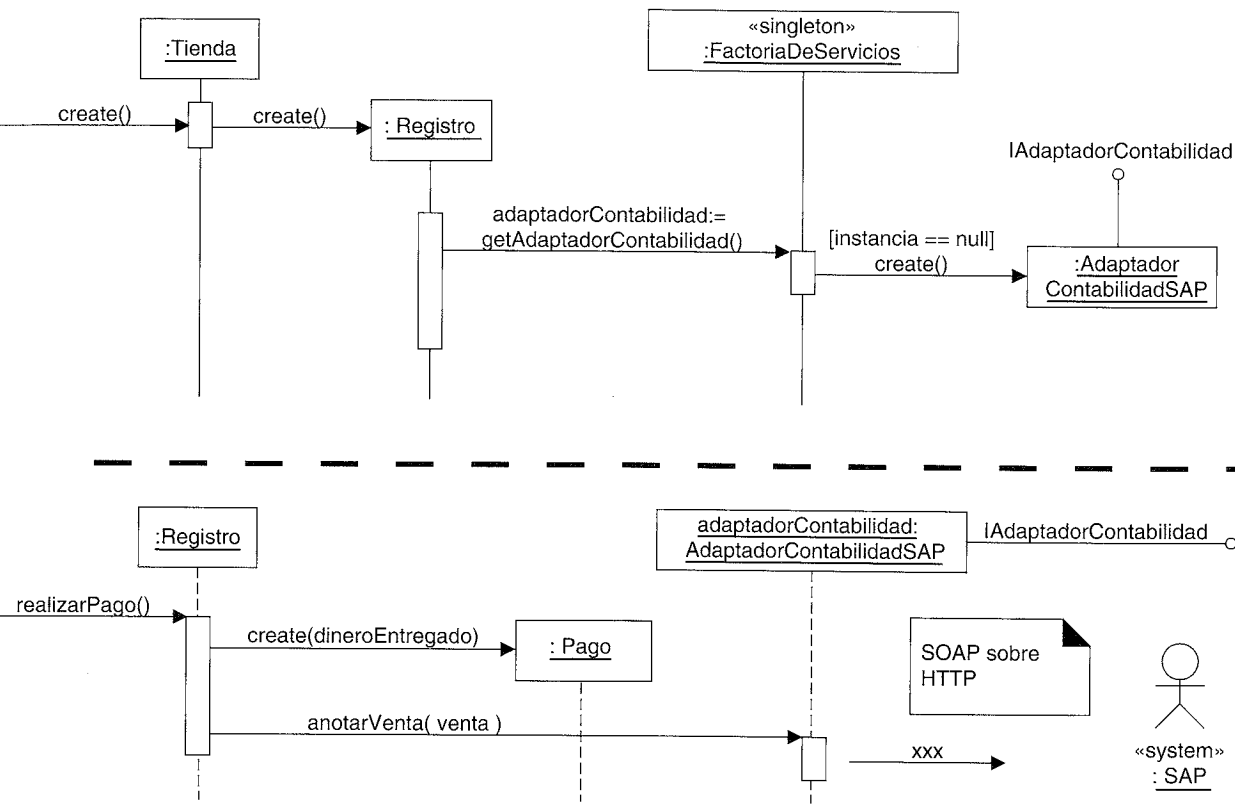


Figura 23.7. Patrones Adaptador, Factoría y Singleton aplicados al diseño.

seño. Espero sinceramente que se pueda ver cómo surge el diseño de la Figura 23.7 razonando en base al Controlador, Creador, Variaciones Protegidas, Bajo Acoplamiento, Alta Cohesión, Indirección, Polimorfismo, Adaptador, Factoría y Singleton.

Nótese lo conciso que puede ser un diseñador en una conversación o en la documentación cuando hay un conocimiento común de los patrones. Puedo decir, “Para resolver el problema de tener diversas interfaces para los servicios externos, podemos utilizar Adaptadores generados desde una Factoría Singleton”. En realidad, los diseñadores de objetos tienen conversaciones de este tipo; utilizando los patrones y los nombres de los patrones se eleva el nivel de abstracción en la comunicación del diseño.

23.6. Estrategia (GoF)

El siguiente problema de diseño que se va a resolver es proporcionar una lógica más compleja para fijar los precios, como descuentos para un día en toda la tienda, descuentos para las personas mayores, etcétera.

La estrategia de fijación de precios (que podría llamarse también regla, política o algoritmo) de una venta puede variar. Durante un periodo podría ser el 10% de descuento en todas las ventas, después podría ser un descuento de 10 € si el total de la venta es superior a 200 €, y muchas otras variaciones. ¿Cómo diseñamos los diversos algoritmos de fijación de precios?

Estrategia (Strategy)

Contexto/Problema

¿Cómo diseñar diversos algoritmos o políticas que están relacionadas? ¿Cómo diseñar que estos algoritmos o políticas puedan cambiar?

Solución

Defina cada algoritmo/política/estrategia en una clase independiente, con una interfaz común.

Puesto que el comportamiento de la fijación de precios varía según la estrategia (o algoritmo), creamos múltiples clases *EstrategiaFijarPreciosVenta*, cada una con un método polimórfico *getTotal* (ver Figura 23.8). A cada método *getTotal* se le pasa como parámetro el objeto *Venta*, de manera que el objeto de la estrategia de fijación de precios puede encontrar el precio anterior al descuento de la *Venta*, para aplicar después la regla de descuento. La implementación de cada *getTotal* será diferente: la *EstrategiaFijarPreciosPorcentajeDescuento*, aplicará el descuento de acuerdo a un porcentaje, y así sucesivamente.

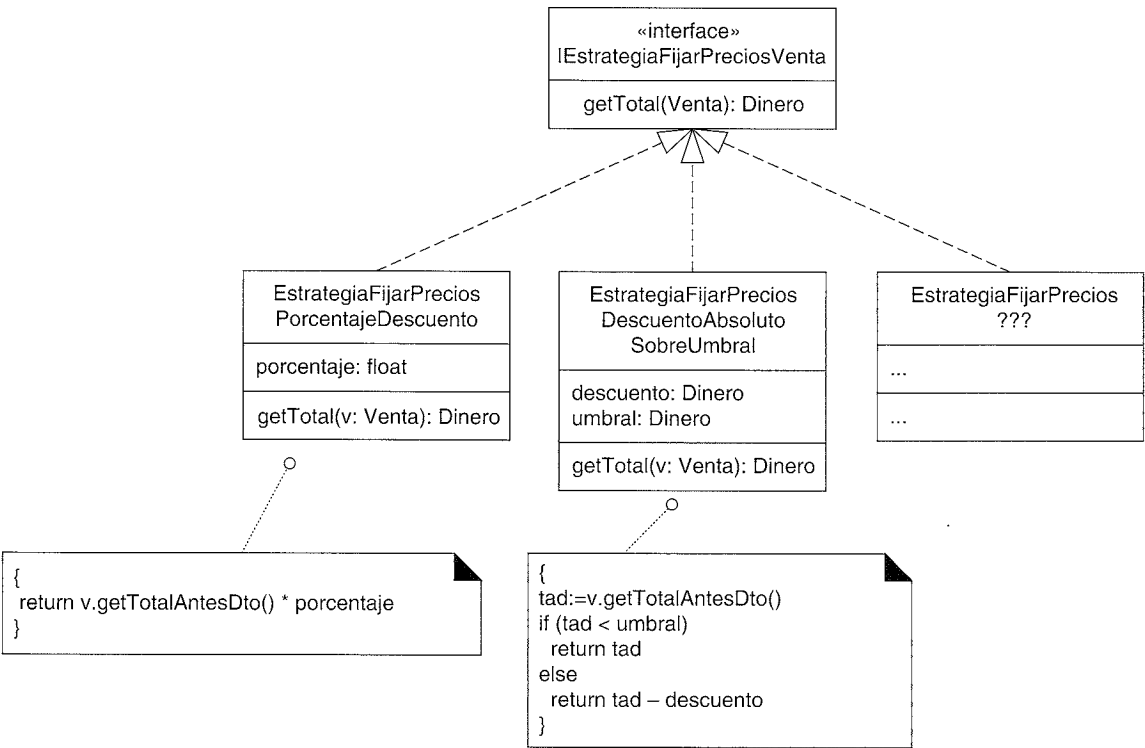


Figura 23.8. Clases para la Estrategia de fijación de precios.

Un objeto estrategia se conecta a un **objeto de contexto** —el objeto al que se aplica el algoritmo—. En este ejemplo, el objeto de contexto es una *Venta*. Cuando se envía el mensaje *getTotal* a la *Venta*, delega parte del trabajo a su objeto estrategia, como se ilustra en la Figura 23.9. No es necesario que el mensaje que se envía al objeto de contexto

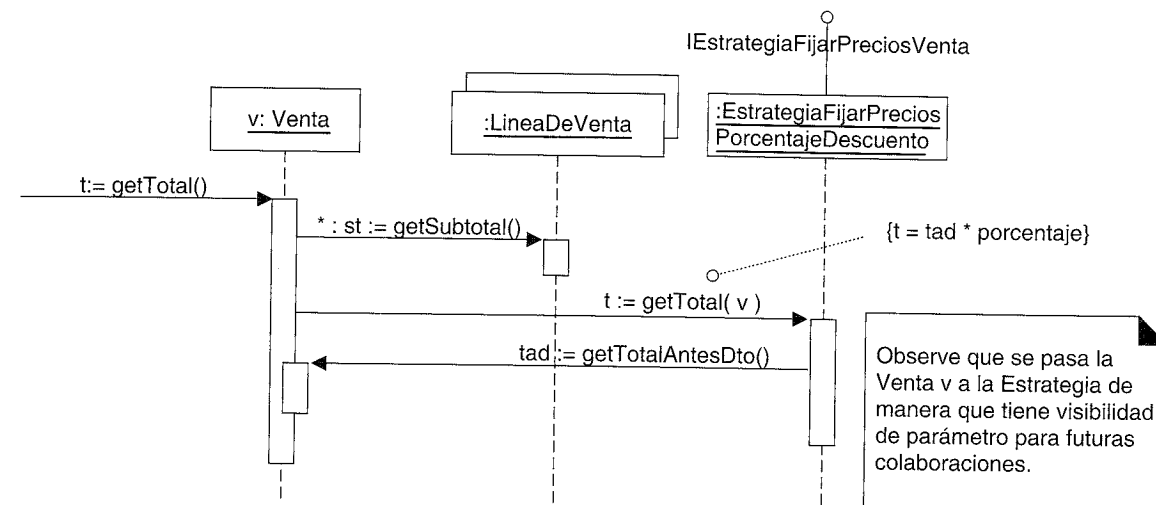


Figura 23.9. Estrategia en colaboración.

y al objeto estrategia tengan el mismo nombre, como en este ejemplo (por ejemplo, *getTotal* y *getTotal*), aunque es lo normal. Sin embargo, es habitual —de hecho, normalmente necesario— que el objeto de contexto pase una referencia a él mismo (*this*) al objeto estrategia, de manera que la estrategia tenga visibilidad de parámetro del objeto contexto, para futuras colaboraciones.

Obsérvese que el objeto de contexto (*Venta*) necesita tener visibilidad de atributo de su estrategia. Esto se refleja en el DCD de la Figura 23.10.

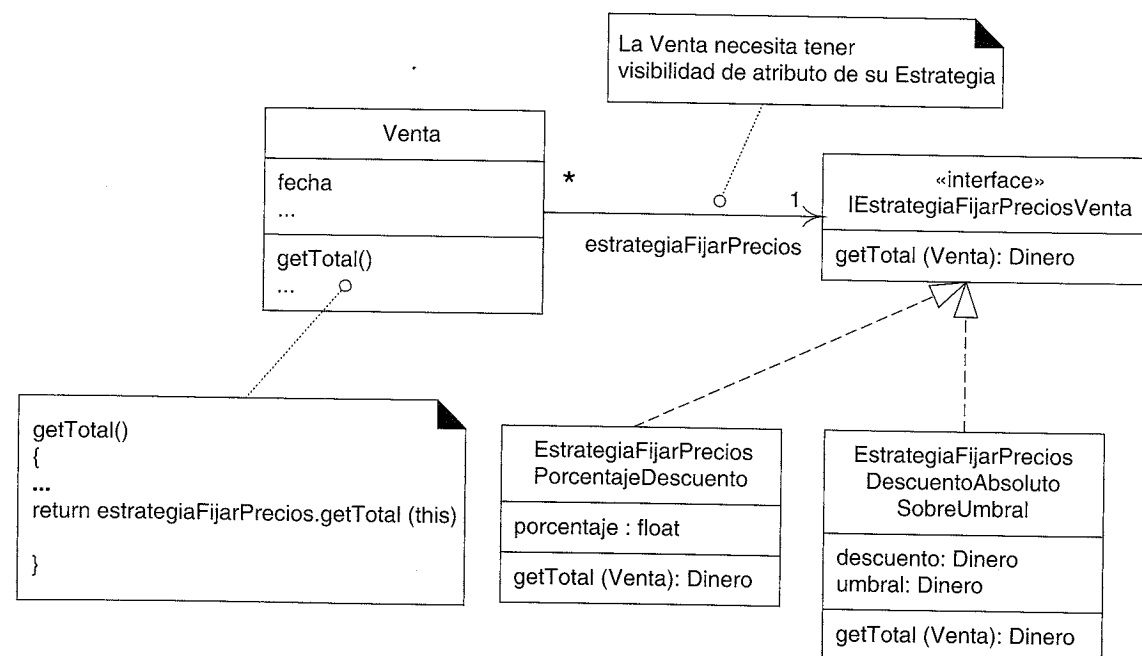


Figura 23.10. Los objetos de contexto necesitan tener visibilidad de atributo de su estrategia.

Creación de una Estrategia con una Factoría

Existen diferentes algoritmos o estrategias de fijación de precios, y cambian con el tiempo. ¿Quién debería crear la estrategia? Un enfoque directo es aplicar de nuevo el patrón Factoría: una *FactoriaDeEstrategiasFijarPrecios* puede ser la responsable de crear todas las estrategias (todos los algoritmos o las políticas conectables o cambiantes) que se necesitan en la aplicación. Como con la *FactoriaDeServicios*, se puede leer el nombre de la clase de implementación de la estrategia de fijación de precios como una propiedad del sistema (o alguna fuente de datos externa), y después crear una instancia. Con este *diseño dirigido por los datos* parcial (o *diseño reflexivo*) uno puede cambiar dinámicamente en cualquier momento —mientras se está ejecutando la aplicación del PDV NuevaEra— la política de fijación de precios, especificando la creación de una clase de Estrategia diferente.

Obsérvese que se utiliza una nueva factoría para las estrategias; es decir, diferente a la *FactoriaDeServicios*. Esto se ajusta al objetivo de Alta Cohesión —cada factoría está centrada de manera cohesiva en la creación de una familia de objetos relacionados—.

Notación UML: Observe que en la Figura 23.10 se referencia mediante una asociación directa a la interfaz *IEstrategiaFijarPreciosVenta*, no a una clase concreta. Eso indica que el atributo de referencia en la *Venta* se declarará en términos de la interfaz, no de una clase, de manera que el atributo se puede ligar con cualquier implementación de la interfaz.

Nótese que debido a que la política de fijación de precios cambia con frecuencia (podría ser cada hora), *no* es conveniente almacenar la instancia de la estrategia creada en un campo de la *FactoriaDeEstrategiasFijarPrecios*, sino crear una cada vez, leyendo la propiedad externa para obtener el nombre de la clase e instanciando luego la estrategia.

Y como con la mayoría de las factorías, la *FactoriaDeEstrategiasFijarPrecios* será un singleton (una instancia) y se accederá mediante el patrón Singleton (ver Figura 23.11).

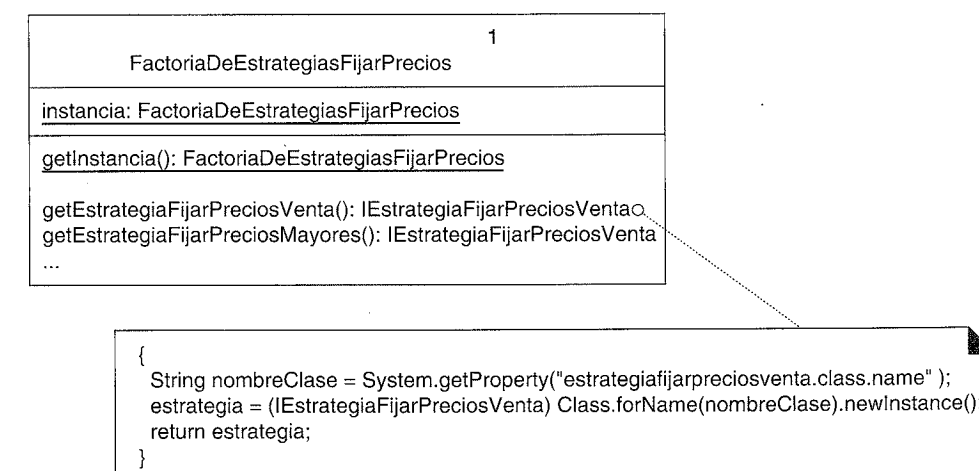


Figura 23.11. Factoría de estrategias.

Cuando se crea una instancia de la *Venta*, puede pedir a la factoría su estrategia de fijación de precios, como se muestra en la Figura 23.12.

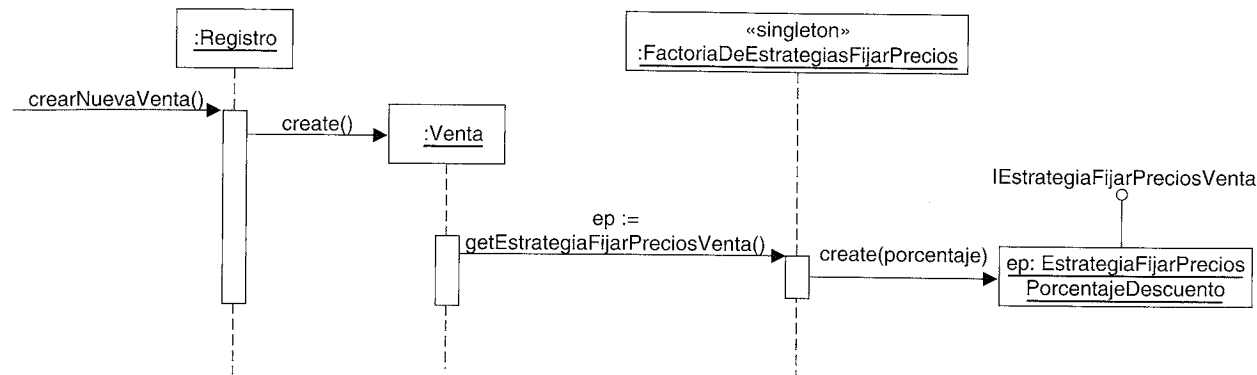


Figura 23.12. Creación de una estrategia.

Lectura e inicialización del valor del porcentaje

Finalmente, un problema de diseño que se ha ignorado hasta ahora es cómo encontrar los diferentes valores de los porcentajes o descuentos absolutos. Por ejemplo, el valor del porcentaje de la *EstrategiaFijarPreciosPorcentajeDescuento* podría ser 10% el lunes, pero 20% el martes.

Nótese también que el porcentaje de descuento podría estar relacionado con el tipo del comprador, como una persona mayor, en lugar de con un periodo de tiempo.

Estos números se almacenarán en algún almacén de datos externo, como una base de datos relacional, por lo que se pueden cambiar fácilmente. Luego, ¿qué objeto los leerá y asegurará que se asignan a la estrategia? Una opción razonable es la propia *FactoriaDeEstrategias*, puesto que crea la estrategia para fijar precios, y puede saber cuál es el porcentaje que hay que leer del almacén de datos (“el descuento actual del almacén”, “el descuento de las personas mayores”, etcétera).

Los diseños para la lectura de estos números procedentes de los almacenamientos de datos externos, varían desde los más simples a los más complejos, como una simple llamada SQL JDBC (por ejemplo, si se utilizan las tecnologías Java) o la colaboración con objetos que añaden niveles de indirección para ocultar la localización concreta, el lenguaje de consulta de los datos, o el tipo de almacén de datos. El análisis de los puntos de variación y evolución con respecto al almacén de datos revelará si es necesario que se proteja contra las variaciones. Por ejemplo, podríamos preguntar, “¿Estamos todos cómodos con el compromiso a largo plazo de utilizar una base de datos relacional que entienda SQL?”. Si es así, podría ser suficiente una simple llamada JDBC desde la *FactoriaDeEstrategias*.

Resumen

Con los patrones Estrategia y Factoría se ha conseguido Variaciones Protegidas con respecto a las políticas para fijar precios que varían dinámicamente. La Estrategia se fundamenta en el Polimorfismo y las interfaces para permitir algoritmos conectables en un diseño de objetos.

Patrones Relacionados

El patrón Estrategia se basa en el Polimorfismo, y proporciona Variaciones Protegidas con respecto a los algoritmos que cambian. Las Estrategias normalmente se crean mediante una Factoría.

23.7. Composite⁵ (GoF) y otros principios de diseño

Todavía surge otro interesante problema en los requisitos y el diseño: ¿cómo gestionamos el caso de varias políticas contradictorias de fijación de precios? Por ejemplo, suponga que una tienda tiene hoy (lunes) en vigor las siguientes políticas:

- Política de descuento del 20% a las personas mayores.
- Descuento del 15% en compras superiores a los 400 € para clientes preferentes.
- Los lunes, hay un 50% de descuento en las compras superiores a 500 €.
- Comprando una caja de té Darjeeling, obtiene el 15% de descuento en todo.

Suponga que una persona mayor que también es cliente preferente compra una caja de té Darjeeling, y 600 € de hamburguesas vegetales (claramente un vegetariano entusiasta al que le encanta el chai). ¿Qué política de fijación de precios se debería aplicar?

Explicándolo más claramente: ahora las estrategias de fijación de precios que se conectan a la venta dependen de tres factores:

- Periodo de tiempo (lunes).
- Tipo de cliente (persona mayor).
- Un producto concreto (té Darjeeling).

Otro punto que hay que aclarar: tres de las cuatro políticas del ejemplo son en realidad simplemente estrategias de “porcentaje de descuento”, lo que simplifica nuestra perspectiva del problema.

Parte de la respuesta a este problema requiere que se defina la **estrategia de resolución de conflictos** de la tienda. Normalmente, una tienda aplica “lo mejor para el cliente” (el precio más bajo) como estrategia de resolución de conflictos, pero no es obligatorio y podría cambiar. Por ejemplo, durante un periodo con dificultades financieras, la tienda podría verse obligada a utilizar la estrategia de resolución de conflictos “el precio más alto”.

El primer punto a tener en cuenta es que pueden existir múltiples estrategias coexistiendo al mismo tiempo, es decir, una venta podría tener asociadas varias estrategias para fijar el precio. Otro punto a destacar es que una estrategia de fijación de precios puede estar relacionada con el tipo de cliente (por ejemplo, una persona mayor). Esto afecta al diseño de la creación: la *FactoriaDeEstrategias* debe conocer el tipo del cliente en el momento de la creación de una estrategia de fijación de precios para el cliente.

Análogamente, una estrategia de fijación de precios puede estar relacionada con el tipo de producto que se compra (por ejemplo, té Darjeeling). Esto, del mismo modo, tiene implicaciones en el diseño de la creación: la *FactoriaDeEstrategias* debe conocer la *EspecificacionDelProducto* en el momento de la creación de una estrategia de fijación de precios influenciada por el producto.

⁵ N. del T.: No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

¿Hay alguna forma de cambiar el diseño de manera que el objeto *Venta* no conozca si está tratando con una o más estrategias, y ofrecer también un diseño para la resolución de conflictos? Sí, con el patrón Composite.

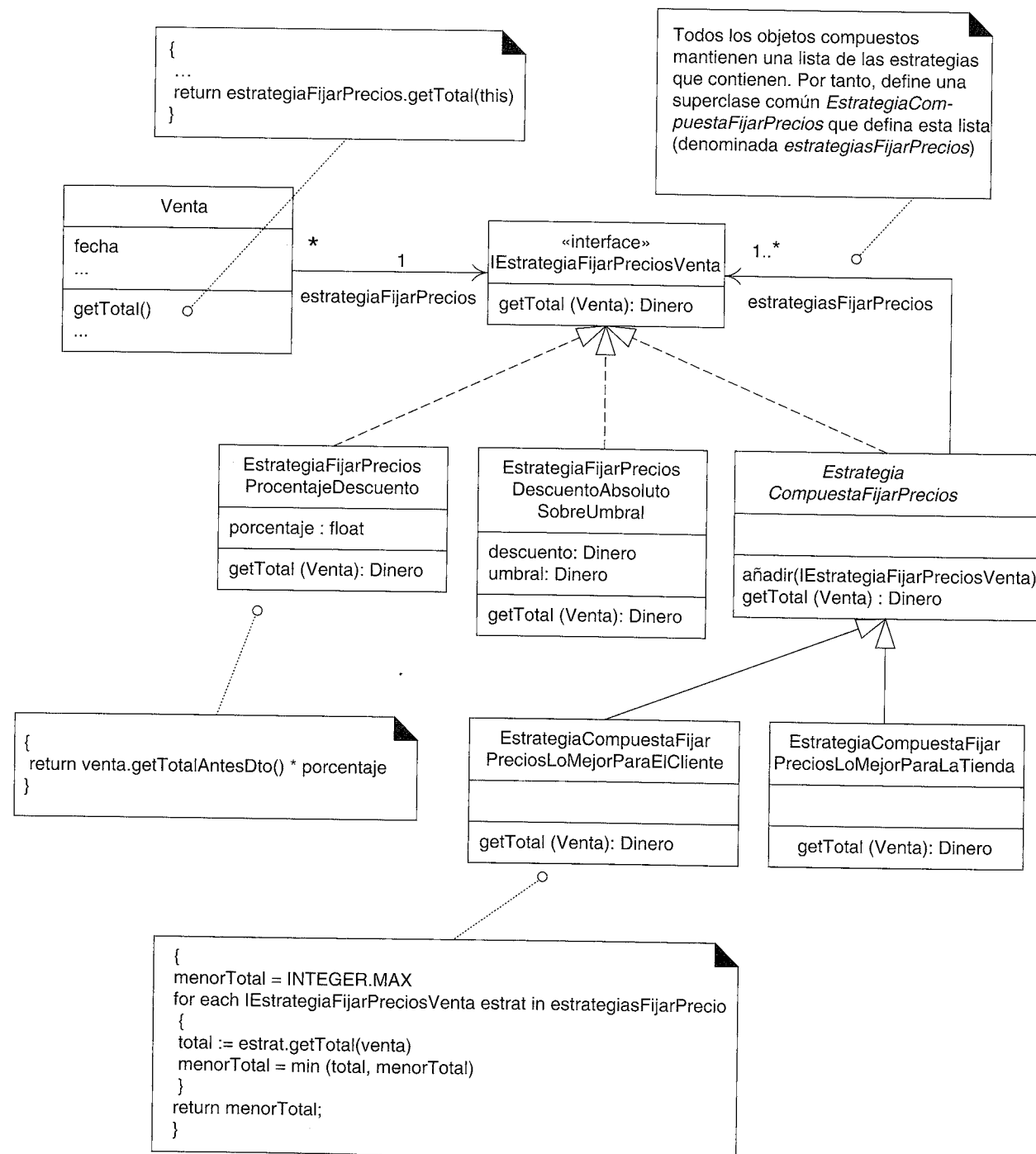


Figura 23.13. El patrón Composite.

Composite

Contexto/Problema

¿Cómo tratar un grupo o una estructura compuesta del mismo modo (polimórficamente) que un objeto no compuesto (atómico)?

Solución

Defina las clases para los objetos compuestos y atómicos de manera que implementen el mismo interfaz.

Por ejemplo, una nueva clase denominada *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* (bueno, por lo menos es descriptivo) puede implementar la *IEstrategiaFijarPreciosVenta* y ella misma contiene otros objetos *IEstrategiaFijarPreciosVenta*. La Figura 23.13 explica en detalle la idea de diseño.

Observe que en este diseño, la clase compuesta como *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* hereda el atributo *estrategiasFijarPrecios* que contiene una lista de más objetos *IEstrategiaFijarPreciosVenta*. Ésta es una característica distintiva de un objeto compuesto: el objeto compuesto externo contiene una lista de los objetos internos, y tanto los objetos externos como los internos implementan la misma interfaz. Es decir, la propia clase compuesta implementa la interfaz *IEstrategiaFijarPreciosVenta*.

Por tanto, podemos conectar al objeto *Venta* tanto a un objeto *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* (que contiene otras estrategias dentro de él) o a un objeto atómico *EstrategiaFijarPreciosPorcentajeDescuento*, y la *Venta* no conoce o no se preocupa si su estrategia de fijación de precios es atómica o compuesta —parecen iguales para el objeto *Venta*—. Es simplemente otro objeto que implementa la interfaz *IEstrategiaFijarPreciosVenta* y entiende el mensaje *getTotal* (Figura 23.14).

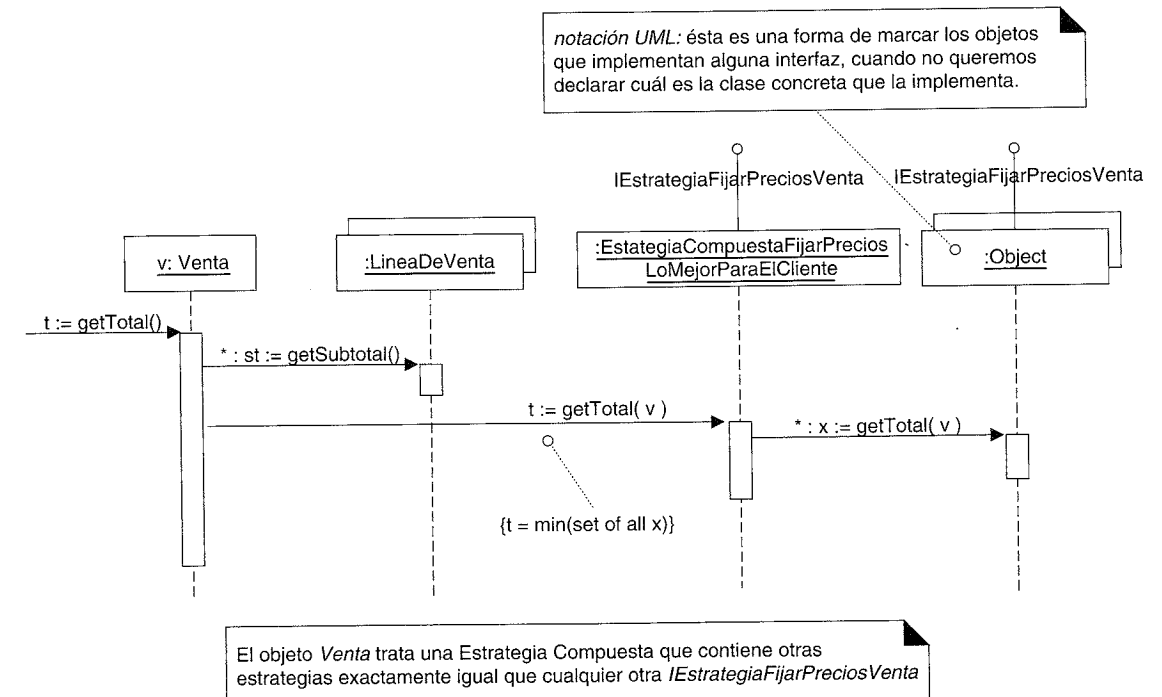


Figura 23.14. Colaboración con un Composite.

Notación UML: En la Figura 23.14, nótese, por favor, el modo de indicar los objetos que implementan una interfaz, cuando no nos preocupa especificar la clase de implementación exacta. Simplemente especificando la clase de implementación como *Object* queremos decir “sin comentarios” acerca de la clase específica. Ésta es una necesidad habitual en la elaboración de diagramas.

A continuación definimos la *EstrategiaCompuestaFijarPrecios* y una de sus subclases para aclarar la explicación con una muestra de código en Java:

```
//superclase luego todas las subclases heredan una List de estrategias

public abstract class EstrategiaCompuestaFijarPrecios
    implements IEstrategiaFijarPreciosVenta
{
    protected List estrategiasFijarPrecios = new ArrayList();

    public añadir(IEstrategiaFijarPreciosVenta e)
    {
        estrategiasFijarPrecios.add(e);
    }

    public abstract Dinero getTotal( Venta v );
}

//final de la clase

//Una Estrategia Compuesta que devuelve el total más pequeño
//de sus instancias EstrategiaFijarPreciosVenta internas

public class EstrategiaCompuestaFijarPreciosLoMejorParaElCliente
    extends EstrategiaCompuestaFijarPrecios
{
    public Dinero getTotal(Venta venta)
    {
        Dinero menorTotal = new Dinero (Integer.MAX_VALUE);

        //iteramos sobre todas las estrategias internas

        for( Iterator i = estrategiasFijarPrecios.iterator(); i.hasNext(); )
        {
            IEstrategiaFijarPreciosVenta estrategia =
                (IEstrategiaFijarPreciosVenta)i.next();
            Dinero total = estrategia.getTotal(venta);
            menorTotal = total.min(menorTotal);
        }
        return menorTotal();
    }
}

//final de la clase
```

Notación UML: la Figura 23.13 introduce nueva notación UML para representar las jerarquías de clases y la herencia, que se explica en la Figura 23.15.

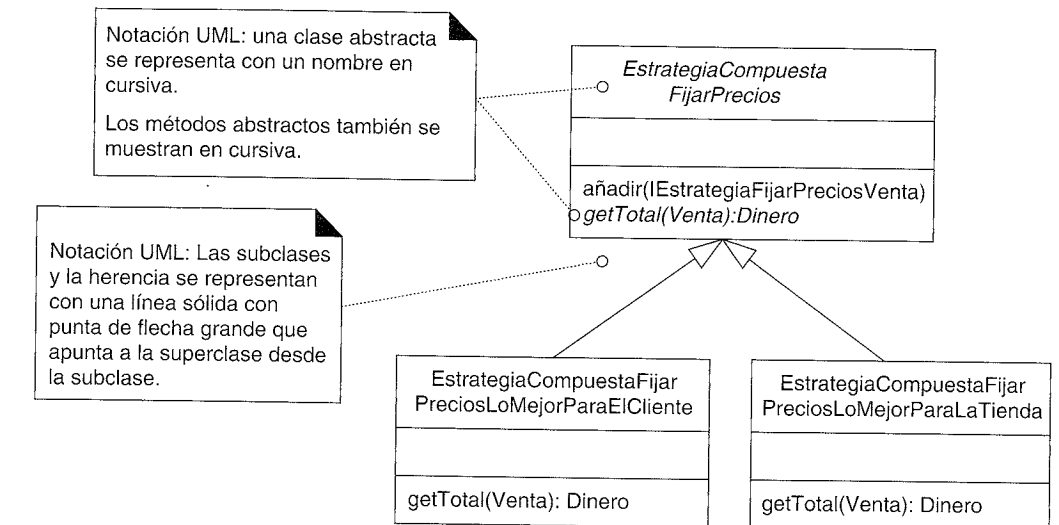


Figura 23.15. Superclases abstractas, métodos abstractos y herencia en UML.

Creación de múltiples instancias EstrategiaFijarPreciosVenta

Con el patrón Composite, hemos creado un grupo de diferentes (y contradictorias) estrategias de fijación de precios, que para *Venta* aparecen como una única estrategia. El objeto compuesto que contiene el grupo también implementa la interfaz *IEstrategiaFijarPreciosVenta*. La parte más desafiante (e interesante) de este problema de diseño es: ¿cuándo creamos estas estrategias?

Un diseño deseable comenzará creando un Composite que contenga las políticas de descuento de la tienda en el momento actual (que podría ser 0% de descuento si no hay ninguna activa), como alguna *EstrategiaFijarPreciosPorcentajeDescuento*. Entonces, si en un paso posterior del escenario, se descubre que se debe aplicar otra estrategia para fijar precios (como el descuento a las personas mayores), se añadirá fácilmente al objeto compuesto utilizando el método heredado *EstrategiaCompuestaFijarPrecios.añadir*.

Existen tres puntos en el escenario donde se podrían agregar al objeto compuesto las estrategias de fijación de precios:

1. Descuento actual a nivel de tienda, se añade cuando se crea la venta.
2. Descuento por el tipo de cliente, se añade cuando se informa al PDV del tipo de cliente.
3. Descuento según el tipo de producto (si compra el té Darjeeling obtendrá un descuento del 15% sobre el total de la venta), se añade cuando se introduce el producto en la venta.

El diseño del primer caso se muestra en la Figura 23.16. Como en el diseño original que se discutió anteriormente, el nombre de la clase estrategia que se va a instanciar se podría leer como una propiedad del sistema, y el valor del porcentaje se podría leer desde un almacenamiento de datos externo.

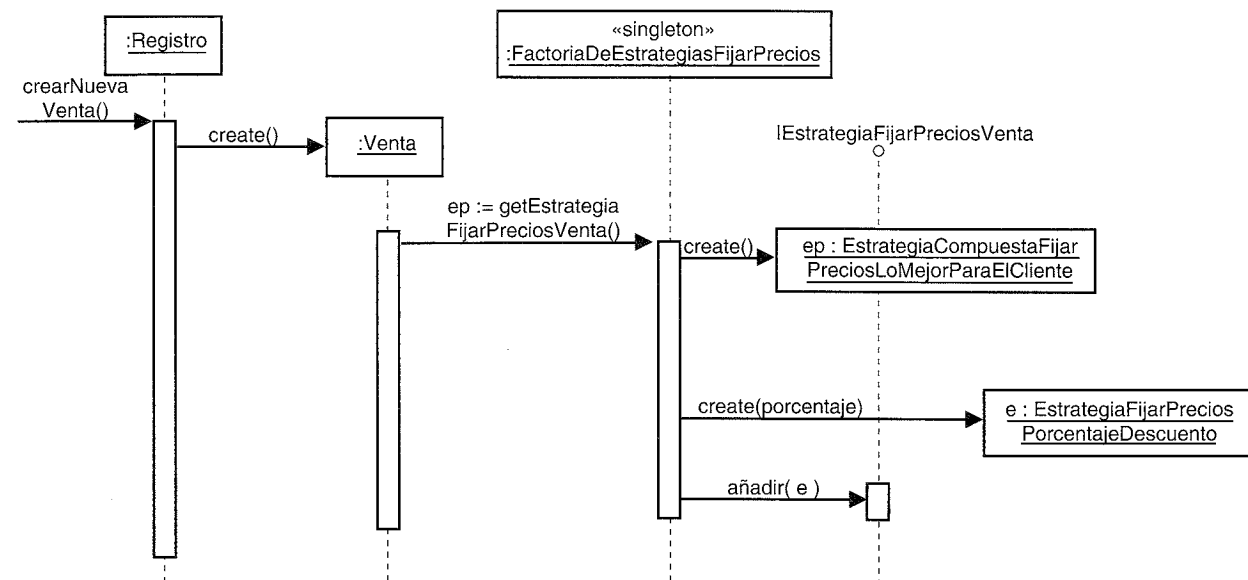


Figura 23.16. Creación de una estrategia compuesta.

Para el segundo caso de descuento según el tipo de cliente, en primer lugar recordemos la extensión del caso de uso que identificó anteriormente este requisito:

Caso de uso UC1: Procesar Venta

... Extensiones (o Flujos Alternativos):

- 5b. El Cliente dice que le son aplicables descuentos (ej. empleado, cliente preferente):
1. El Cajero señala la petición de descuento.
 2. El Cajero introduce la identificación del Cliente.
 3. El Sistema presenta el descuento total, basado en las reglas de descuento.

Esto indica una nueva operación del sistema en el sistema de PDV, además de *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta* y *realizarPago*. Llamaremos a esta quinta operación del sistema *introducirClienteParaDescuento*; opcionalmente podría tener lugar después de la operación *finalizarVenta*. Esto implica que se deberá introducir algún tipo de identificación del cliente a través de la interfaz de usuario, el *clienteID*. Quizás podría capturarse mediante un lector de tarjetas o a través del teclado.

El diseño de este segundo caso se muestra en las Figuras 23.17 y 23.18. No sorprende que el objeto factoría sea el responsable de la creación de la estrategia de fijación de precios adicional. Podría crear otra *EstrategiaFijarPreciosPorcentajeDescuento* que represente, por ejemplo, un descuento para las personas mayores. Pero como con el di-

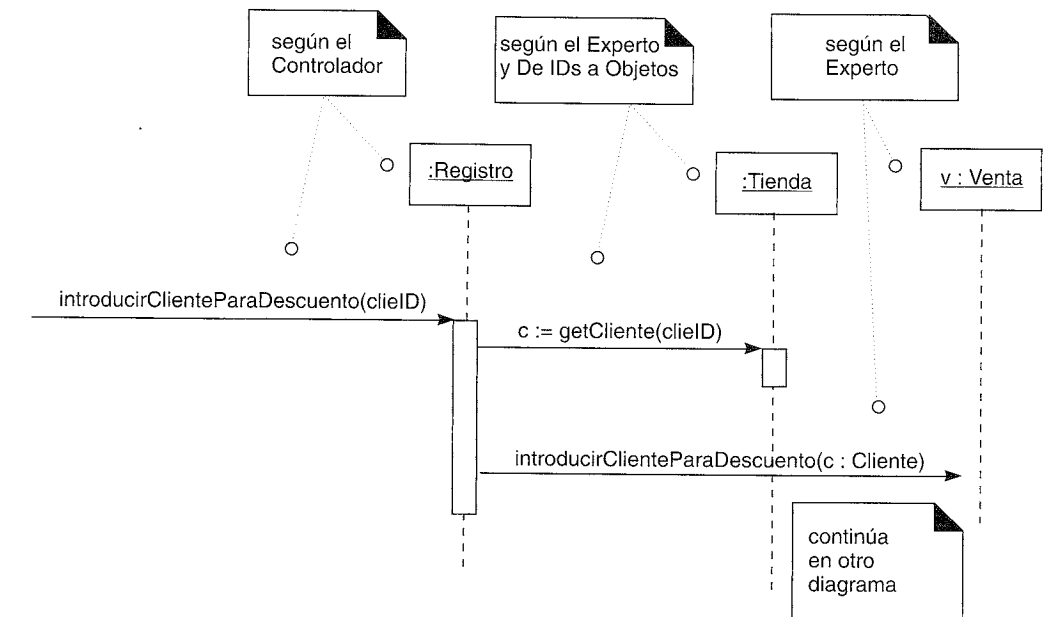


Figura 23.17. Creación de la estrategia de fijación de precios para el descuento de los clientes, parte 1.

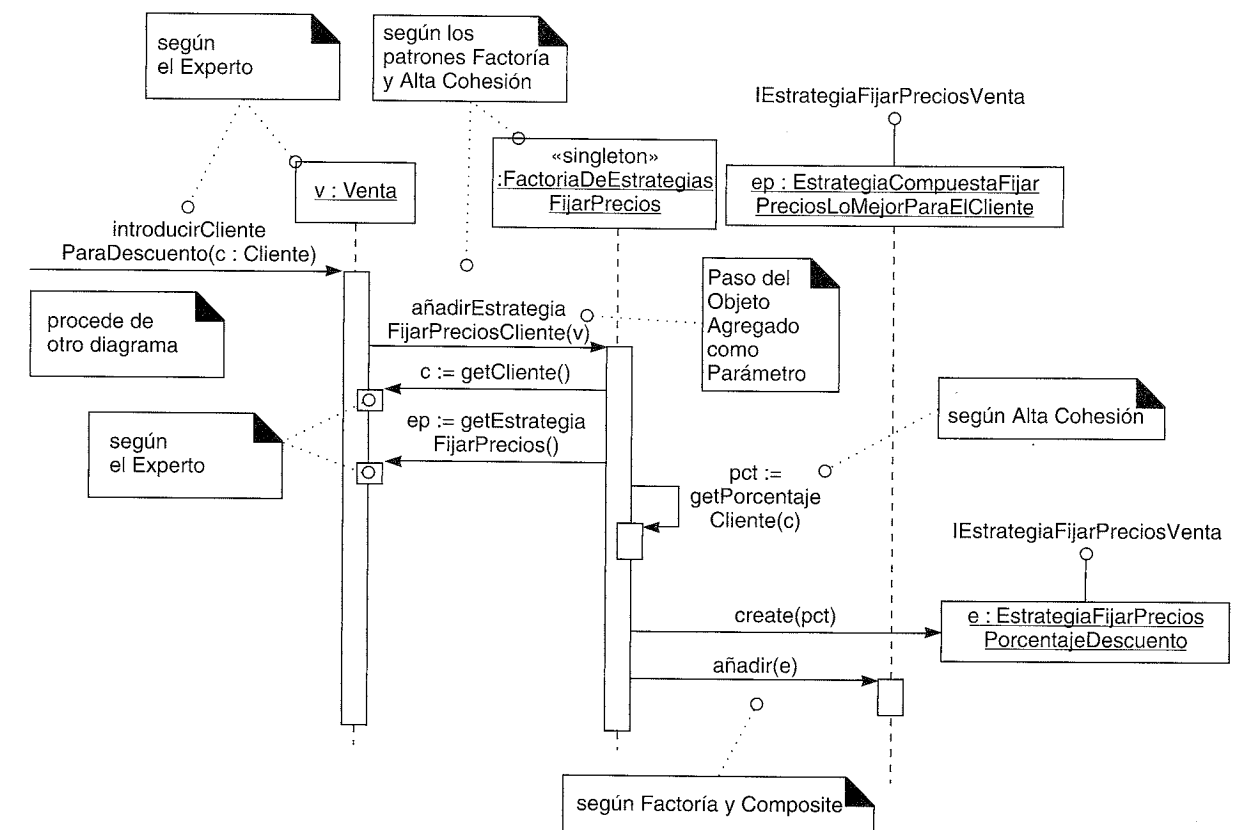


Figura 23.18. Creación de la estrategia de fijación de precios para el descuento de los clientes, parte 2.

seño de la creación original, la elección de la clase se leerá como una propiedad del sistema, al igual que el porcentaje específico para el tipo de cliente, para proporcionar Variaciones Protegidas con respecto a los cambios de las clases o los valores. Nótese que en virtud del patrón Composite, la *Venta* podría tener conectadas dos o tres estrategias contradictorias de fijación de precios, pero sigue pareciendo una única estrategia para el objeto *Venta*.

Notación UML: Las Figuras 23.17 y 23.18 muestran una importante idea UML sobre los diagramas de interacción: la división de un diagrama en dos, para que cada uno de ellos sea más legible.

Consideración de los patrones GRASP y otros principios en el diseño

Revisemos el razonamiento en función de algunos de los patrones GRASP básicos: para este segundo caso, ¿por qué no envía el *Registro* un mensaje a la *FactoriaDeEstrategiasFijarPrecios*, para crear esta nueva estrategia de fijación de precios y pasársela entonces a la *Venta*? Uno de los motivos es para mantener Bajo Acoplamiento. La *Venta* ya está acoplada a la factoría; si hacemos que el *Registro* también colabore con ella, incrementaríamos el acoplamiento en el diseño. Además, la *Venta* es el Experto en Información que conoce su estrategia actual de fijación de precios (que se va a modificar); luego, según el Experto, está justificado que se delegue en la *Venta*.

Obsérvese en el diseño que el *clienteID* se transforma en un objeto *Cliente*, preguntando el *Registro* a la *Tienda* por un *Cliente*, dado un ID. En primer lugar, se puede justificar que se otorgue a la *Tienda* la responsabilidad *getClientes*; de acuerdo con el Experto en Información y el objetivo de salto en la representación reducido, la *Tienda* puede conocer a todos los *Clientes*. Y el *Registro* pregunta a la *Tienda*, porque el *Registro* ya tiene visibilidad de atributo de la *Tienda* (a partir del trabajo de diseño previo); si la *Venta* tuviera que preguntar a la *Tienda*, la *Venta* necesitaría una referencia a la *Tienda*, lo que incrementaría el acoplamiento más allá de los niveles actuales y, por tanto, no mantendría Bajo Acoplamiento.

Transformación de IDs en objetos

Segundo, ¿por qué transformar *clienteID* (un “ID” —quizás un número—) en un objeto *Cliente*? Ésta es una práctica habitual del diseño de objetos —transformar claves e identificadores (IDs) de las cosas en verdaderos objetos—. A menudo esta transformación tiene lugar poco después de que se introduzca el ID o la clave en la capa del dominio del Modelo de Diseño desde la capa de UI. No tiene nombre de patrón, pero podría ser un candidato puesto que es un estilo habitual entre los diseñadores de objetos con experiencia —quizás *De IDs a Objetos*—. ¿Por qué preocuparse? Porque tener un auténtico objeto *Cliente* que encapsula un conjunto de información sobre el cliente, y que puede tener comportamiento (relacionado con el Experto en Información, por ejemplo), con frecuencia se vuelve beneficioso y flexible cuando crece el diseño, incluso si el diseñador no creyó que fuera necesario un verdadero objeto y pensó en cambio que un simple número sería suficiente. Nótese que en el diseño inicial, la transformación del *articuloID* en un objeto *EspecificacionDelProducto* es otro ejemplo del patrón *De IDs a Objetos*.

Paso de objetos agregados como parámetros

Finalmente, observe que en el mensaje *añadirEstrategiaFijarPrecioCliente(v:Venta)* pasamos una *Venta* a la factoría, y entonces la factoría se vuelve y pregunta a la *Venta* por el *Cliente* y su *EstrategiaFijarPrecios*.

¿Por qué no extraer exactamente estos dos objetos de la *Venta*, y, en lugar de lo anterior, pasar a la factoría el *Cliente* y la *EstrategiaFijarPrecios*? La respuesta es otro estilo del diseño de objetos típico: evitar extraer los objetos hijos de un objeto padre o agregado, y entonces pasar los objetos hijos. Mejor, pase el objeto agregado que contiene los hijos.

Si se sigue este principio se incrementa la flexibilidad, porque entonces la factoría puede colaborar con la *Venta* completa de algunas maneras que no habíamos anticipado que fueran necesarias previamente (que es muy normal), y como corolario, reduce la necesidad de anticipar lo que necesita el objeto factoría; el diseñador pasa exactamente la *Venta* completa, sin conocer los objetos más específicos que podría necesitar la factoría. Aunque este estilo no tiene nombre, está relacionado con el Bajo Acoplamiento y Variaciones Protegidas. Quizás podría llamarse patrón *Paso de Objetos Agregados como Parámetro*.

Resumen

De este problema de diseño se han extraído muchos consejos de diseño de objetos. Un diseñador de objetos experto ha incorporado a su memoria muchos de estos patrones estudiando las explicaciones que se han publicado, y ha asimilado los principios fundamentales, como los que se describen en la familia GRASP.

Por favor, obsérvese que aunque esta explicación del Composite se ha realizado para una familia de Estrategias, el patrón Composite se puede aplicar a otros tipos de objetos, no sólo estrategias. Por ejemplo, es habitual que se creen “macro commands” —commands que contienen otros commands— mediante el uso del Composite. El patrón Command se describirá en un capítulo posterior.

Ejercicios Propuestos

Ejercicio 1

La compra de un producto específico da lugar a un nuevo descuento en la venta completa. Por ejemplo, si compró té Darjeeling, el 15% de descuento en el total de la venta.

Ejercicio 2

Todas las políticas de fijación de precios que se han considerado hasta el momento se aplican sobre el total de la venta, se denominan algunas veces descuentos a nivel de transacción. Pero el reto de diseño más interesante es gestionar los descuentos de las líneas de venta. Por ejemplo:

- Comprando dos trajes, obtenga uno gratis.
- Comprando tres ordenadores X, obtenga un descuento del 50% en la impresora Y.

¿Hay una forma elegante de diseñar esto con los objetos Estrategia?

Patrones Relacionados

El patrón Composite se utiliza normalmente junto con los patrones Estrategia y Command. El Composite se basa en el Polimorfismo y proporciona Variaciones Protegidas a los clientes de manera que no les afecta si el objeto con el que se relacionan es atómico o compuesto.

23.8. Fachada (GoF)

Otro requisito que se ha elegido para esta iteración es dar soporte a *reglas de negocio conectables*. Es decir, en puntos predecibles de los escenarios, como cuando tienen lugar *crearNuevaVenta* e *introducirArticulo* en el caso de uso *Procesar Venta*, o cuando un cajero comienza a vender, a distintos clientes que deseen comprar el PDV NuevaEra les gustaría adaptar ligeramente su comportamiento.

Siendo más precisos, asuma que se desea que las reglas puedan invalidar una acción. Por ejemplo:

- Suponga que cuando se crea una nueva venta, es posible identificar que se pagará mediante un vale-regalo (lo que es posible y habitual). Entonces, una tienda podría tener una regla para permitir que sólo se compre un único artículo si se utiliza un vale-regalo. En consecuencia, deberían invalidarse las operaciones *introducirArticulo* que sigan a la primera.
- Si se paga la venta mediante un vale-regalo, se invalidan todos los tipos de devoluciones de dinero al cliente excepto otro vale-regalo. Por ejemplo, si el cajero solicita el cambio en dinero en efectivo o como crédito para la cuenta del cliente en la tienda, se deben invalidar estas peticiones.
- Suponga que cuando se crea una nueva venta es posible identificar que es una donación benéfica (de la tienda a una ONG). La tienda podría tener una regla que sólo permitiera la entrada de artículos de valor menor a 250 € cada uno, y también añadir sólo artículos a la venta si el “cajero” que inició la sesión es un encargado.

En cuanto al análisis de requisitos, se deben identificar los puntos concretos del escenario en todos los casos de uso (*introducirArticulo*, *elegirCambioEnEfectivo*,...). Para este estudio, sólo se considerará el punto *introducirArticulo*, pero se puede aplicar la misma solución a todos los puntos.

Suponga que el arquitecto software quiere un diseño que afecte poco a los componentes software que ya existen. Es decir, quiere diseñar separando los intereses, y factorizar esta regla en un interés separado. Es más, suponga que el arquitecto no está seguro de cuál es la mejor implementación para gestionar esta regla conectable, y podría querer experimentar con diferentes soluciones para representar, cargar y evaluar las reglas. Por ejemplo, las reglas se pueden implementar siguiendo el patrón Estrategia, o con intérpretes de reglas de libre distribución que leen e interpretan un conjunto de reglas IF-THEN, o con intérpretes de reglas comerciales, que hay que comprar, entre otras soluciones.

Para solucionar este problema de diseño, se puede utilizar el patrón Fachada.

Fachada (Facade)

Contexto/Problema

Se requiere una interfaz común, unificada para un conjunto de implementaciones o interfaces dispares —como en un subsistema—. Podría no ser conveniente acoplarla con muchas cosas del subsistema, o la implementación del subsistema podría cambiar. ¿Qué hacemos?

Solución

Defina un único punto de conexión con el subsistema —un objeto fachada que envuelve al subsistema—. Este objeto fachada presenta una única interfaz unificada y es responsable de colaborar con los componentes del subsistema.

Una Fachada es un objeto “front-end” que es el único punto de entrada para los servicios de un subsistema⁶; la implementación y otros componentes del subsistema son privados y no pueden verlos los componentes externos. La Fachada proporciona Variaciones Protegidas frente a los cambios en las implementaciones de un subsistema.

Por ejemplo, definiremos un subsistema “motor de reglas”, cuya implementación específica no se conoce todavía. Será responsable de evaluar un conjunto de reglas contra una operación (mediante alguna implementación oculta), e indicar entonces si alguna de las reglas invalida la operación.

El objeto fachada de este subsistema se llamará *FachadaMotorReglasPDV*. El diseñador decide colocar las llamadas a esta fachada cerca del comienzo de los métodos que se han definido como los puntos para las reglas conectables, como en este ejemplo:

```
public class Venta
{

    public void crearLineaDeVenta( EspecificacionDelProducto espec, int
        cantidad)
    {
        LineaDeVenta ldv = new LineaDeVenta(espec, cantidad);

        //llamada a la fachada
        if (FachadaMotorReglasPDV.getInstancia().esInvalido (ldv,this))
            return;

        lineasDeVenta.add(ldv);
    }

    //...

} //final de la clase
```

Véase el uso del patrón Singleton. Normalmente se accede a la Fachada por medio del Singleton.

Con este diseño, la complejidad y la implementación del modo en el que se representarán y evaluarán las reglas se oculta en el subsistema del “motor de reglas”, al que se accede por medio de la fachada *FachadaMotorReglasPDV*. Obsérvese que el subsistema que oculta el objeto fachada podría contener docenas o cientos de clases de objetos, o incluso una solución no orientada a objetos, únicamente como cliente del subsistema sólo vemos su único punto de acceso público.

Y se ha conseguido una separación de intereses en cierta medida — se han delegado en otro subsistema todas las cuestiones de manejo de reglas—.

Resumen

El patrón Fachada es sencillo y se utiliza ampliamente. Oculta un subsistema detrás de un objeto.

⁶ El término “subsistema” se está utilizando en un sentido informal para designar a un grupo de componentes relacionados, no exactamente como se define en UML.

Ejercicios Propuestos

Ejercicio 1

Diseñe la gestión de las reglas con el patrón Estrategia, cuyos nombres de clase se obtienen dinámicamente leyendo de una fuente externa.

Ejercicio 2

Si se implementa en Java, diseñe la gestión de las reglas con Jess, un intérprete de reglas de libre distribución para uso con fines académicos disponible en <http://herzberg.ca.sandia.gov/jess/>

Patrones Relacionados

Normalmente se accede a las fachadas por medio del patrón Singleton. Los dos proporcionan Variaciones Protegidas de la implementación de un subsistema, añadiendo un objeto de Indirección que ayuda a mantener Bajo Acoplamiento. Los objetos externos se acoplan a un punto del subsistema: el objeto fachada.

Como se describe en el patrón Adaptador, un objeto adaptador puede utilizarse para envolver el acceso a sistemas externos que tienen interfaces diferentes. Esto es una clase de fachada pero el énfasis está en proporcionar adaptación a interfaces diferentes, y por ello se llama más específicamente un adaptador.

Notación UML: UML proporciona una notación para agrupaciones de propósito general denominadas **paquetes**, cuyo icono es un tipo de carpeta etiquetada. Los paquetes se pueden utilizar para mostrar agrupaciones lógicas de objetos; se podría corresponder con algo como los paquetes de Java o los *namespaces* de C++, o con otros componentes agregados o subsistemas lógicamente distintos. Nótese que en la Figura 23.19 sólo la *FachadaMotorReglasPDV* es pública con respecto a su paquete.

Existe notación UML más compleja para representar subsistemas, pero la notación de la Figura 23.19 será suficiente por ahora. El diseño con paquetes se estudiará con más detalle en la siguiente iteración.

23.9. Observador/Publicar-Suscribir/Modelo de Delegación de Eventos (GoF)

Otro requisito de la iteración es añadir la capacidad de que una ventana GUI actualice la información que muestra sobre el total de la venta cuando éste cambia (ver Figura 23.20). La idea es solucionar el problema para este caso particular, y después en iteraciones posteriores, extender la solución para actualizar la información de la GUI también para los cambios de otros datos.

¿Por qué no hacer lo siguiente como solución? Cuando la *Venta* cambia su total, el objeto *Venta* envía un mensaje a la ventana, pidiéndole que actualice la información que muestra.

Recordemos que el principio de Separación Modelo-Vista disuade de tales soluciones. Establece que los objetos del “modelo” (objetos que no pertenecen al UI como la *Venta*) no deberían conocer los objetos de la vista o presentación tales como una ventana. Dicho principio fomenta Bajo Acoplamiento entre los objetos de otras capas y los de la capa de presentación (UI).

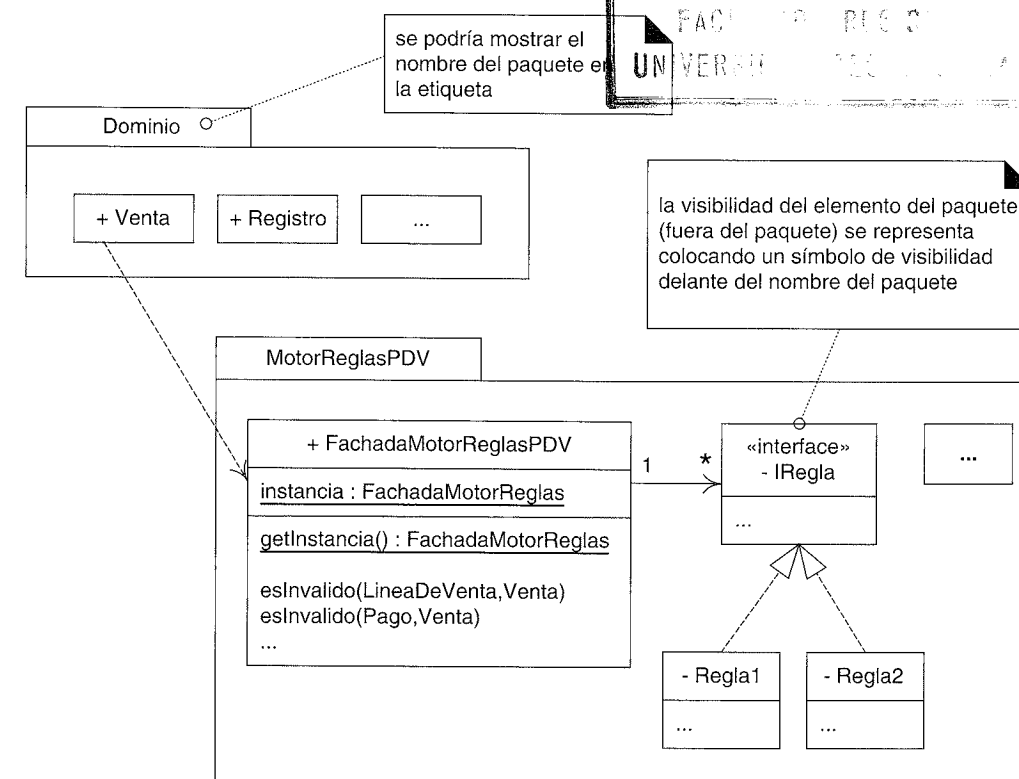


Figura 23.19. Notación de paquetes UML.

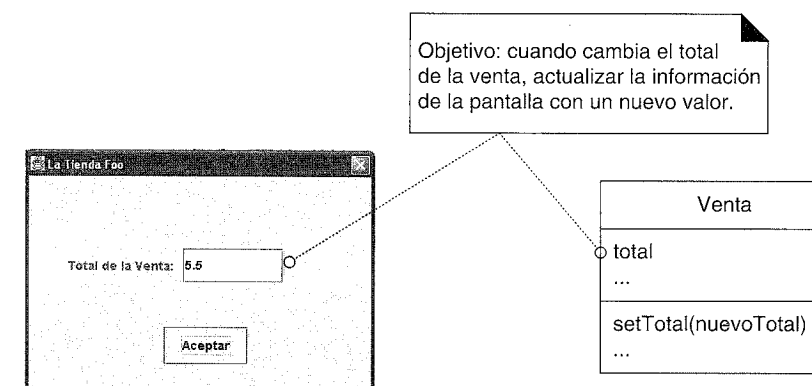
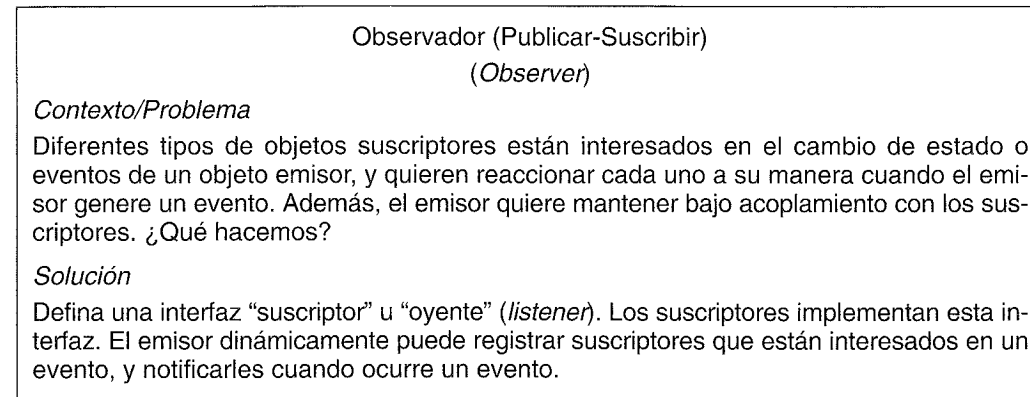


Figura 23.20. Actualización de la interfaz cuando cambia el total de la venta.

Una consecuencia de mantener este bajo acoplamiento es que permite reemplazar la vista o capa de presentación por una nueva, o una ventana concreta por otra nueva, sin afectar a los objetos que no pertenecen a la UI. Si los objetos del modelo no conocen los objetos Swing de Java (por ejemplo), entonces es posible desconectar una interfaz Swing, o desconectar una ventana concreta, y conectar algo más.

Por tanto, la Separación Modelo-Vista mantiene Variaciones Protegidas con respecto a los cambios en la interfaz de usuario.

Para solucionar este problema de diseño se puede utilizar el patrón Observador.



Una solución de ejemplo se describe en detalle en la Figura 23.21.

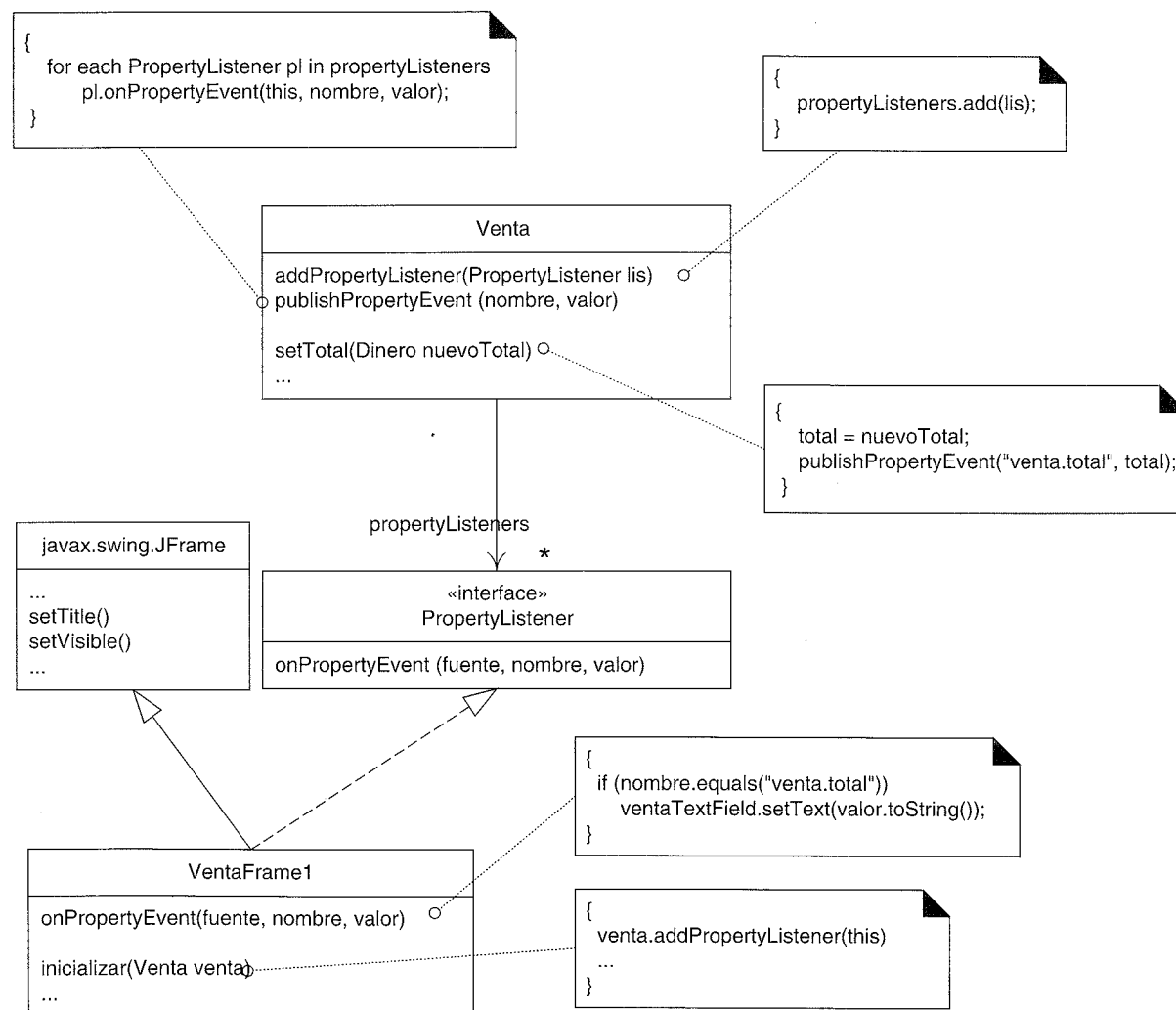


Figura 23.21. El patrón Observador.

Las ideas y pasos fundamentales de este ejemplo son:

1. Se define una interfaz; en este caso *PropertyListener*⁷ con la operación *onPropertyEvent*.
2. Se define la ventana que implementa la interfaz.
 - *VentaFrame1* implementará el método *onPropertyEvent*.
3. Cuando se inicializa la ventana *VentaFrame1*, se le pasa la instancia de *Venta* de la que está mostrando el total.
4. La ventana *VentaFrame1* se registra o suscribe a la instancia de *Venta* para que le notifique acerca de los “eventos sobre la propiedad”, por medio del mensaje *addPropertyListener*. Es decir, cuando una propiedad (como el total) cambia, la ventana quiere que se le notifique.
5. Observe que la *Venta* no conoce los objetos *VentaFrame1*; más bien sólo conoce los objetos que implementan la interfaz *PropertyListener*. Esto disminuye el acoplamiento entre la *Venta* y la ventana —se acopla sólo con una interfaz, no con la clase de la GUI—.
6. Por tanto, la instancia de *Venta* es un *emisor* de “eventos sobre la propiedad”. Cuando cambia el total, itera sobre todos los objetos *PropertyListener* que están suscritos, y se lo notifica a cada uno.

Notación UML: Obsérvese en la Figura 23.21 que a los métodos interesantes se les añaden notas de comentario que muestran la implementación. Estas notas añaden información sobre el comportamiento dinámico a un diagrama de tipos estáticos. En algunos casos, un diagrama de clases con estas notas puede sustituir la necesidad de diagramas de interacción adicionales. Esto no significa que estemos aconsejando que se eviten los diagramas de interacción, sino que indica enfoques de notación alternativos.

El objeto *VentaFrame1* es el observador/suscriptor/oyente. En la Figura 23.22, *suscribe* su interés en los eventos sobre las propiedades de la *Venta*, que es un *emisor* de

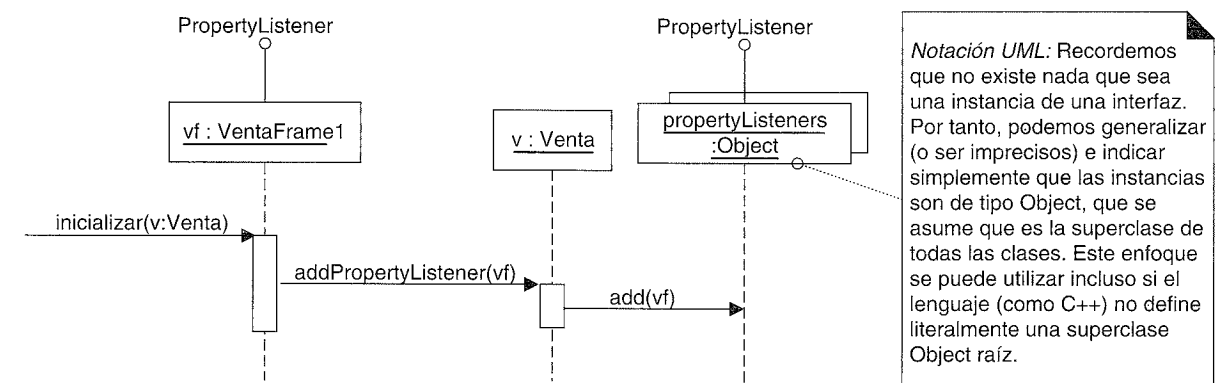


Figura 23.22. El observador *VentaFrame1* se suscribe al emisor *Venta*.

⁷ N. del T.: No se han traducido los nombres de los métodos en los ejemplos de esta sección ya que corresponden a la implementación en Java del patrón.

eventos sobre la propiedad. La *Venta* añade el objeto a su lista de suscriptores de tipo *PropertyListener*. Nótese que la *Venta* no conoce a *VentaFrame1* como un objeto *VentaFrame1* sino como un objeto *PropertyListener*; esto disminuye el acoplamiento entre la capa del modelo y la vista.

Como se ilustra en la Figura 23.23, cuando cambia el total de la *Venta*, itera sobre todos los suscriptores que se han registrado, y “publica un evento” enviando a cada uno el mensaje *onPropertyEvent*.

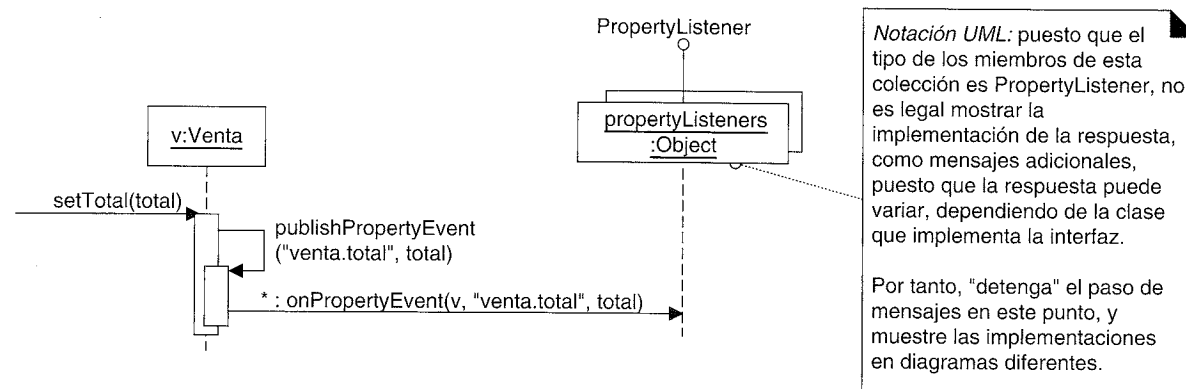


Figura 23.23. La *Venta* publica un evento sobre la propiedad a todos sus suscriptores.

Notación UML: Observe el enfoque para gestionar los mensajes polimórficos en un diagrama de interacción, en la Figura 23.23.

VentaFrame1, que implementa la interfaz *PropertyListener*, por tanto implementa el método *onPropertyEvent*. Cuando el *VentaFrame1* recibe el mensaje, envía un mensaje a su objeto *TextField* que es un elemento gráfico de la GUI para actualizar el nuevo total de la venta. Ver Figura 23.24.

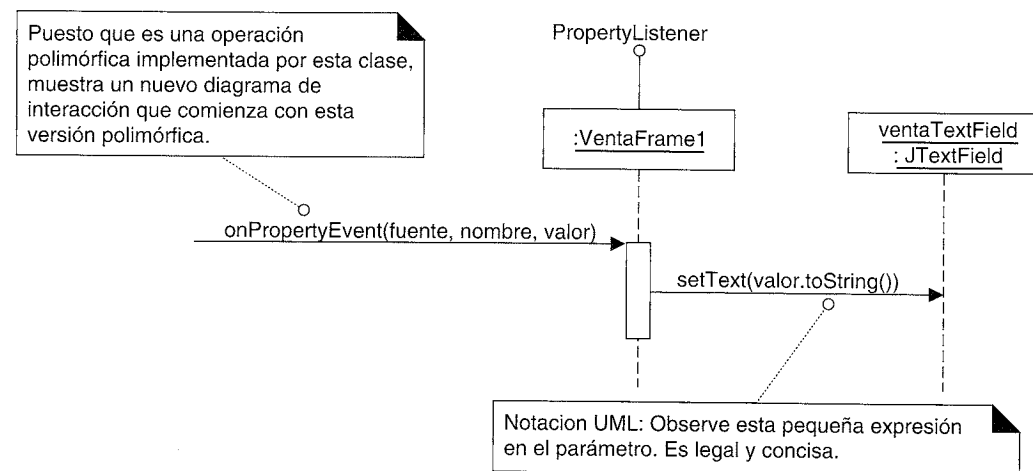


Figura 23.24. El suscriptor *VentaFrame1* recibe la notificación de un evento publicado.

En este patrón, existe todavía algo de acoplamiento entre el objeto del modelo (la *Venta*) y el objeto de la vista (el *VentaFrame1*). Pero se trata de un acoplamiento débil con

una interfaz independiente de la capa de presentación —la interfaz *PropertyListener*—. Y el diseño realmente no necesita que se registre ningún objeto suscriptor en el emisor (ningún objeto tiene que estar escuchando). Es decir, la lista de objetos *PropertyListener* que se registran en la *Venta* puede estar vacía. Resumiendo, el acoplamiento con una interfaz genérica de objetos que no necesita estar presente y a la que se le pueden añadir (y eliminar) elementos dinámicamente, mantiene bajo acoplamiento. Por tanto, se ha conseguido Variaciones Protegidas con respecto a los cambios en la interfaz de usuario, mediante el uso de una interfaz y el polimorfismo.

¿Por qué se le denomina Observador, Publicar-Suscribir, o Modelo de Delegación de Eventos?

Originalmente, este estilo se llamó publicar-suscribir, y todavía se le conoce ampliamente por este nombre. Un objeto “publica eventos”, como la *Venta* que publica el “evento sobre la propiedad” cuando cambia el total. Podría pasar que ningún objeto estuviera interesado en este evento, en cuyo caso, la *Venta* no tendría ningún suscriptor registrado. Pero los objetos que están interesados, “suscriben” o registran su interés en un evento pidiéndole al emisor que le notifique. Esto se hizo con el mensaje *Venta--addPropertyListener*. Cuando tiene lugar el evento, se notifica a los suscriptores que están registrados mediante un mensaje.

Se le ha llamado Observador porque el oyente o suscriptor está observando el evento; ese término se hizo popular en Smalltalk a principio de los ochenta.

También se le ha denominado Modelo de Delegación de Eventos (en Java) porque el emisor delega la gestión de los eventos a los “oyentes” (suscriptores; ver Figura 23.25).

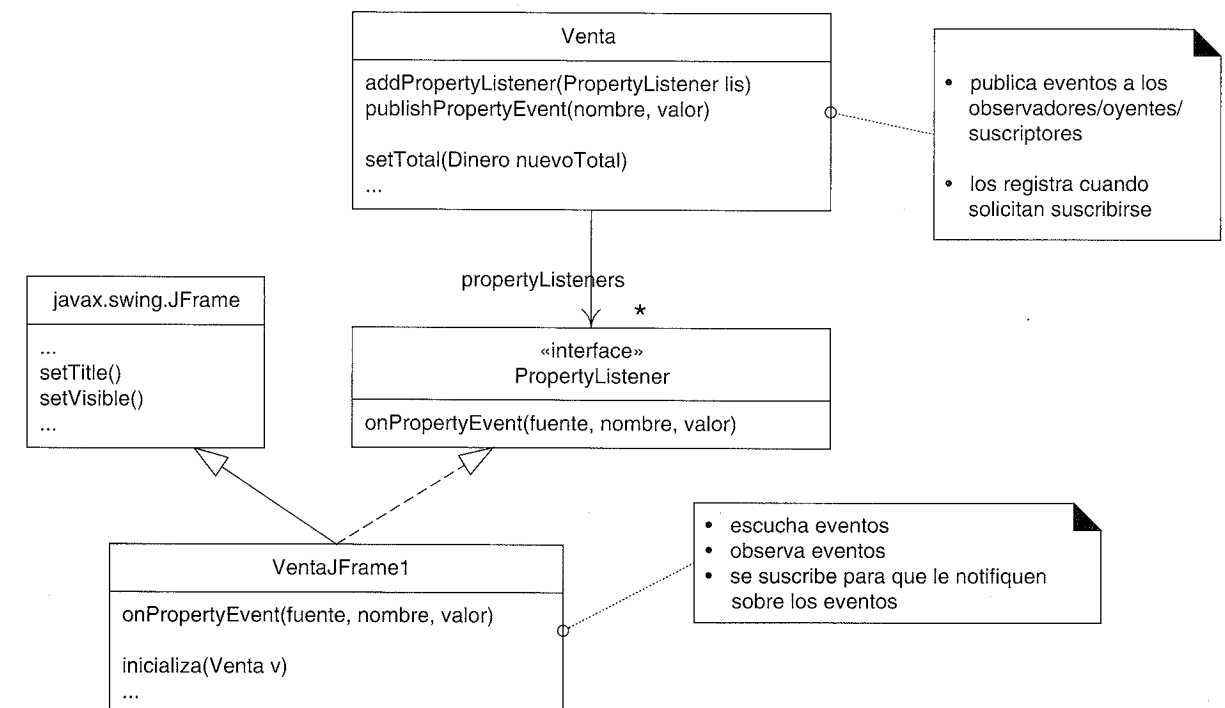


Figura 23.25. ¿Quién es el observador, oyente, suscriptor y emisor?

El Observador no sólo es para conectar UIs y objetos del modelo

El ejemplo anterior ilustraba la conexión con el Observador entre los objetos que no pertenecen a la UI y los objetos de la UI. Sin embargo, existen otros usos habituales.

El uso más común de ese patrón es para el manejo de los eventos de los elementos gráficos de la GUI, tanto en la tecnología Java (AWT y Swing) como en la .NET de Microsoft. Cada elemento gráfico es un emisor de eventos relacionados con la GUI, y otros objetos pueden suscribir su interés en ellos. Por ejemplo, un *JButton* de las Swing publica un “evento de acción” cuando se presiona. Otro objeto se registrará con el botón de manera que cuando se presione, se envía un mensaje al objeto y puede realizar alguna acción.

Otro ejemplo se muestra en la Figura 23.26 donde se ilustra un *RelojAlarma*, que es un emisor de eventos de alarma, y varios suscriptores. Este ejemplo es ilustrativo ya que pone de relieve que la interfaz *AlarmaListener* puede ser implementada por muchas clases, se pueden registrar simultáneamente muchos objetos como oyentes, y cada uno reaccionará ante el “evento de alarma” de manera diferente.

Un emisor puede tener muchos suscriptores a un evento

Como se desprende de la Figura 23.26, una instancia de emisor podría tener de uno a muchos suscriptores registrados. Por ejemplo, una instancia de *RelojAlarma* podría tener registrados tres objetos *VentanaDeAlarma*, cuatro objetos *Busca* y un *GuardianDeFiabilidad*. Cuando ocurre un evento de alarma, se le notifica a los ocho objetos *AlarmaListener* por medio del evento *onAlarmaEvent*.

Implementación

Eventos

Tanto en las implementaciones del Observador en Java como en C# .NET, se comunica un “evento” por medio de un mensaje ordinario, como *onPropertyEvent*. Además, en ambos casos, el evento se define más formalmente como una clase, y se rellena con los datos del evento apropiados. Entonces el evento se pasa como parámetro en el mensaje de evento.

Por ejemplo:

```
class PropertyEvent extends Event
{
    private Object fuenteDelEvento;
    private String nombrePropiedad;
    private Object valorAntiguo;
    private Object valorNuevo;
    //...
}

//...
```

```
class Venta
{
    private void publishPropertyEvent(
        String nombre, Object antiguo, Object nuevo)
    {
        PropertyEvent evt =
            new PropertyEvent (this, "venta.total", antiguo, nuevo);

        for each AlarmaListener al in alarmaListeners
            al.onPropertyEvent(evt);
    }

    //...
}
```

Java

Cuando se lanzó el JDK 1.0 en enero de 1996, contenía una implementación simple de publicar-suscribir basada en una clase y una interfaz denominadas *Observable* y

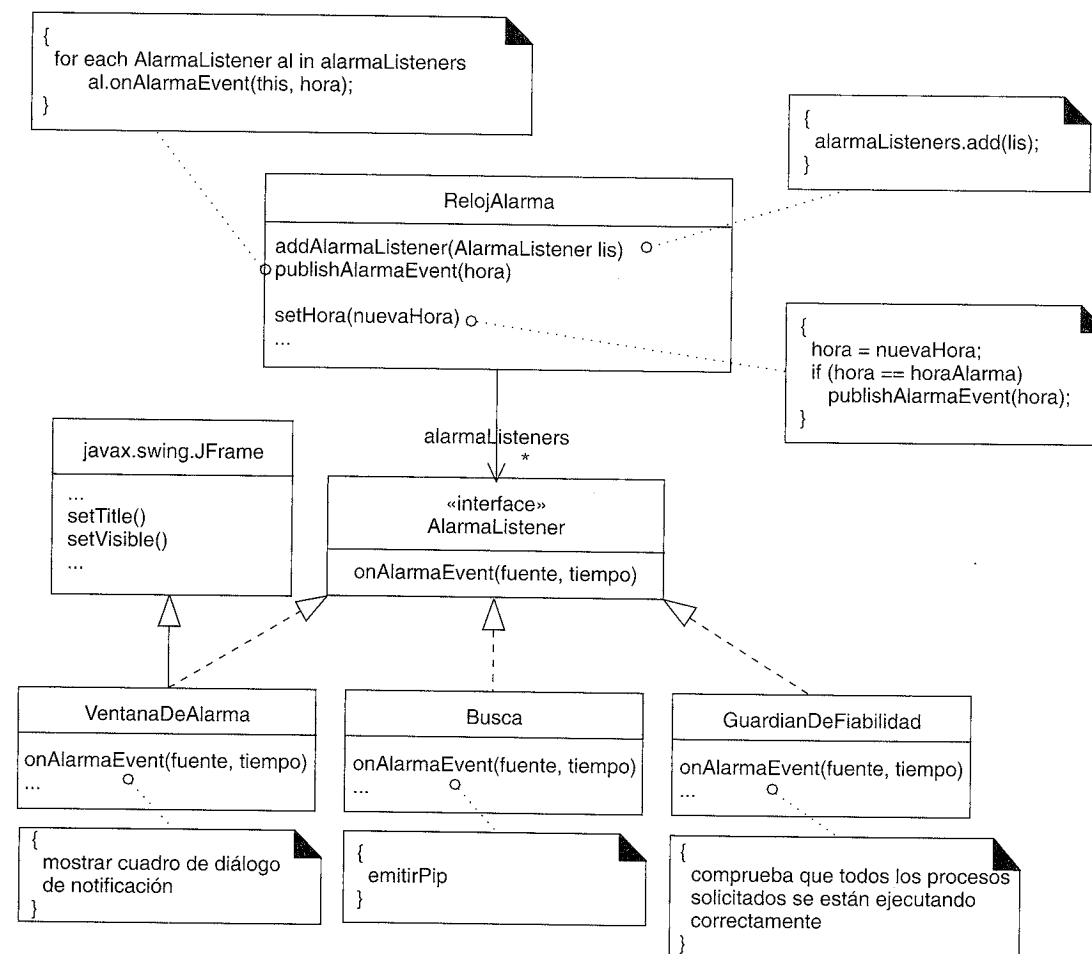


Figura 23.26. El Observador aplicado a los eventos de la alarma, con diferentes suscriptores.

Observer, respectivamente. Esto se copió básicamente, sin ninguna mejora, del enfoque de primeros de los ochenta de la implementación de publicar-suscribir en Smalltalk.

Por tanto, a finales de 1996, dentro de la versión JDK 1.1, se sustituyó el diseño Observable-Observer por el Modelo de Delegación de Eventos de Java (DEM, *Delegation Event Model*), una versión más robusta de publicar-suscribir, aunque se mantuvo el diseño original por compatibilidad con las versiones anteriores (pero en general debe evitarse).

Los diseños descritos en este capítulo son consistentes con el DEM, pero se han simplificado ligeramente para resaltar las ideas fundamentales.

Resumen

El Observador proporciona un modo de acoplar débilmente los objetos en términos de la comunicación. Los emisores conocen a los suscriptores sólo a través de una interfaz, y los suscriptores pueden registrarse (o darse de baja) de los emisores dinámicamente.

Patrones Relacionados

El Observador se basa en el Polimorfismo, y proporciona Variaciones Protegidas en cuanto a que protege al emisor del conocimiento de la clase específica de objetos, y número de objetos, con los que se comunica cuando genera un evento.

23.10. Conclusión

La lección principal que se puede extraer de esta exposición es que se pueden diseñar los objetos y asignar las responsabilidades con la ayuda de los patrones. Esto proporciona un conjunto explicable de estilos mediante los cuales se pueden construir sistemas orientados a objetos bien diseñados.

23.11. Lecturas adicionales

El libro de texto básico es *Design Patterns* de Gamma, Helm, Johnson y Vlissides, y es fundamental que lo lean todos los diseñadores de objetos.

Cada año se celebra un congreso "Pattern Languages of Programs" (PLOP, Lenguajes de Patrones de Programas) a partir del cual se publica una recopilación de patrones, en la serie *Pattern Languages of Program Design*, volumen 1, 2, etcétera. Se recomienda toda la serie.

Pattern-Oriented Software Architecture, volumen 1 y 2, promueve la discusión sobre los patrones en cuestiones de arquitectura a gran escala. El volumen 1 presenta una taxonomía de patrones.

Hay publicados cientos de patrones. Rising resume un porcentaje respetable de ellos en *The Pattern Almanac*.

PARTE 5 ELABORACIÓN EN LA ITERACIÓN 3

LA ITERACIÓN 3 Y SUS REQUISITOS

24.1. Requisitos de la Iteración 3

- Cuando no se puede acceder a los servicios remotos proporcionar el mantenimiento de los servicios ante los fallos mediante servicios locales. Por ejemplo, si no se puede acceder al producto remoto de la base de datos, utilice una versión local de datos almacenados en la caché.
- Proporcionar el soporte para el manejo de los dispositivos de PDV, como el cajón de caja y el dispensador de monedas.
- Manejar las autorizaciones de pago a crédito.
- Soporte para objetos persistentes.

24.2. Énfasis de la Iteración 3

Durante la fase de inicio y la Iteración 1 se exploraron una variedad de cuestiones fundamentales en el análisis de requisitos y el A/DOO. La Iteración 2 se ocupó en detalle del diseño de objetos. Esta tercera iteración considera de nuevo una perspectiva más amplia, explorando una extensa variedad de temas relacionados con el análisis y diseño, entre los que se encuentran:

- Relaciones entre casos de uso.
- Generalización y especialización.
- Modelado de estados.
- Arquitecturas en capas.
- El diseño de paquetes.
- Análisis arquitectural.
- Más patrones de diseño GoF.
- El diseño de *frameworks*, en particular un framework de persistencia.

RELACIONES ENTRE CASOS DE USO

¿Por qué los programadores confunden Halloween y las navidades? Porque OCT(31)=DEC(25).

Objetivos

- Relacionar los casos de uso mediante las asociaciones *include* y *extend*.
-

Introducción

Los casos de uso se pueden relacionar entre ellos. Por ejemplo, un caso de uso de subfunción como *Gestionar Pagos a Crédito* podría formar parte de varios casos de uso ordinarios, como *Procesar Venta* o *Procesar Alquiler*. La organización de los casos de uso mediante relaciones no influye en el comportamiento o los requisitos del sistema. Más bien, es simplemente una forma de organizar para (idealmente) mejorar la comunicación y la comprensión de los casos de uso, reducir la duplicación de texto y mejorar la gestión de los documentos de casos de uso.

Una advertencia

En algunas organizaciones que trabajan con casos de uso, se dedica mucho tiempo improductivo a debatir sobre cómo relacionar los casos de uso en los diagramas de casos de uso, en lugar de dedicarlo al trabajo importante de los casos de uso: escribir texto. En consecuencia, aunque este capítulo presenta las relaciones entre los casos de uso, el tema y la dedicación que merece deberían ponerse en perspectiva: tiene algo de valor, pero el trabajo importante es escribir el texto de los casos de uso. La especificación de los requisitos se hace escribiendo, no organizando los casos de uso, que es un paso opcional, posiblemente para ayudar a comprenderlos o reducir duplicaciones. Si un equipo co-

mienza el modelado de los casos de uso dedicando horas (o peor, días) discutiendo un diagrama de caso de uso y las relaciones entre los casos de uso (“¿Ésa debería ser una relación *include* o *extend*? ¿Deberíamos especializar este caso de uso?”), en lugar de centrarse en escribir rápidamente el texto clave del caso de uso, se está haciendo un esfuerzo significativo en el sitio equivocado.

Es más, la organización de los casos de uso utilizando las relaciones puede evolucionar iterativamente en pequeñas etapas a lo largo de la fase de elaboración; no es útil intentar aplicar un esfuerzo del estilo del modelo en cascada en el que se define y refina totalmente un diagrama completo de caso de uso y el conjunto de relaciones en una etapa al comienzo del proyecto.

25.1. La relación de inclusión (*include*)

Ésta es la relación más común e importante.

Es habitual tener algún comportamiento parcial común a varios casos de uso. Por ejemplo, la descripción del pago a crédito tiene lugar en varios casos de uso, entre los que se encuentran *Procesar Venta*, *Procesar Alquiler*, *Contribuir a Plan de Ahorro*, etcétera. En lugar de duplicar este texto, es conveniente separarlo en su propio caso de uso de subfunción, e indicar su inclusión. Esto es sencillamente factorizar y enlazar texto para evitar duplicaciones¹.

Por ejemplo:

UC1: Procesar Venta

...
Escenario principal de éxito:
1. El Cliente llega a un terminal PDV con mercancías y/o servicios para comprar.
...
7. El Cliente paga y el Sistema gestiona el pago.
...
Extensiones:
7b. Pago a crédito: Incluye Gestionar Pago a Crédito.
7c. Pago con cheque: Incluye Gestionar Pago con Cheque.
...

UC7: Procesar Alquiler

...
Extensiones:
6b. Pago a crédito: Incluye Gestionar Pago a Crédito.
...

¹ Sirve de ayuda que los enlaces se implementen también con hipervínculos navegables.

UC12: Gestionar Pago a Crédito

...
Nivel: Subfunción.
Escenario principal de éxito:
1. El Cliente introduce la información acerca de su cuenta de crédito.
2. El Sistema envía la solicitud de autorización del pago a un Sistema para el Servicio de Autorización de Pago, y solicita la aprobación del pago.
3. El Sistema recibe la aprobación del pago y la notifica al Cajero.
4. ...
Extensiones:
2a. El sistema detecta algún fallo al colaborar con el sistema externo:
1. El Sistema informa del error al Cajero.
2. El Cajero le pide al Cliente un modo de pago alternativo.
...

Ésta es la relación **de inclusión** (*include*).

Una notación algo más breve (y, por tanto, quizás se prefiera) para indicar el caso de uso que se incluye es simplemente subrayarlo o destacarlo de alguna manera. Por ejemplo:

UC1: Procesar Venta

...
Extensiones:
7b. Pago a crédito: Gestionar Pago a Crédito.
7c. Pago con cheque: Gestionar Pago con Cheque.

Nótese que el caso de uso de subfunción *Gestionar Pago a Crédito* se encontraba originalmente en la sección de *Extensiones* del caso de uso *Procesar Venta*, pero se factorizó aparte para evitar duplicaciones. También observe que en el caso de uso de subfunción se utilizan las mismas estructuras *Escenario principal de éxito* y *Extensiones* que en los casos de uso de procesos de negocio ordinarios como *Procesar Venta*.

Fowler ofrece una guía sencilla y práctica sobre cuándo utilizar la relación de inclusión [FS00]:

Utilice *include* cuando se está repitiendo en dos o más casos de usos separados y quiere evitar repeticiones.

Otro motivo es simplemente descomponer un caso de uso abrumadoramente largo en subunidades para mejorar la comprensión.

Utilización de include con el manejo de eventos asíncronos

La relación de inclusión también se utiliza para describir el manejo de un evento asíncrono, como cuando un usuario es capaz de seleccionar o bifurcar, en cualquier momento, a una ventana, función o página web específica, o en un rango de pasos.

De hecho, la notación de los casos de uso para representar esta bifurcación asíncrona ya se estudió en la introducción a los casos de uso en el Capítulo 6, pero en ese momento no se presentó la incorporación de una invocación a un subcaso de uso que se incluye.

La notación básica es utilizar etiquetas siguiendo el estilo *a**, *b**... en la sección de Extensiones. Recordemos que esto implica una extensión o evento que puede ocurrir en cualquier momento. Una variación menor es utilizar una etiqueta con un rango, como 3-9, cuando el evento asíncrono puede ocurrir en un rango relativamente amplio de los pasos del caso de uso, pero no todos.

UC1: Procesar Algo

...
Escenario principal de éxito:
1. ...
Extensiones:
a*. En cualquier momento, el Cliente selecciona la edición de la información personal: *Editar Información Personal*
b*. En cualquier momento, el Cliente selecciona imprimir la ayuda: *Presentar Ayuda Impresa*
2-11. El Cliente cancela: *Cancelar Confirmación de Transacción*
...

Resumen

La relación de inclusión se puede utilizar para la mayoría de problemas de relaciones de casos de uso. En resumen:

Factorice casos de uso de subfunción separados y utilice la relación *include* cuando:

- Están duplicados en otros casos de uso.
- Un caso de uso es *muy* complejo y largo, y separarlos en subunidades facilita la comprensión.

Como explicaremos, existen otras relaciones: extensión y generalización. Pero Cockburn, un modelador de casos de uso experto, aconseja escoger la relación de inclusión por encima de las relaciones de extensión y generalización:

Como primera regla empírica, utilice siempre la relación *include* entre los casos de uso. La gente que sigue esta regla declara que existen menos malentendidos entre ellos y sus lectores sobre lo que escriben que la gente que mezcla *include* con *extend* y *generalizes* [Cockburn01].

25.2. Terminología: casos de uso concretos, abstractos, base y adicional

Un **caso de uso concreto** es iniciado por un actor y lleva a cabo todo el comportamiento que desea el actor [RUP]. Éstos son los casos de uso de los procesos del negocio ele-

mentales. Por ejemplo, *Procesar Venta* es un caso de uso concreto. En cambio, un **caso de uso abstracto** nunca se instancia por él mismo; es un caso de uso de subfunción que forma parte de otro caso de uso. *Gestionar Pago a Crédito* es abstracto; no se mantiene independiente, sino que siempre forma parte de otra historia, como *Procesar Venta*.

Un caso de uso que incluye otro caso de uso, o que es extendido o especializado por otro caso de uso se denomina **caso de uso base**. *Procesar Venta* es un caso de uso base con respecto al caso de uso de subfunción *Gestionar Pago a Crédito* que incluye. Por otro lado, el caso de uso que es una inclusión, extensión o especialización se denomina **caso de uso adicional**. *Gestionar Pago a Crédito* es el caso de uso adicional en la relación de inclusión de *Procesar Venta*. Los casos de uso adicionales normalmente son abstractos. Los casos de uso base generalmente son concretos.

25.3. La relación de extensión (*extend*)

Suponga que el texto de un caso de uso no debiera modificarse (al menos no de manera significativa) por alguna razón. Quizás modificar continuamente el caso de uso con innumerables nuevas extensiones y pasos condicionales es un dolor de cabeza de mantenimiento, o se ha establecido el caso de uso como un artefacto estable, y no se puede tocar. ¿Cómo añadir al caso de uso sin modificar su texto original?

La relación **de extensión** (*extend*) proporciona una respuesta. La idea es crear un caso de uso que extiende o añade, y con él, describe dónde y bajo qué condiciones extiende el comportamiento de algún caso de uso base. Por ejemplo:

UC1: Procesar Venta (el caso de uso base)

...
Puntos de Extensión: *Cliente VIP*, paso 1. *Pago*, paso 7.
Escenario principal de éxito:
1. El Cliente llega a un terminal PDV con mercancías y/o servicios para comprar.
...
7. El Cliente paga y el Sistema gestiona el pago.
...

UC15: Gestionar Pago con Vale-Regalo (el caso de uso que extiende)

...
Activa: El Cliente quiere pagar con vale-regalo
Puntos de Extensión: Pago en Procesar Venta
Nivel: Subfunción.
Escenario Principal de Éxito:
1. El Cliente le da el vale-regalo al Cajero.
2. El Cajero introduce el identificador del vale-regalo.
...

Éste es un ejemplo de una relación de **extensión**. Observe el uso de un **punto de extensión**, y que el caso de uso que extiende se activa por alguna condición. Los puntos de extensión son etiquetas en el caso de uso base que extiende referencia como puntos de extensión, de manera que los números de los pasos del caso de uso base pueden cambiar sin afectar al caso de uso que extiende —indirección una vez más—.

Algunas veces, el punto de extensión es simplemente “En cualquier punto del caso de uso X”. Esto es especialmente habitual en sistemas con muchos eventos asíncronos, como un procesador de texto (“Hacer ahora la corrección ortográfica”, “Hacer ahora una búsqueda en el tesoro”), o sistemas de control reactivo. Observe sin embargo, como se describe en la sección anterior de la relación de inclusión, que también se puede utilizar la relación de inclusión para describir el manejo de eventos asíncronos. La alternativa *extend* es una opción cuando el caso de uso base está cerrado a las modificaciones.

Observe que una señal de calidad de la relación de extensión es que el caso de uso base (*Procesar Venta*) no referencia al caso de uso que lo extiende (*Gestionar Pago con Vale-Regalo*) y, por tanto, no define o controla las condiciones bajo las cuales se activan las extensiones. El caso de uso *Procesar Venta* es completo y forma un todo por él mismo, sin conocer los casos de uso que lo extiende.

Fíjese que el caso de uso adicional *Gestionar Pago con Vale-Regalo* de manera alternativa se podría haber referenciado en *Procesar Venta* mediante una relación de inclusión, como con *Gestionar Pago a Crédito*. Con frecuencia es adecuado. Pero este ejemplo estaba motivado por la restricción de que el caso de uso *Procesar Venta* no se iba a modificar, que es la situación en la que se utiliza la extensión en lugar de la inclusión.

Además, nótese que este escenario del vale-regalo se podría haber registrado simplemente añadiéndolo como una extensión en la sección de *Extensiones* de *Procesar Venta*. Este enfoque evita tanto la relación de inclusión como la de extensión, y la creación de un caso de uso de subfunción separado.

De hecho, normalmente es preferible actualizar simplemente la sección de *Extensiones*, en lugar de crear complejas relaciones de casos de uso.

Algunas guías de casos de uso recomiendan la utilización de casos de uso que extienden y la relación de extensión para modelar el comportamiento condicional u opcional del caso de uso base. Esto no es incorrecto, pero no comprende que el comportamiento condicional u opcional se puede registrar simplemente como texto en la sección de *Extensiones* del caso de uso base. La dificultad de utilizar la relación de extensión y más casos de uso no está motivada solamente por el comportamiento opcional.

Lo que motiva esencialmente el uso de la técnica de extensión es cuando no es conveniente por alguna razón modificar el caso de uso base.

25.4. La relación de generalización (*generalize*)

La discusión acerca de la relación de generalización queda fuera del alcance de este libro. Sin embargo, observe que los expertos en casos de uso han estado realizando con éxito el trabajo sobre casos de uso sin esta relación opcional, que añade otro nivel de complejidad a los casos de uso, y todavía no existe un acuerdo entre los que la utilizan

sobre una guía de buenas prácticas de cómo sacar provecho de esta idea. Los consultores sobre casos de uso observan que habitualmente da lugar a complicaciones y que se dedica mucho tiempo improductivo a la inclusión de muchas relaciones de casos de uso.

25.5. Diagramas de casos de uso

La Figura 25.1 ilustra la notación UML para la relación de inclusión, que es la única que se va a utilizar en el caso de estudio, siguiendo el consejo de los expertos en casos de uso de mantener las cosas sencillas y preferir la relación de inclusión.

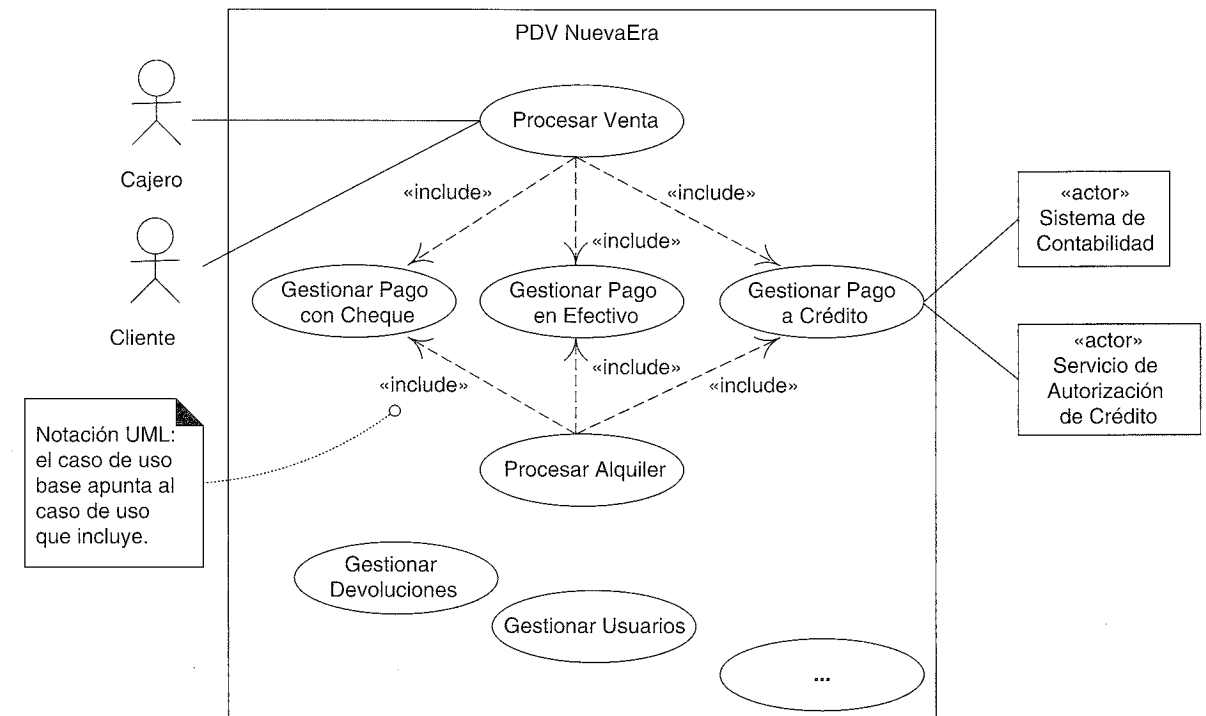


Figura 25.1. Relación de inclusión de los casos de uso en el Modelo de Casos de Uso.

La notación de la relación de extensión se ilustra en la Figura 25.2.

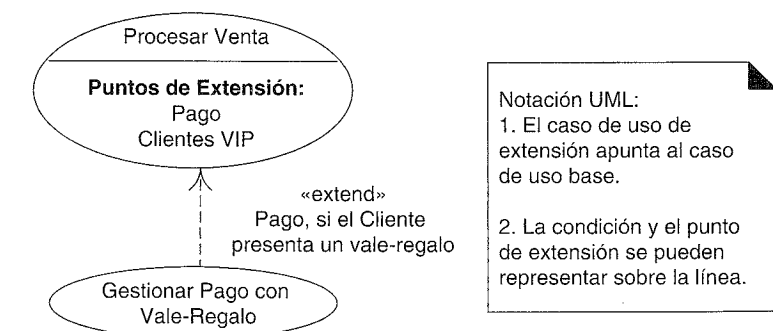


Figura 25.2. La relación de extensión.



MODELADO DE LA GENERALIZACIÓN

Las clasificaciones simplonas y las generalizaciones falsas son las maldiciones de la vida organizada.

Una generalización de H. G. Wells

Objetivos

- Crear jerarquías de generalización-especialización.
 - Identificar cuándo merece la pena mostrar una subclase.
 - Aplicar las pruebas del “100%” y “Es-un” para validar las subclases.
-

Introducción

La generalización y la especialización son conceptos fundamentales en el modelado del dominio que favorece una economía de palabras; más aún, las jerarquías de clases conceptuales a menudo constituyen la fuente de inspiración para las jerarquías de clases software que se aprovechan de la herencia y reducen la duplicación de código.

26.1. Nuevos conceptos para el Modelo del Dominio

Como en la iteración 1, el Modelo del Dominio del UP podría desarrollarse incrementalmente considerando los conceptos en los requisitos para esta iteración. Nos servirán de ayuda técnicas como la *Lista de Categorías de Conceptos* y la identificación de frases nominales. Un enfoque efectivo para desarrollar un modelo del dominio robusto y rico es estudiar el trabajo de otros autores sobre este tema, como [Fowler96]. Muchas de las cuestiones sutiles de modelado que ellos exploran quedan fuera del alcance de este libro.

Lista de Categorías de Conceptos

La Tabla 26.1 muestra algunos conceptos relevantes que se van a tener en cuenta en esta iteración.

Tabla 26.1. Lista de Categorías de Conceptos.

Categoría	Ejemplos
objetos físicos o tangibles	TarjetaDeCredito, Cheque
especificaciones, diseños o descripciones de cosas	
lugares	
transacciones	PagoEnEfectivo, PagoACredito, PagoConCheque
líneas de la transacción	
roles de la gente	
contenedores de otras cosas	
cosas en un contenedor	
otros sistemas informáticos o electro-mecánicos externos a nuestro sistema	ServicioAutorizacionCredito ServicioAutorizacionCheque
conceptos abstractos	
organizaciones	ServicioAutorizacionCredito ServicioAutorizacionCheque
eventos	
reglas y políticas	
catálogos	
registros de cuentas, trabajo, contratos, cuestiones legales	CuentasPorCobrar
instrumentos financieros y servicios	
manuales, libros	

Identificación de frases nominales

Recordemos que la identificación de frases nominales no se puede aplicar mecánicamente para identificar los conceptos relevantes que se van a incluir en el modelo del dominio. Se debe aplicar el sentido común y desarrollar las abstracciones adecuadas, puesto que el lenguaje natural es ambiguo y los conceptos relevantes no siempre se encuentran de manera explícita o clara en el texto existente. Sin embargo, es una técnica práctica en el modelado del dominio puesto que es directa.

Esta iteración maneja los escenarios del caso de uso *Procesar Venta* para los pagos a crédito y con cheque. A continuación se muestran algunas de las frases nominales identificadas a partir de estas extensiones:

Caso de Uso UC1: Procesar Venta

- ...
- Extensiones:**
- 7b. Pago a crédito:
- 1. El Cliente introduce la **información de su cuenta de crédito**.
 - 2. El Sistema envía la **petición de autorización del pago** al Sistema de **Servicio de Autorización de Pago** externo, y solicita la **aprobación del pago**.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 - 1. El Sistema señala el error al Cajero.
 - 2. El Cajero le pide al Cliente un modo de pago alternativo.
 - 3. El Sistema recibe la **aprobación del pago** y lo notifica al Cajero.
 - 3a. El Sistema recibe la **denegación del pago**:
 - 1. El Sistema señala la denegación al Cajero.
 - 2. El Cajero le pide al Cliente un modo de pago alternativo.
 - 4. El Sistema registra el **pago a crédito**, que incluye la aprobación del pago.
 - 5. El Sistema presenta el mecanismo de entrada para la firma del pago a crédito.
 - 6. El Cajero le pide al Cliente que firme el pago a crédito. El Cliente introduce la firma.
 - 7c. Pago con cheque:
 - 1. El Cliente escribe un **cheque**, y lo da junto con su **carnet de conducir** al Cajero.
 - 2. El Cajero escribe el número del carnet de conducir en el cheque, lo introduce, y solicita la **autorización del pago con cheque**.
 - 3. Genera una **solicitud del pago con cheque** y lo envía al **Servicio de Autorización de Cheques**.
 - 4. Recibe la aprobación del pago con cheque y notifica la aprobación al Cajero.
 - 5. El sistema registra el **pago con cheque**, que incluye la aprobación del pago.
- ...

Transacciones del servicio de autorización

La identificación de frases nominales revela conceptos como *SolicitudPagoACredito* y *RespuestaAprobacionCredito*. Éstos en realidad podrían verse como transacciones de los servicios externos, y en general, resulta útil identificar tales transacciones porque las actividades y los procesos tienden a girar alrededor de ellos.

Estas transacciones no tienen que representar registros informáticos o bits que viajan por una línea. Representan la abstracción de la transacción independientemente del modo en el que se ejecute. Por ejemplo, la solicitud de un pago a crédito podría ejecutarse mediante una llamada telefónica o el envío de registros o mensajes entre dos ordenadores, etcétera.

26.2. Generalización

Los conceptos de *PagoEnEfectivo*, *PagoACredito*, y *PagoConCheque* son todos muy parecidos. En esta situación, es posible (y útil ¹) organizarlos (como en la Figura 26.1) en una **jerarquía de clases de generalización-especialización** (o simplemente **jerarquía**

¹ Más tarde, en este capítulo, estudiaremos los motivos para definir jerarquías de clases.

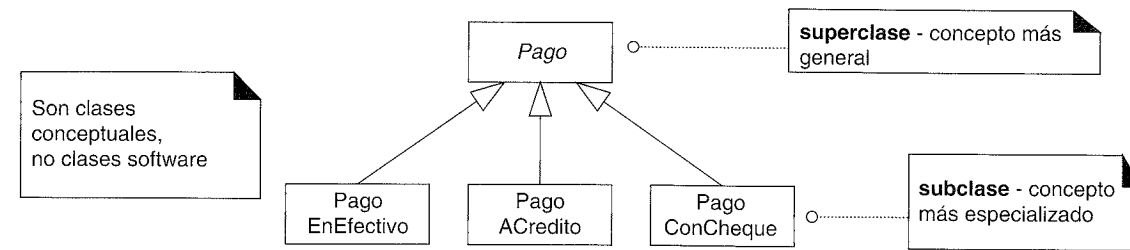


Figura 26.1. Jerarquía de generalización-especialización.

de clases) en la cual, la **superclase** *Pago* representa un concepto más general, y las **subclases** conceptos más especializados.

Nótese que la presentación de las clases en este capítulo se refiere a las clases *conceptuales* y no a las clases *software*.

La **generalización** es la actividad de identificar elementos comunes entre los conceptos y definir las relaciones de superclase (concepto general) y subclase (concepto especializado). Es una forma de construir clasificaciones taxonómicas entre los conceptos que entonces se representan en jerarquías de clases.

La identificación de una superclase y las subclases es útil en un modelo del dominio porque su presencia nos permite entender los conceptos en términos más generales, refinados y abstractos. Esto nos lleva a una economía de expresión, a mejorar la comprensión y a reducir la información repetida. Y aunque ahora nos estamos centrando en el Modelo del Dominio del UP y no en el Modelo de Diseño del software, el diseño e implementación posterior de la superclase y subclases, como clases software que utilizan la herencia, producirán un mejor software.

Por tanto:

Identifique las superclases y subclases del dominio relevantes para el estudio actual, y represéntelas en el Modelo del Dominio.

Notación UML: Revisemos la notación de la generalización que se introdujo en un capítulo anterior, en UML, la relación de generalización entre elementos se representa con una línea con un triángulo hueco grande en el extremo que apunta a la clase más general desde las más especializadas (ver Figura 26.2). Se puede utilizar tanto un estilo con líneas separadas o unidas.

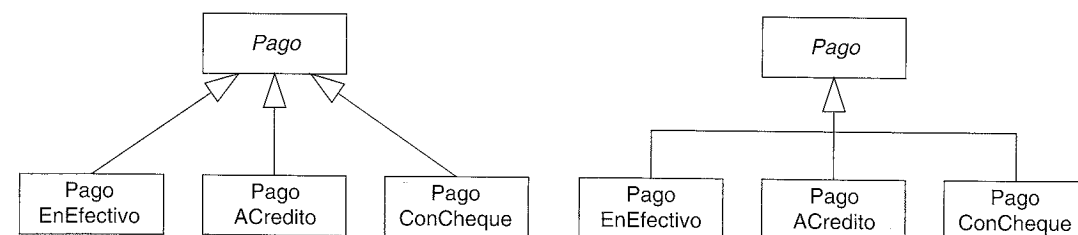


Figura 26.2. Jerarquía de clases con notaciones de líneas separadas o unidas.

26.3. Definición de superclases y subclases conceptuales

Puesto que es útil identificar las super- y subclases conceptuales, es conveniente entender con claridad y precisión la generalización, superclases y subclases en términos de la definición de clase y conjuntos de clases². De esto se ocupan las siguientes secciones.

Generalización y definición de clase conceptual

¿Cuál es la relación de una superclase conceptual con una subclase?

La definición de una superclase conceptual es más general y abarca más que la definición de una subclase.

Por ejemplo, considere la superclase *Pago* y sus subclases (*PagoEnEfectivo*, etcétera). Asuma que la definición del *Pago* representa la transacción de transferir dinero (no necesariamente en efectivo) para una compra desde una parte a otra, y que todos los pagos tienen una cantidad de dinero que es la que transfieren. El modelo que se corresponde a esta definición es el que se muestra en la Figura 26.3.

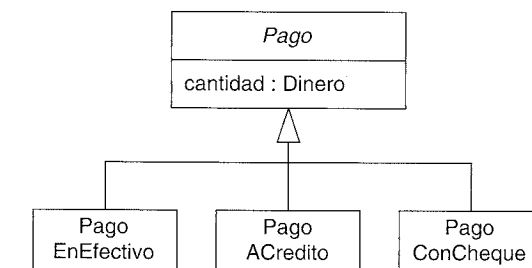


Figura 26.3. Jerarquía de clases del pago.

Un *PagoACredito* es una transferencia de dinero por medio de una institución de crédito que necesita que se autorice. Mi definición de *Pago* abarca más y es más general que mi definición de *PagoACredito*.

Generalización y conjuntos de clases

Las superclases y las subclases conceptuales están relacionadas en términos de pertenencia a un conjunto.

Todos los miembros del conjunto de una subclase conceptual son miembros del conjunto de su superclase.

² Es decir, la intensión y extensión de la clase. Esta discusión está inspirada en [MO95].

Por ejemplo, en términos de la pertenencia a un conjunto, todas las instancias del conjunto de *PagoACredito* también son miembros del conjunto de *Pago*. En un diagrama de Venn, se representa como en la Figura 26.4.

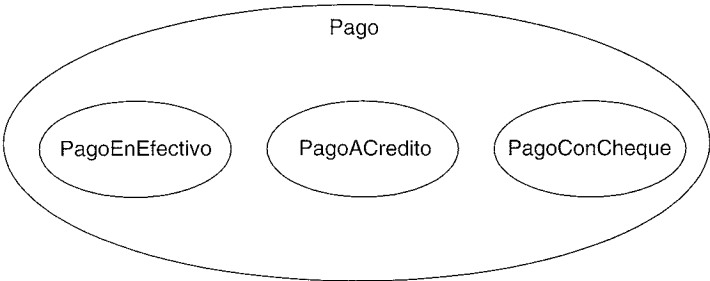


Figura 26.4. Diagrama de Venn de las relaciones de conjuntos.

Conformidad de la definición de la subclase conceptual

Cuando se crea una jerarquía de clases, se hacen declaraciones sobre las superclases que se aplican a las subclases. Por ejemplo, la Figura 26.5 establece que todos los *Pagos* tienen una *cantidad* y que se asocian con una *Venta*.

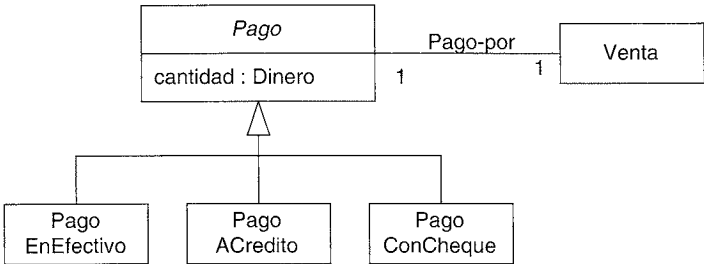


Figura 26.5. Conformidad de las subclases.

Todas las subclases de *Pago* deben ajustarse a tener una cantidad y ser el pago de una *Venta*. En general, esta regla de conformidad con la definición de una superclase es la *Regla del 100%*:

Regla del 100%

El 100% de la definición de la superclase conceptual se debe poder aplicar a la subclase. La subclase debe ajustarse al 100% de los:

- Atributos.
- Asociaciones de la superclase.

Conformidad del conjunto de la subclase conceptual

Una subclase conceptual debería ser un miembro del conjunto de la superclase. Por tanto, un *PagoACredito* debería ser un miembro del conjunto de los *Pagos*.

Informalmente, esto expresa la noción de que la subclase conceptual *es un tipo de* superclase. Un *PagoACredito es un tipo de Pago*. De manera más concisa, *es-un-tipo-de* se denomina *es-un*.

Este tipo de conformidad es la *Regla Es-un*:

Regla Es-un:

Todos los miembros del conjunto de una subclase deben ser miembros del conjunto de su superclase.

En lenguaje natural, esto puede comprobarse informalmente formando la sentencia: Subclase **es una** Superclase.

Por ejemplo, la sentencia *PagoACredito es un Pago* tiene sentido, y transmite la noción de conformidad con la pertenencia a un conjunto.

¿Qué es una subclase conceptual correcta?

A partir de la exposición anterior, aplique las siguientes pruebas³ para definir una subclase correcta cuando construya un modelo del dominio:

Una subclase potencial debería estar de acuerdo con:

- La Regla del 100% (conformidad en la definición).
- La Regla Es-un (conformidad con la pertenencia al conjunto).

26.4 Cuándo definir una clase conceptual

Se han presentado las reglas que aseguran que una subclase es correcta (las reglas del 100% y Es-un). Sin embargo, ¿cuándo deberíamos preocuparnos de definir una subclase? En primer lugar, una definición: Una **partición de clases conceptuales** es una división de las clases conceptuales en subclases disjuntas (o **tipos** según la terminología de Odell) [MO95].

La pregunta se podría volver a enunciarse como:

“¿Cuándo es útil representar una partición de clases conceptuales?”

Por ejemplo, en el dominio del PDV, el *Cliente* podría particionarse correctamente en (o dividirse en las subclases) *ClienteHombre* y *ClienteMujer*. Pero, ¿es relevante o útil mostrar esto en nuestro modelo (ver Figura 26.6)?

Esta partición no es útil para nuestro dominio; la siguiente sección explica por qué.

³ Se han elegido estos nombres de las clases por su ayuda nemotécnica en lugar de por precisión.

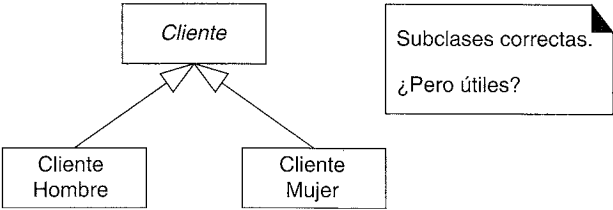


Figura 26.6. Partición de clases conceptuales legales, ¿pero es útil en nuestro dominio?

Razones para particionar una clase conceptual en subclases

A continuación presentamos razones de peso para particionar una clase en subclases:

Cree una subclase conceptual de una superclase cuando:

1.

La subclase tiene atributos adicionales de interés.

2.

La subclase tiene asociaciones adicionales de interés.

3.

El concepto de la subclase funciona, se maneja, reacciona, o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.

4.

El concepto de la subclase representa una cosa animada (por ejemplo, animal o robot) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.

En base al criterio anterior, la partición del *Cliente* en las subclases *ClienteHombre* y *ClienteMujer* no está justificada porque no tienen atributos adicionales o asociaciones, y no se opera sobre ellos (se les trata) de manera diferente, y no se comportan de manera diferente de alguna manera que sea de interés⁴.

La Tabla 26.2 muestra algunos ejemplos de particiones de clases en el dominio de los pagos y otras áreas, utilizando este criterio.

Tabla 26.2. Ejemplo de particiones de subclases.	
Motivación de la subclase conceptual	Ejemplos
La subclase tiene atributos adicionales de interés	Pagos: no aplicable Biblioteca: <i>Libro</i> , subclase de <i>RecursoPrestable</i> , tiene un atributo <i>ISBN</i> .
La subclase tiene asociaciones adicionales de interés	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , asociada con una <i>TarjetaDeCredito</i> . Biblioteca: <i>Vídeo</i> , subclase de <i>RecursoPrestable</i> , asociada con un <i>Encargado</i> .

continúa

⁴ Los hombres y las mujeres tienen diferentes hábitos a la hora de comprar. Sin embargo, éstos no son relevantes para los requisitos del caso de uso actual —el criterio que delimita nuestro estudio—.

Tabla 26.2. Ejemplo de particiones de subclases. (Continuación)

Motivación de la subclase conceptual	Ejemplos
El concepto de la subclase funciona, se gestiona, reacciona, o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , se gestiona de manera diferente a otros tipos de pagos en el modo de autorizarlo. Biblioteca: <i>Software</i> , subclase de <i>RecursoPrestable</i> , requiere un depósito antes de que pueda prestarse.
El concepto de la subclase representa una cosa animada (por ejemplo, animal o robot) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.	Pagos: no aplicable. Biblioteca: no aplicable. Investigación de Mercado: <i>Hombre</i> , subclase de <i>Persona</i> , se comporta de manera diferente a la <i>Mujer</i> con respecto a los hábitos de compras.

26.5. Cuándo definir una superclase conceptual

Normalmente, se aconseja generalizar en una superclase común cuando se identifican elementos comunes entre subclases potenciales. A continuación presentamos los motivos para generalizar y definir una superclase:

Cree una superclase conceptual en una relación de generalización de subclases cuando:

•

Cuando las subclases potenciales representan variaciones de un concepto similar.

•

Las subclases se ajustarán a las reglas del 100% y Es-un.

•

Todas las subclases tienen el mismo atributo que se puede factorizar y expresar en la superclase.

•

Todas las subclases tienen la misma asociación que se puede factorizar y relacionar con la superclase.

Las siguientes secciones ilustran estos puntos.

26.6. Jerarquías de clases conceptuales del PDV NuevaEra

Clases de Pago

Si nos basamos en el criterio anterior para particionar la clase *Pago*, es útil crear una jerarquía de clases de varios tipos de pagos. La justificación de las clases y subclases se muestra en la Figura 26.7.

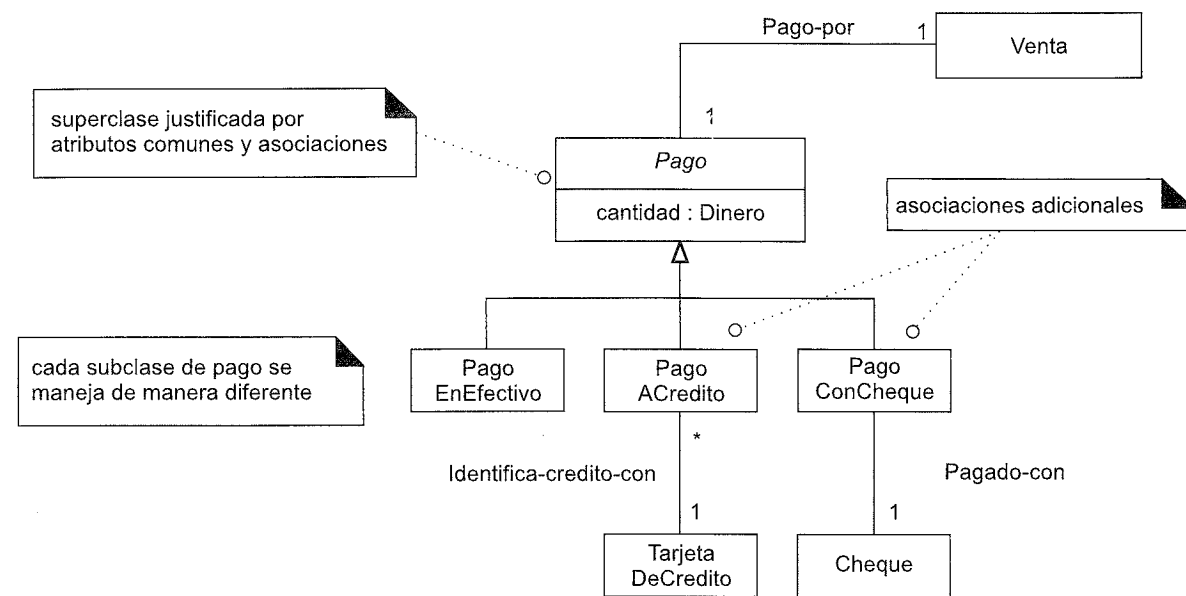


Figura 26.7. Justificación de las subclases de *Pago*.

Clases de Servicio de Autorización

Los servicios de autorización de crédito y cheques son variaciones de un concepto similar, y tienen atributos comunes de interés. Esto nos lleva a la jerarquía de clases de la Figura 26.8.

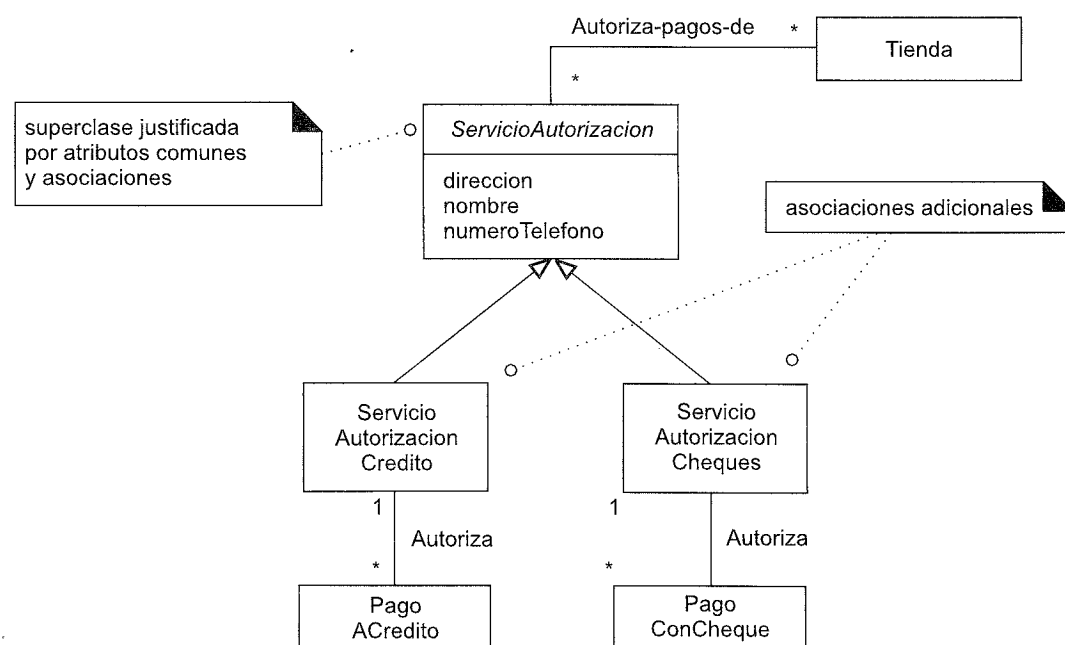


Figura 26.8. Justificación de la jerarquía de *ServicioAutorización*.

Clases de Transacción de Autorización

El modelado de distintos tipos de transacciones del servicio de autorización (solicitudes y respuestas) presenta un caso interesante. En general, es útil mostrar las transacciones de los servicios externos en un modelo del dominio porque las actividades y los procesos suelen girar alrededor de ellas. Son conceptos importantes.

¿Debería el modelador representar *todas* las variaciones de una transacción de un servicio externo? Depende. Como se mencionó, los modelos del dominio no son necesariamente correctos o incorrectos, sino más bien, son más o menos útiles. Son útiles, porque cada clase de transacción está relacionada con diferentes conceptos, procesos y reglas del negocio⁵.

Una segunda pregunta interesante es el grado de generalización que es conveniente mostrar en un modelo. Por motivos de la explicación, asumamos que cada clase transacción tiene una fecha y hora. Estos atributos comunes, junto con el deseo de crear una generalización final para esta familia de conceptos relacionados, justifica la creación de *TransaccionAutorizacionPago*.

¿Pero es útil generalizar una respuesta en *RespuestaAutorizacionPagoACredito* y *RespuestaAutorizacionPagoConCheque*, como se muestra en la Figura 26.9, o es suficiente menos generalización, como se muestra en la Figura 26.10?

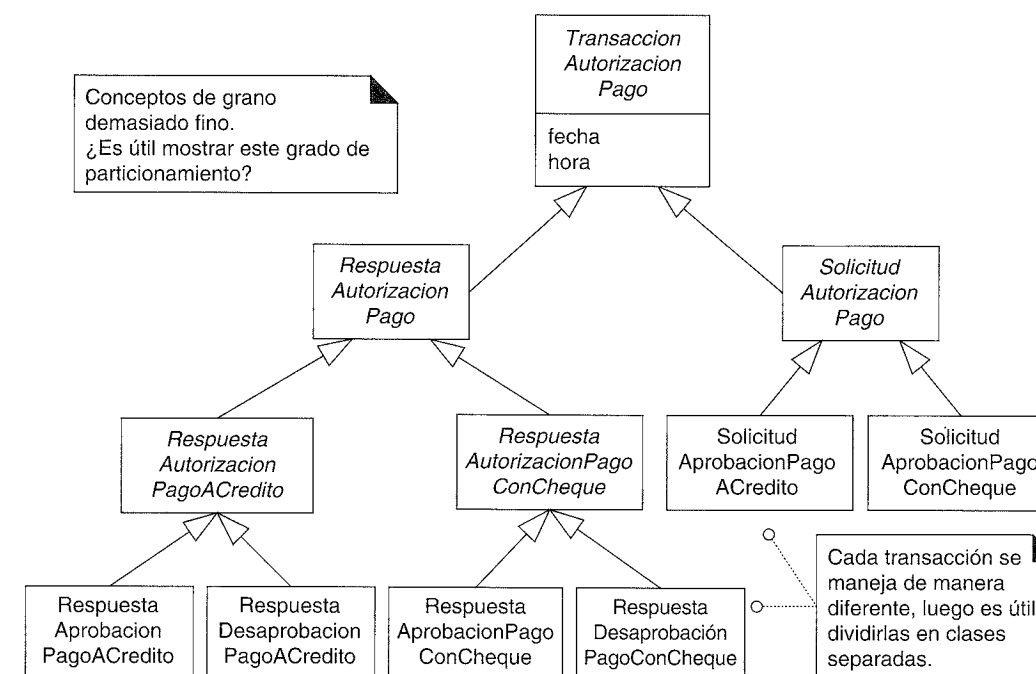


Figura 26.9. Una posible jerarquía de clases para las transacciones con los servicios externos.

⁵ En los modelos del dominio de las telecomunicaciones, es igualmente útil identificar cada tipo de mensaje de intercambio o cambio.

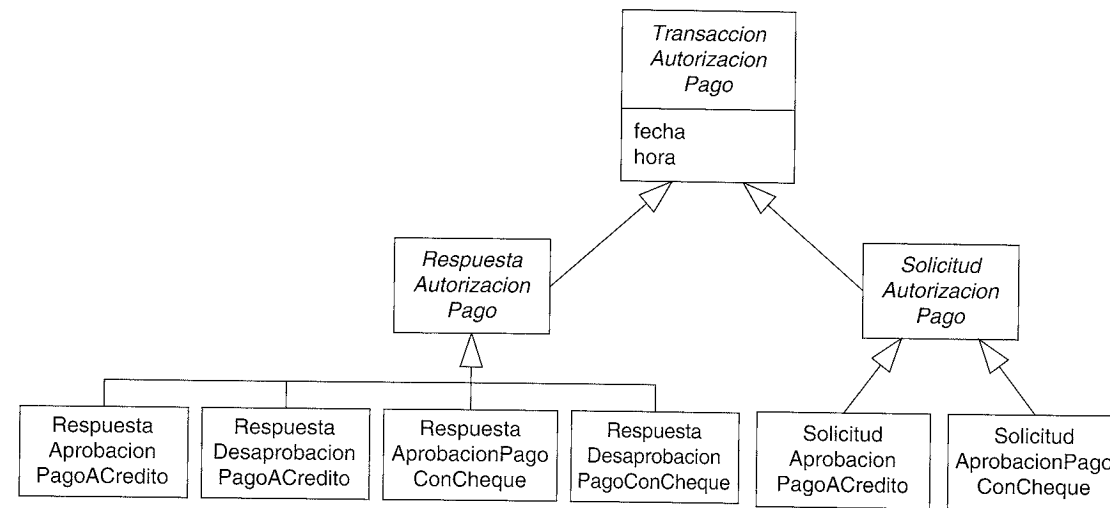


Figura 26.10. Una jerarquía de clases de transacción alternativa.

La jerarquía de clases que se muestra en la Figura 26.10 es suficientemente útil en cuanto a la generalización, porque las generalizaciones adicionales no añaden ningún valor obvio. La jerarquía de la Figura 26.9 expresa una granularidad de generalización más fina que no enriquece de manera significativa nuestra comprensión de los conceptos y reglas del negocio, sino que hace el modelo más complejo —y no es conveniente añadir complejidad a menos que proporcione otros beneficios.

26.7. Clases conceptuales abstractas

Es útil identificar las clases abstractas en el modelo del dominio porque restringe las clases que pueden tener instancias concretas; por tanto, clarifica las reglas del dominio del problema.

Si cada miembro de una clase C debe ser también un miembro de una subclase, entonces la clase C se denomina **clase conceptual abstracta**.

Por ejemplo, asuma que cada instancia de *Pago* deber ser más concretamente una instancia de la subclase *PagoACredito*, *PagoEnEfectivo* o *PagoConCheque*. Esto se ilustra en el diagrama de Venn de la Figura 26.11 (b). Puesto que cada miembro de *Pago* también es un miembro de una de las subclases, el *Pago* es una clase conceptual por definición.

En cambio, si puede haber instancias de *Pago* que no son miembros de una subclase, no es una clase abstracta, como se ilustra en la Figura 26.11 (a).

En el dominio del PDV, cada *Pago* es realmente un miembro de una subclase. La Figura 26.11 (b) es la descripción correcta de los pagos, el *Pago* es una clase conceptual abstracta.

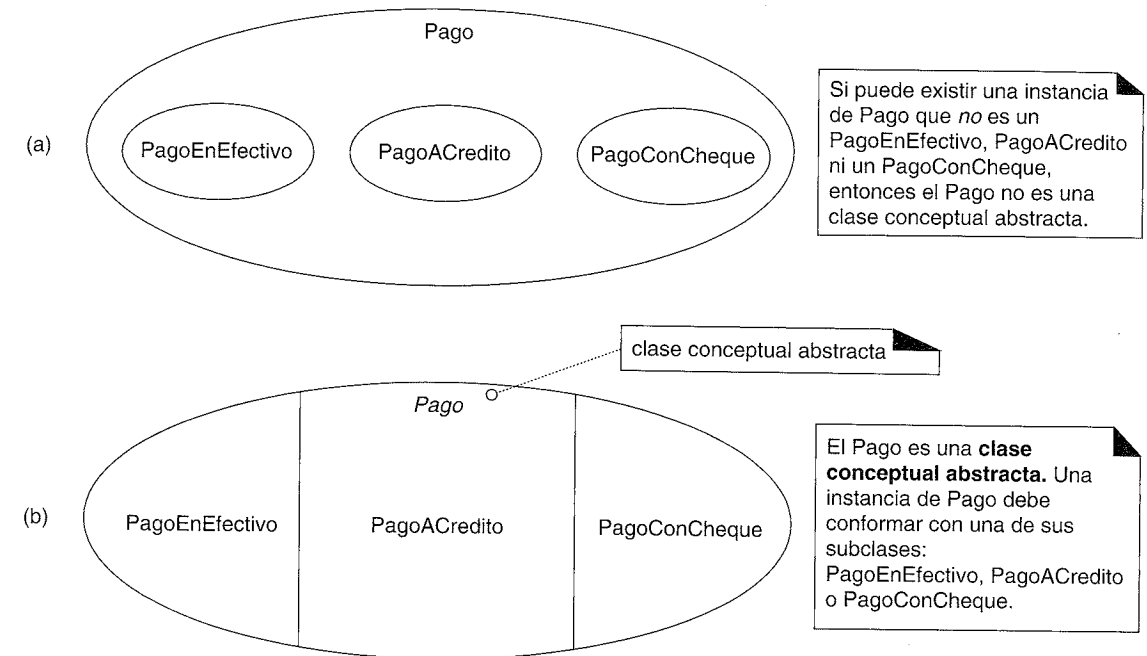


Figura 26.11. Clases conceptuales abstractas.

Notación para las clases abstractas en UML

Recordemos que UML proporciona una notación para representar las clases abstractas —el nombre de la clase en cursiva (ver Figura 26.12)—.

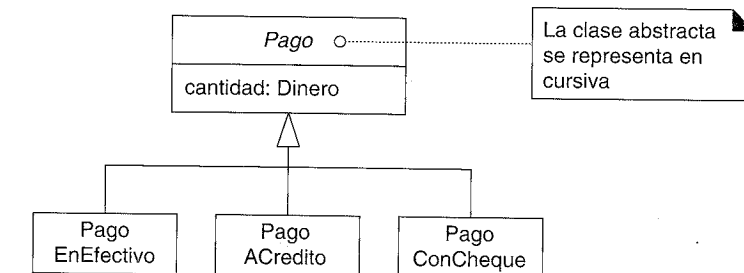


Figura 26.12. Notación para las clases abstractas.

Identifique las clases abstractas y represéntelas con un nombre en cursiva en el Modelo del Dominio.

26.8. Modelado de los cambios de estado

Asuma que un pago puede estar o en estado autorizado o no autorizado, y es significativo mostrarlo en el modelo del dominio (podría no serlo realmente, pero asúmalo para la exposición). Como se muestra en la Figura 26.13, un enfoque de modelado es definir

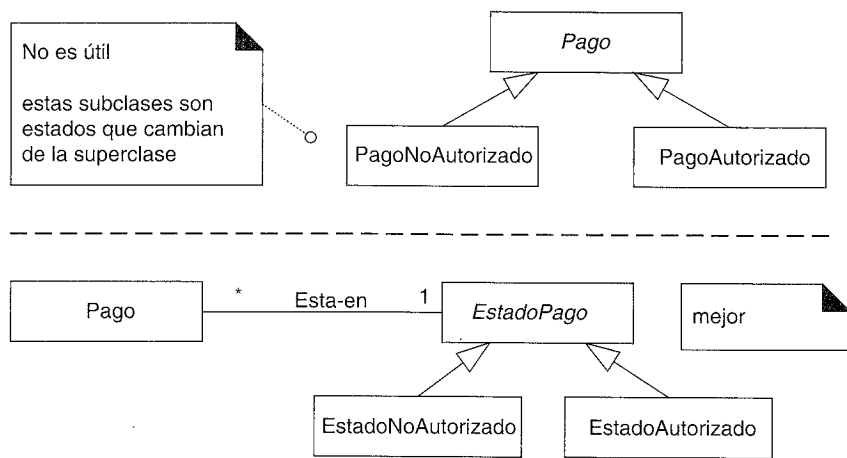


Figura 26.13. Modelado de los cambios de estado.

subclases de *Pago*: *PagoNoAutorizado* y *PagoAutorizado*. Sin embargo, observe que un pago no permanece en uno de estos estados; normalmente cambia de no autorizado a autorizado. Esto nos lleva a la siguiente guía:

No modele el estado de un concepto X como subclases de X. Más bien, o:

- Defina una jerarquía de estados y asocie los estados con X, o
- Ignore la representación de los estados de un concepto en el modelo del dominio; en lugar de eso represente los estados en diagramas de estados.

26.9. Jerarquías de clases y herencia en el software

La explicación de las jerarquías de clases conceptuales no ha mencionado la *herencia* porque la discusión se centra en un modelo del dominio de cosas del mundo, no en artefactos software. En un lenguaje de programación orientado a objetos, una subclase software **hereda** la definición de los atributos y operaciones de sus superclases mediante la creación de **jerarquías de clases software**. La **herencia** es un mecanismo software para hacer que las cosas de la superclase se apliquen a las subclases. Permite factorizar código de las subclases y subirlo arriba de la jerarquía de clases. Por tanto, la herencia no tiene ningún papel real que desempeñar en la discusión del modelo del dominio, aunque lo tiene cuando pasamos al punto de vista del diseño o la implementación.

Las jerarquías de clases conceptuales generadas aquí podrían reflejarse, o no, en el Modelo de Diseño. Por ejemplo, la jerarquía de clases de transacciones de los servicios externos podría reunirse o expandirse en jerarquías de clases alternativas, dependiendo de las características del lenguaje y otros factores. Por ejemplo, las clases plantilla (*template*) de C++ algunas veces pueden reducir el número de clases.

REFINAMIENTO DEL MODELO DEL DOMINIO

PRESENTE, n. Aquella parte de la eternidad que divide el dominio de la desilusión del reino de la esperanza.
Ambrose Bierce.

Objetivos

- Añadir clases asociación al Modelo del Dominio.
- Añadir relaciones de agregación.
- Modelar los intervalos de tiempo de la información donde es aplicable.
- Elegir cómo modelar roles.
- Organizar el Modelo del Dominio en paquetes.

Introducción

Este capítulo presenta ideas útiles y notación adicional disponible para el modelado del dominio, y las aplica para refinar aspectos del Modelo del Dominio del PDV NuevaEra.

27.1 Clases asociación

Los siguientes requisitos del dominio disponen el escenario para las clases asociación:

- Los servicios de autorización asignan un ID de comerciante a cada tienda para que se identifique en las comunicaciones.
- Una solicitud de autorización de pago desde la tienda a un servicio de autorización necesita el ID de comerciante que identifica la tienda al servicio.

- Además, una tienda tiene un ID de comerciante distinto para cada servicio.
¿Dónde debería residir el ID de comerciante en el Modelo del Dominio del UP?

Es incorrecto colocar el *comercianteID* en la *Tienda* porque una *Tienda* puede tener más de un valor para el *comercianteID*. Lo mismo ocurre si lo colocamos en el *ServicioAutorizacion* (ver Figura 27.1).

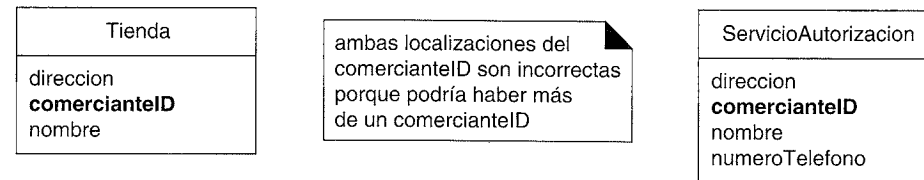


Figura 27.1. Uso inapropiado de un atributo.

Esto nos lleva al siguiente principio de modelado:

En un modelo del dominio, si una clase C puede tener simultáneamente muchos valores para el mismo tipo de atributo A, no coloque el atributo A en C. Coloque el atributo A en otra clase que esté asociada con C.

Por ejemplo:

- Una *Persona* podría tener muchos números de teléfono. Coloque el número de teléfono en otra clase, como *NumeroTelefono* o *InformacionDeContacto*, y asocie muchas de éstas a *Persona*.

El principio anterior sugiere que es más apropiado un modelo como el de la Figura 27.2. En el mundo de los negocios, ¿qué concepto recoge formalmente la información relacionada con los servicios que un servicio proporciona a un cliente? —un *Contrato* o *Cuenta*—.

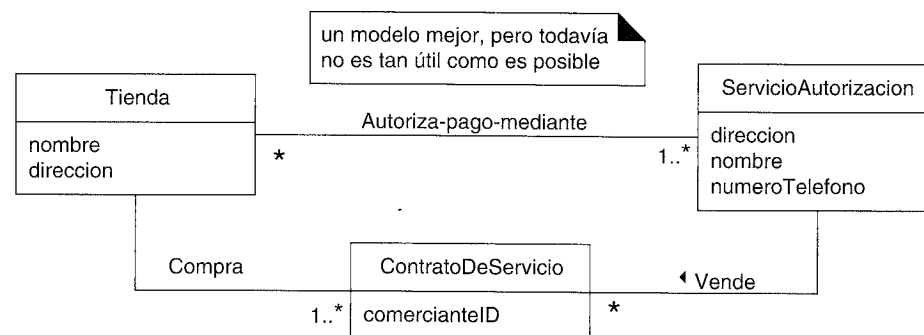


Figura 27.2. Primer intento de modelado del problema del comercianteID.

El hecho de que tanto *Tienda* como *ServicioAutorizacion* estén relacionados con *ContratoDeServicio* es un síntoma de que depende de la relación entre los dos. Se podría pensar en el *comercianteID* como un atributo relacionado con la asociación entre la *Tienda* y el *ServicioAutorizacion*.

Esto nos lleva a la noción de **clase asociación**, en la que podemos añadir características a la propia asociación. Se podría modelar *ContratoDeServicio* como una clase asociación relacionada con la asociación entre *Tienda* y *ServicioAutorizacion*.

En UML, esto se representa con una línea punteada desde la asociación a la clase asociación. La Figura 27.3 transmite visualmente la idea de que un *ContratoDeServicio* y sus atributos están relacionados con la asociación entre una *Tienda* y un *ServicioAutorizacion*, y que el tiempo de vida del *ContratoDeServicio* depende de la relación.

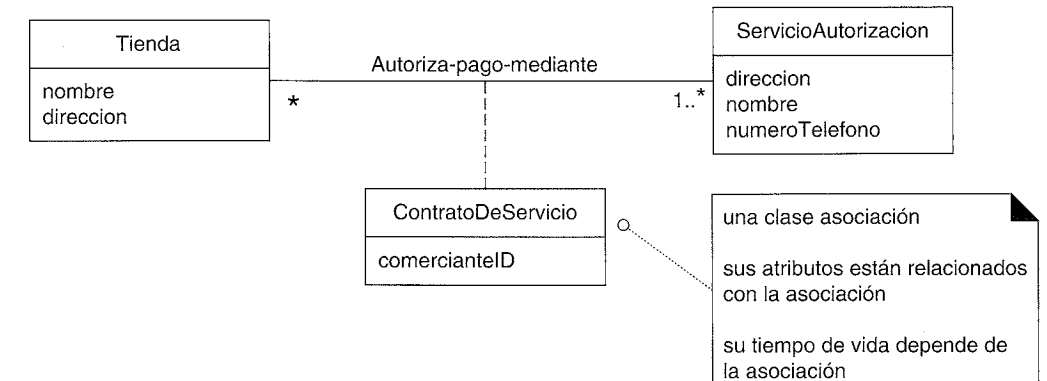


Figura 27.3. Una clase asociación.

Guías

Entre las guías para incluir clases asociaciones se encuentran las siguientes:

Indicios de que podría ser útil una clase asociación en un modelo del dominio:

- Un atributo está relacionado con una asociación.
- El tiempo de vida de las instancias de la clase asociación depende de la asociación.
- Existe una asociación muchos-a-muchos entre dos conceptos, e información asociada con la propia asociación.

La presencia de una asociación muchos-a-muchos es un signo típico de que una clase asociación útil está escondida en segundo plano en algún sitio; cuando vea una, tenga en cuenta una clase asociación.

La Figura 27.4 ilustra algunos otros ejemplos de clases asociación.

27.2. Agregación y composición

La **agregación** es un tipo de asociación que se utiliza para modelar las relaciones todo-parte entre las cosas. El todo se denomina **compuesto**.

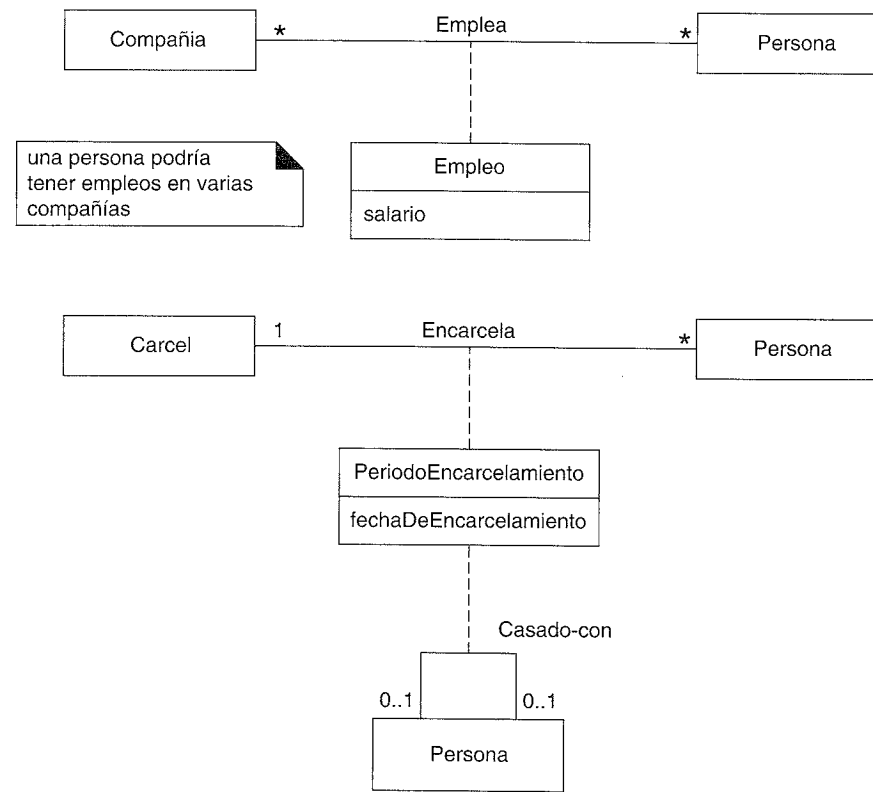


Figura 27.4. Clases asociación.

Por ejemplo, los ensamblajes físicos se organizan mediante relaciones de agregación, del mismo modo que una *Mano* agrega *Dedos*.

Agregación en UML

La agregación se representa en UML mediante un rombo hueco o relleno en el extremo del compuesto de una asociación todo-parte (ver Figura 27.5).

La agregación es una propiedad de un rol de asociación¹.

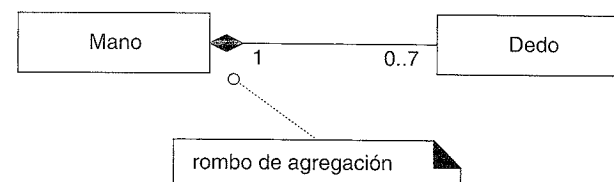


Figura 27.5. Notación de la agregación.

¹ Recordemos que cada extremo de asociación es un rol, y que un rol en UML tiene varias propiedades, como *multiplicidad*, *nombre*, *navegabilidad* y *esAgregado*.

El nombre de la asociación se excluye a menudo de las relaciones de agregación puesto que se piensa habitualmente como *Tiene-parte*. Sin embargo, uno puede acostumbrarse a proporcionar más detalles semánticos.

Agregación de composición: rombo relleno

La **agregación de composición**, o **composición**, significa que la parte es un miembro de un único objeto compuesto y que existe una dependencia de existencia y disposición de la parte sobre el compuesto. Por ejemplo, existe una relación de composición entre la mano y un dedo.

En el Modelo de Diseño, la composición y su implicación de dependencia de existencia indica que los objetos software compuestos crean (o provocan la creación de) los objetos software de la parte (por ejemplo, la *Venta* crea los objetos *LineaDeVenta*).

Pero en el Modelo del Dominio, puesto que no representa objetos software, rara vez es relevante la noción del todo que crea las partes (una venta real no crea una línea de venta real). Sin embargo, existe todavía una analogía. Por ejemplo, en el modelo del dominio de un “cuerpo humano”, uno piensa en la mano que incluye los dedos, luego si uno dice, “Se ha formado una mano”, entendemos que también significa que se han formado los dedos igualmente.

La composición se denota con un rombo relleno. Esto implica que solamente el compuesto posee la parte, y que se encuentran en una jerarquía de partes en forma de árbol; es la forma de agregación más común que se presenta al modelar.

Por ejemplo, un dedo es una parte de una única mano (¡esperamos!), por tanto el rombo de agregación está relleno para indicar agregación de composición (ver Figura 27.6).

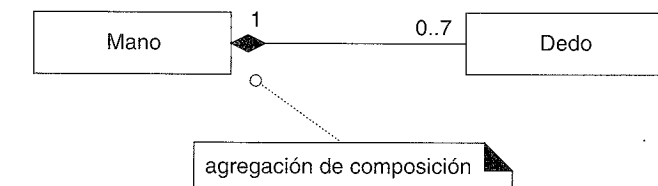


Figura 27.6. Agregación de composición.

Si la multiplicidad en el extremo del compuesto es exactamente uno, la parte *no* podría existir separada de algún compuesto. Por ejemplo, si se quita un dedo de una mano, debe conectarse inmediatamente a otro objeto compuesto (otra mano, pie,...); al menos, eso es lo que está declarando el modelo, ¡independientemente de los méritos mé-dicos de esta idea!

Si la multiplicidad en el extremo del compuesto es 0..1, entonces podría eliminarse la parte del compuesto, y existir todavía sin pertenecer a ningún otro compuesto. Luego, si quiere que los dedos floten en el aire por sí mismos, utilice 0..1.

Agregación compartida: rombo hueco

La **agregación compartida** significa que la multiplicidad en el extremo del compuesto podría ser más de uno, y se representa mediante un rombo hueco. Implica que la parte podría estar simultáneamente en muchas instancias del compuesto. La agregación compartida rara vez (si existe alguna) existe en agregaciones físicas, sino más bien en conceptos no físicos.

Por ejemplo, se podría considerar que un paquete UML agrega sus elementos. Pero un elemento podría ser referenciado en más de un paquete (pertenecer a un paquete, y es referenciado en otros), lo cual es un ejemplo de agregación compartida (ver Figura 27.7).

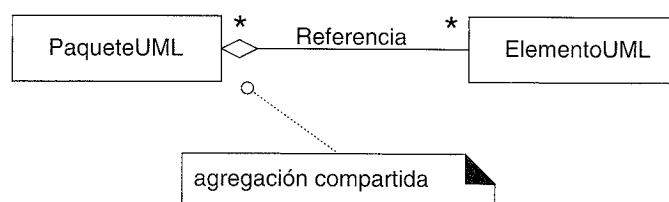


Figura 27.7. Agregación compartida.

Cómo identificar la agregación

En algunos casos, la presencia de la agregación es obvia —normalmente en ensamblajes físicos—. Pero algunas veces, no está claro.

En la agregación: si hay duda, descártela.

A continuación presentamos algunas guías que sugieren cuando mostrar una agregación:

Considere mostrar una agregación cuando:

- El tiempo de vida de la parte está ligado al tiempo de vida del compuesto —existe una dependencia de creación-eliminación de la parte en el todo—.
- Existe un ensamblaje obvio todo-parte físico o lógico.
- alguna propiedad del compuesto se propaga a las partes, como la ubicación.
- Las operaciones que se aplican sobre el compuesto se propagan a las partes, como la destrucción, movimiento o grabación.

Aparte de lo que es un ensamblaje de partes obvio, la siguiente pista más útil es la presencia de una dependencia de creación-eliminación de la parte en el todo.

Un beneficio de la representación de la agregación

Identificar y representar la agregación *no* es excesivamente importante; es bastante posible que se excluya del modelo del dominio. La mayoría —si no todos— los modeladores del dominio con experiencia han visto que se malgasta tiempo improductivo debatiendo los puntos más sutiles de estas asociaciones.

Descubra y muestre las agregaciones si eso le proporciona los siguientes beneficios, la mayoría de ellos se relacionan con el diseño en lugar de con el análisis, que es por lo que no es muy significativa su exclusión del modelo del dominio.

- Aclara las restricciones del dominio en cuanto a la existencia que se desea de la parte independiente del todo. En la agregación de composición, la parte no podría existir fuera del tiempo de vida del todo.
- Durante el trabajo de diseño, esto influye en las dependencias de creación-eliminación entre las clases software del todo y la parte y los elementos de la base de datos (en cuanto a la integridad referencial y los caminos de eliminación en cascada).
- Ayuda en la identificación de un creador (el compuesto) utilizando el patrón GRASP Creador.
- Las operaciones —como la copia y la eliminación— que se aplican al todo a menudo se propagan a las partes.

Agregación en el Modelo del Dominio del PDV

En el dominio del PDV, se podría considerar las instancias *LineaDeVenta* como parte de una *Venta* compuesta; en general, las líneas de una transacción se ven como parte de una transacción agregada (ver Figura 27.8). Además de ajustarse a ese patrón, existe una dependencia de creación-eliminación de las líneas de venta con la *Venta* —sus tiempos de vida están ligados al tiempo de vida de la *Venta*—.

Por una justificación parecida, el *CatalogoDeProductos* es un agregado de objetos *EspecificacionDelProducto*.

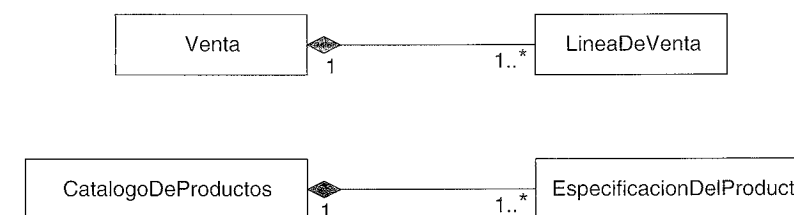


Figura 27.8. Agregación en la aplicación de punto de venta.

Ninguna otra relación es una combinación convincente que sugiera la semántica todo-parte, la dependencia creación-eliminación, y “En caso de duda, descártela”.

27.3. Intervalos de tiempo y precios de los productos: arreglar un “error” de la Iteración 1

En la primera iteración, las instancias de *LineaDeVenta* se asociaron con las instancias de *EspecificacionDelProducto*, que recogen el precio de un artículo. Esto era una simplificación razonable para las primeras iteraciones, pero necesita que se corrija. Surge la cuestión interesante —y ampliamente aplicable— de los **intervalos de tiempo** asociados con información, contratos y otras cosas por el estilo.

Si una *LineaDeVenta* siempre recuperaba el precio actual registrado en una *EspecificacionDelProducto*, entonces cuando se cambió el precio en el objeto, las antiguas ventas referenciarían a los nuevos precios, lo cual es incorrecto. Lo que se necesita es distinguir entre el precio histórico de cuando se creó la venta, y el precio actual.

Dependiendo de los requisitos de información, hay al menos dos formas de modelar esto. Una es simplemente copiar el precio del producto en la *LineaDeVenta* y mantener el precio actual en la *EspecificacionDelProducto*.

El otro enfoque, más robusto, es asociar una colección de objetos *PrecioProducto* con una *EspecificacionDelProducto*, cada uno con un intervalo de tiempo aplicable asociado. Por tanto, la organización puede registrar todos los precios anteriores (para resolver el problema de los precios de venta y para análisis de tendencias) y también registrar los futuros precios previstos (ver Figura 27.9). Diríjase a [CLD99] para una discusión más amplia sobre los intervalos de tiempo, bajo la categoría de arquetipos **Momento-Intervalo**.

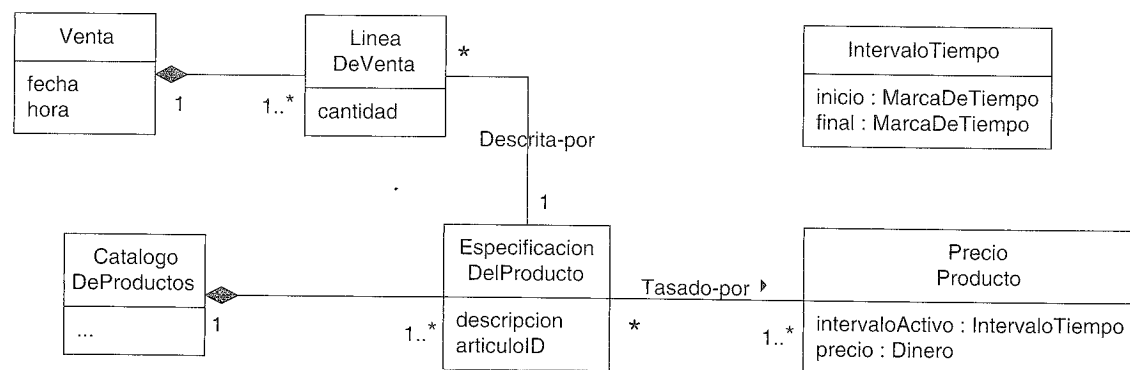


Figura 27.9. PrecioProducto e intervalos de tiempo.

Es habitual que se necesite mantener una colección de información relacionada con intervalos de tiempo, en lugar de un valor simple. Medidas físicas, médicas y científicas, y muchos artefactos de contabilidad y jurídicos, tienen este requisito.

27.4 Nombres de los roles de asociación

Cada extremo de asociación es un rol, que tiene varias propiedades como:

- nombre
- multiplicidad

Un nombre de rol identifica un extremo de una asociación e idealmente describe el papel que juegan los objetos en la asociación. La Figura 27.10 muestra ejemplos de nombres de roles.

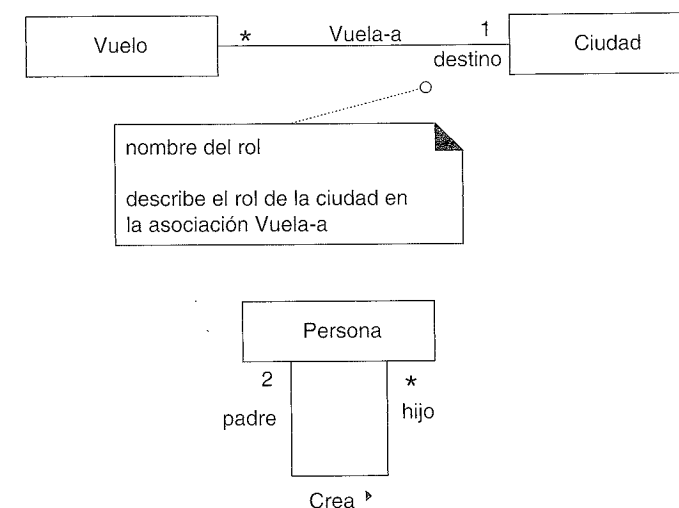


Figura 27.10. Nombres de roles.

No se requiere un nombre de rol explícito —es útil cuando no está claro el rol del objeto—. Normalmente comienza con una letra en minúscula. Si no se muestra explícitamente, asuma que, por defecto, el nombre del rol es igual al nombre de la clase con la que se relaciona, aunque empezando con minúscula.

Como se presentó previamente durante la explicación del paso del diseño al código, los roles utilizados en los DCDs podrían interpretarse como base para los nombres de los atributos durante la generación de código.

27.5. Roles como conceptos vs. roles en asociaciones

En un modelo del dominio, un rol del mundo real —especialmente un rol humano— podría modelarse de varias formas, como un concepto separado, o representado como un rol en una asociación². Por ejemplo, el rol del cajero y el encargado se pueden expresar al menos de las dos formas que se muestran en la Figura 27.11.

El primer enfoque podría llamarse “roles en asociaciones”; el segundo “roles como conceptos”. Ambos enfoques tienen ventajas.

Los roles en asociaciones son atractivos porque son una forma relativamente precisa de expresar que la misma instancia de una persona asume múltiples (y cambiantes dinámicamente) roles en varias asociaciones. Yo, una persona, simultáneamente o sucesivamente, podría asumir el rol de escritor, diseñador de software, padre, etcétera.

² Por simplicidad, se han ignorado otras excelentes soluciones como las que se discuten en [Fowler96].

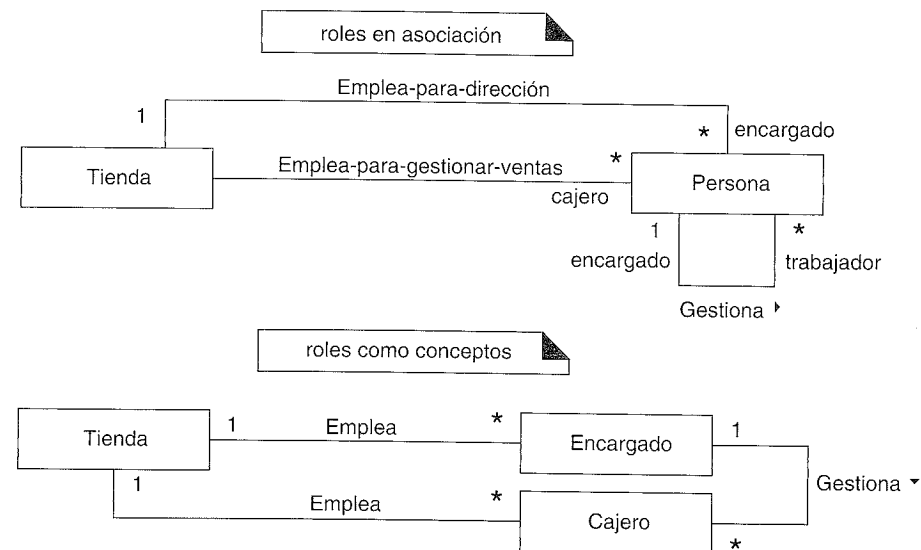


Figura 27.11. Dos formas de modelar los roles humanos.

Por otro lado, los roles como conceptos facilitan y hacen más flexible la inclusión de atributos únicos, asociaciones y semántica adicional. Además, la implementación de los roles como clases separadas es más sencillo debido a las limitaciones de los lenguajes de programación orientados a objetos comerciales actuales —no es conveniente cambiar dinámicamente una instancia de una clase a otra, o añadir dinámicamente comportamiento y atributos cuando cambia el rol de una persona—.

27.6. Elementos derivados

Un elemento derivado puede ser determinado a partir de otros. Los atributos y las asociaciones son los elementos derivados más comunes. ¿Cuándo se deben mostrar los elementos derivados?

Evite mostrar los elementos derivados en un diagrama, puesto que añaden complejidad sin información nueva. Sin embargo, añada un elemento derivado cuando sea un elemento destacado en la terminología, y si se excluye se perjudica la comprensión.

Por ejemplo, el *total* de una *Venta* se puede derivar a partir de la información de los objetos *LineaDeVenta* y *EspecificacionDelProducto* (ver Figura 27.12). En UML, se representa anteponiendo una “/” al nombre del elemento.

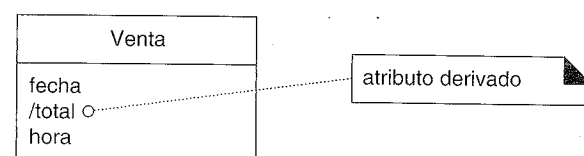


Figura 27.12. Atributo derivado.

Otro ejemplo, la *cantidad* de una *LineaDeVenta* realmente se puede derivar a partir del número de instancias de *Articulo* asociadas con la línea de venta (ver Figura 27.13).

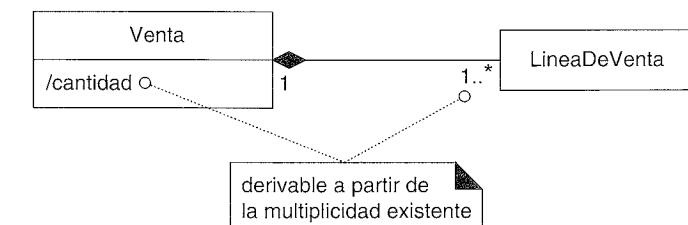


Figura 27.13. Atributo derivado relacionado con la multiplicidad.

27.7. Asociaciones calificadas

En una asociación podría utilizarse un **calificador**; que permite distinguir dentro del conjunto de objetos en el otro extremo de la asociación en base al valor del calificador. Una asociación con un calificador es una **asociación calificada**.

Por ejemplo, podrían distinguirse los objetos *EspecificacionDelProducto* en un *CatalogoDeProductos* según su *articuloID*, como se ilustra en la Figura 27.14 (b). Comparando las figuras (a) y (b) en la Figura 27.14, la calificación reduce la multiplicidad en el extremo más alejado del calificador, normalmente la disminuye de muchos a uno. La inclusión de un calificador en un modelo del dominio comunica cómo, en el dominio, se distinguen las instancias de una clase en relación con otra clase. No debería utilizarse, en el modelo del dominio, para expresar decisiones de diseño sobre claves de búsqueda, aunque es conveniente en otros diagramas que ilustren decisiones de diseño.

Los calificadores normalmente no añaden nueva información útil convincente, y podemos caer en la trampa de “pensar-diseño”. Sin embargo, utilizados juiciosamente, pueden mejorar el conocimiento sobre el dominio. La asociación calificada entre *CatalogoDeProductos* y *EspecificacionDelProducto* proporciona un ejemplo razonable de un calificador con valor añadido.

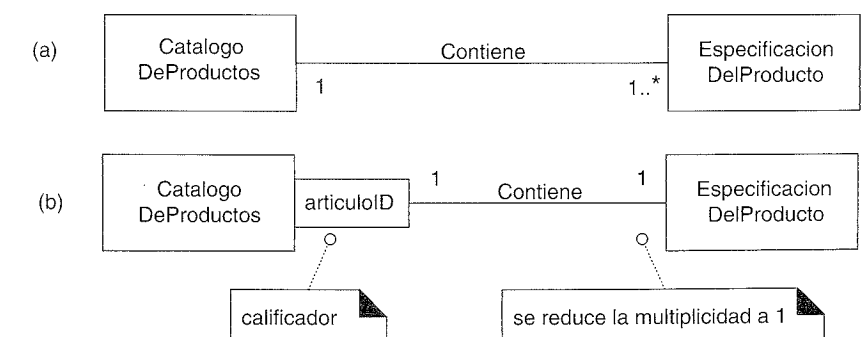


Figura 27.14. Asociación calificada.

27.8. Asociaciones reflexivas

Un concepto podría estar asociado con él mismo; esto se conoce como **asociación reflexiva**³ (ver Figura 27.15).

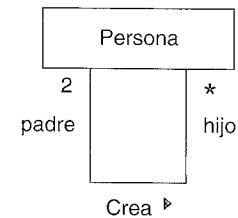


Figura 27.15. Asociación reflexiva.

27.9. Elementos ordenados

Si los objetos asociados están ordenados, esto se puede representar como en la Figura 27.16. Por ejemplo, las instancias de una *LineaDeVenta* se deben mantener en el orden de entrada.

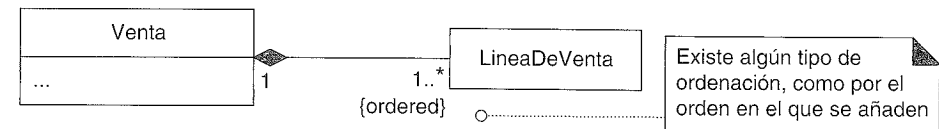


Figura 27.16. Elementos ordenados.

27.10. Utilización de paquetes para organizar el Modelo del Dominio

Un modelo del dominio puede crecer fácilmente y llegar a ser lo suficientemente amplio para que sea conveniente dividirlo en paquetes que incluyen conceptos fuertemente relacionados, esto sirve de ayuda para mejorar la comprensión y para abordar trabajo de análisis en paralelo, en el que diferentes personas realizan el análisis del dominio en diferentes subdominios. Las siguientes secciones ilustran una estructura de paquetes para el Modelo del Dominio del UP.

Notación de paquetes en UML

Recordemos que un paquete en UML se representa mediante una carpeta (ver Figura 27.17). Podrían mostrarse dentro de un paquete otros paquetes subordinados. Si el pa-

³ [MO95] restringe más aún la definición de las asociaciones reflexivas.

quete describe sus elementos, el nombre del paquete se coloca en la etiqueta; en otro caso, se centra en la propia carpeta.

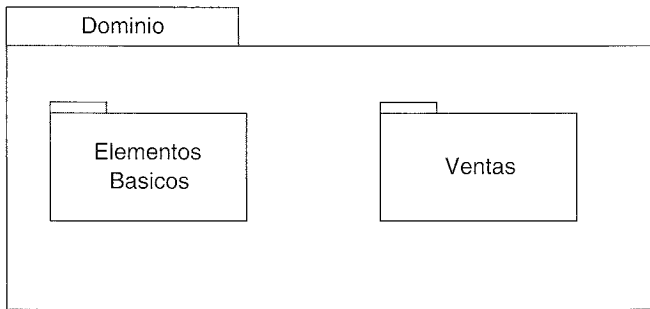


Figura 27.17. Un paquete UML.

Pertenencia y referencias

Un elemento *pertenece* al paquete donde está definido, pero podría ser referenciado en otros paquetes. En ese caso, el nombre del elemento se califica con el nombre del paquete utilizando el formato del nombre de camino *NombrePaquete::NombreElemento* (ver Figura 27.18). Una clase que se muestra en un paquete que no es al que pertenece se podría modificar con nuevas asociaciones, pero por lo demás permanece sin alterar.

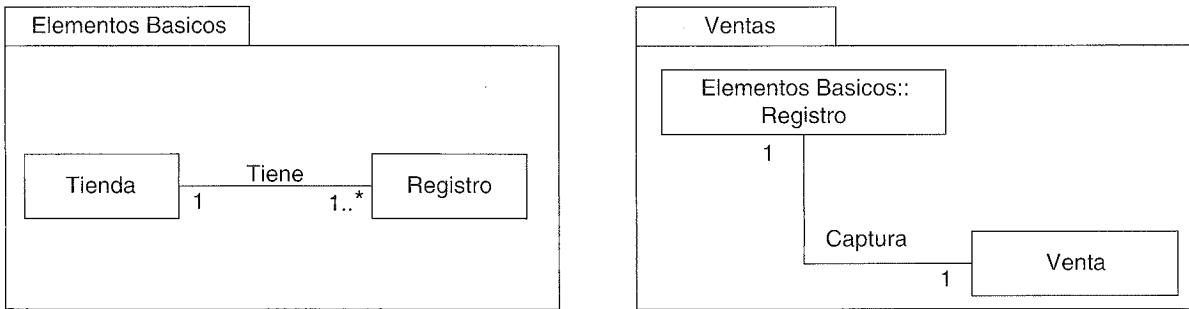


Figura 27.18. Una clase referenciada en un paquete.

Dependencias entre paquetes

Si un elemento del modelo depende de algún modo de otro, se podría representar la dependencia con una relación de dependencia, descrita por una línea con punta de flecha. Una dependencia entre paquetes indica que los elementos del paquete dependiente conocen o están acoplados de algún modo con los elementos del paquete destino.

Por ejemplo, si un paquete referencia a un elemento que pertenece a otro, existe una dependencia. Por tanto, el paquete de *Ventas* tiene una dependencia con el paquete *Elementos Basicos* (ver Figura 27.19).

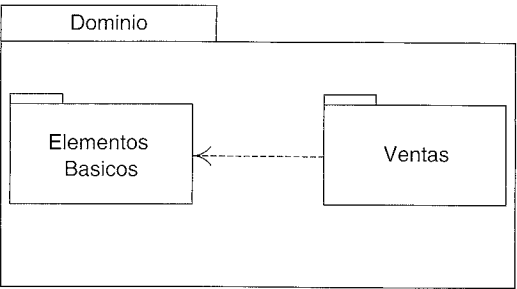


Figura 27.19. Dependencia entre paquetes.

Indicación del paquete sin el diagrama de paquetes

A veces, no es conveniente dibujar un diagrama de paquetes, pero no obstante es deseable indicar el paquete al que pertenecen los elementos.

En esta situación, incluya una nota (un rectángulo con la esquina doblada), como se ilustra en la Figura 27.20.

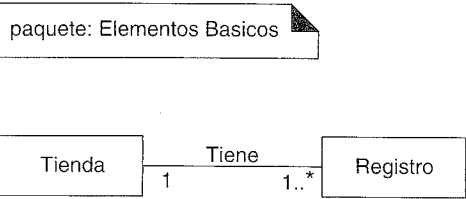


Figura 27.20. Representación de la pertenencia a un paquete con una nota.

Cómo se particiona el Modelo del Dominio

¿Cómo deberían organizarse en paquetes las clases del modelo del dominio? Aplique las siguientes guías generales:

- Para particionar el modelo del dominio en paquetes, ponga juntos los elementos que:
- se encuentran en el mismo área de interés —estrechamente relacionados por conceptos u objetivos—
 - están juntos en una jerarquía de clases
 - participan en los mismos casos de uso
 - están fuertemente asociados

Resulta útil que todos los elementos relacionados con el modelo del dominio tengan como raíz un paquete denominado *Dominio*, y todos los conceptos básicos, comunes, compartidos, se definan en un paquete que se puede llamar algo así como *Elementos Basicos* o *Conceptos Comunes*, en ausencia de cualquier otro paquete significativo en el que colocarlos.

Paquetes del Modelo del Dominio del PDV

En base al criterio anterior, la organización de paquetes para el Modelo del Dominio del PDV se muestra en la Figura 27.21.

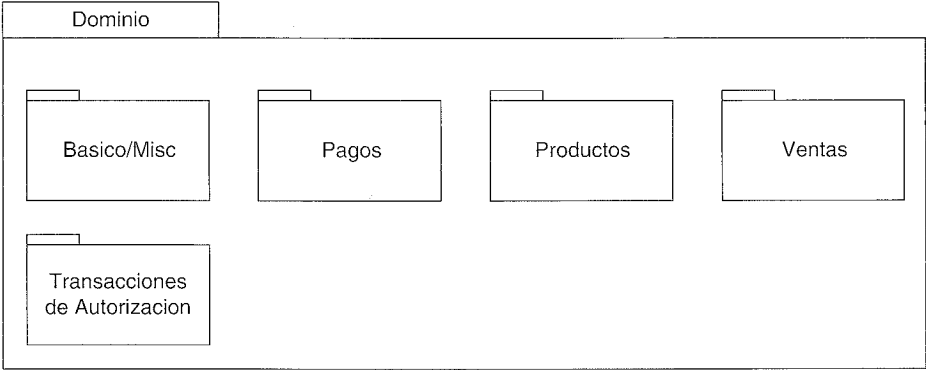


Figura 27.21. Paquetes de conceptos del dominio.

Paquete Basico/Misc

Un paquete Basico/Misc (ver Figura 27.22) es conveniente que contenga conceptos ampliamente compartidos o aquéllos sin una ubicación obvia. En referencias posteriores, se abreviará el nombre del paquete a *Basico*.

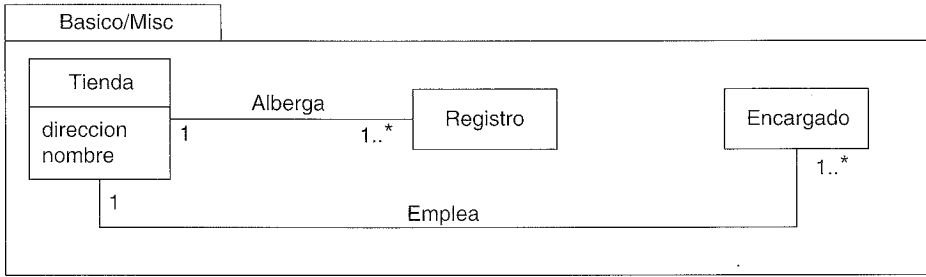


Figura 27.22. Paquete básico.

No existen nuevos conceptos ni asociaciones específicas de esta iteración en este paquete.

Pagos

Como en la iteración 1, sobre todo la consideración del criterio necesito-conocer motiva la aparición de nuevas asociaciones. Por ejemplo, se necesita registrar la relación entre el *PagoACredito* y la *TarjetaDeCredito*. En cambio, se añaden algunas asociaciones más para mejorar la comprensión, como *CarnetConducir Identifica Cliente* (ver Figura 27.23).

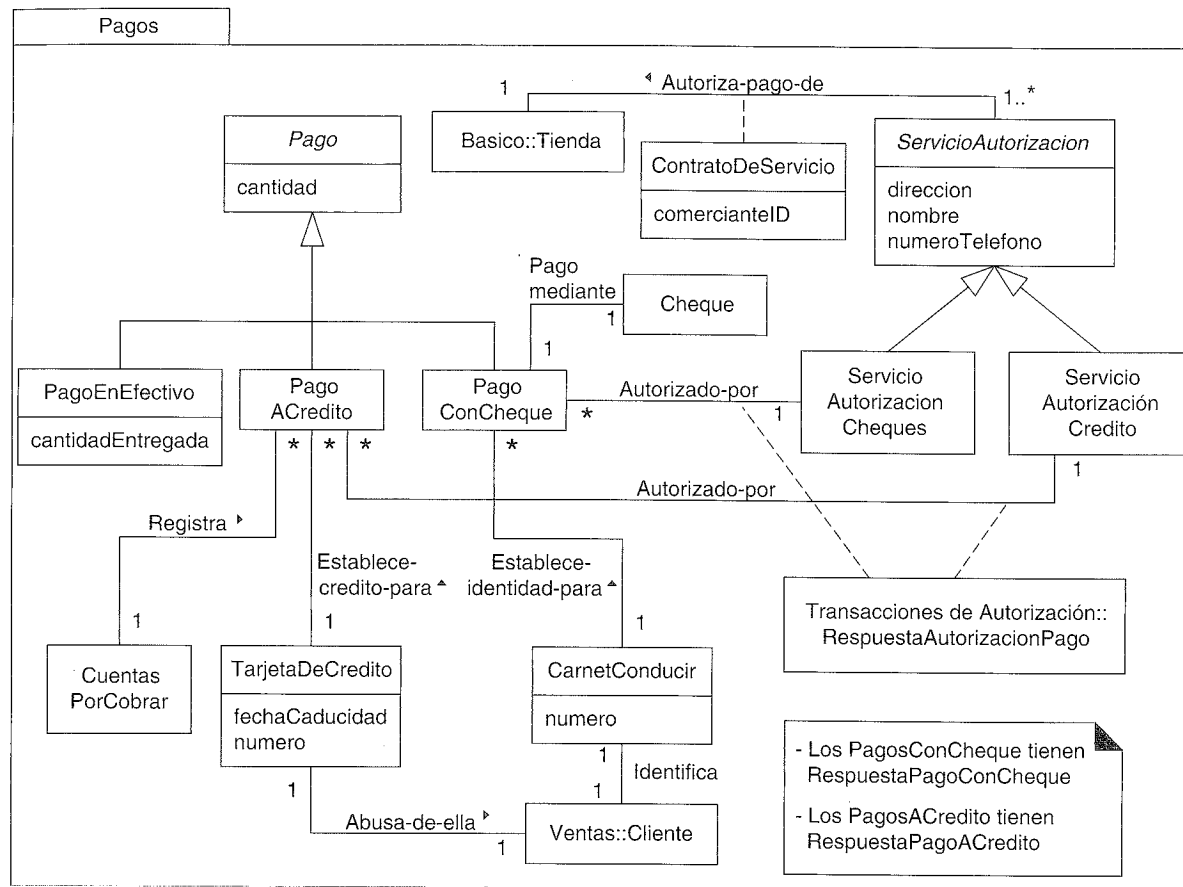


Figura 27.23. Paquete de pagos.

Nótese que la *RespuestaAutorizacionPago* se representa como una clase asociación. Una respuesta surge de la asociación entre un pago y su servicio de autorización.

Productos

Con la excepción de la agregación de composición, no existen nuevos conceptos o asociaciones específicas de esta iteración (ver Figura 27.24).

Ventas

Con la excepción de la agregación de composición y los atributos derivados, no existen nuevos conceptos o asociaciones específicas de esta iteración (ver Figura 27.25).

Transacciones de Autorización

Aunque se recomienda proporcionar nombres significativos a las asociaciones, en algunos casos podría no ser indispensable, especialmente si se considera que el propósito

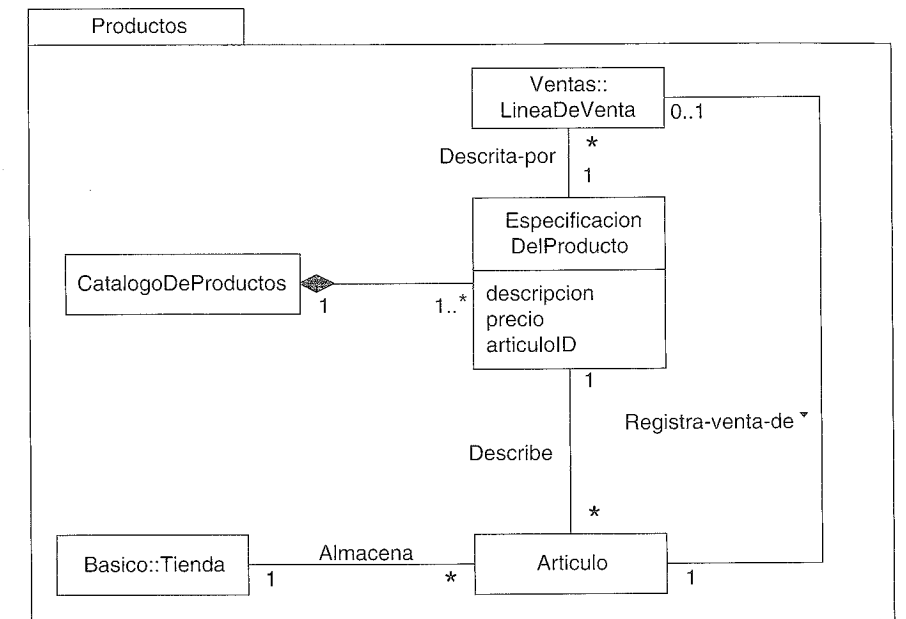


Figura 27.24. Paquete de productos

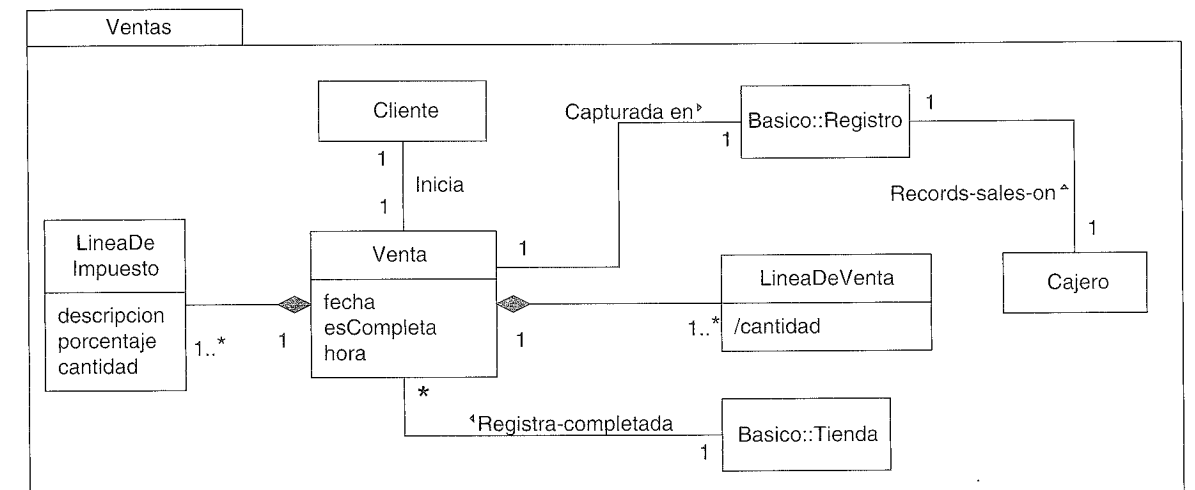


Figura 27.25. Paquete de ventas.

de la asociación es obvio para los destinatarios. Un ejemplo de este caso es la asociación entre los pagos y sus transacciones. Los nombres se han dejado sin especificar porque podemos asumir que las personas que van a leer el diagrama de clases de la Figura 27.26 entenderán que las transacciones son para los pagos; añadir los nombres simplemente enmaraña el diagrama.

¿Es este diagrama demasiado detallado y muestra demasiadas especializaciones? Depende. El verdadero criterio es la utilidad. Aunque no es incorrecto, ¿añade algún valor a mejorar la comprensión del dominio? La respuesta debería influir en el número de especializaciones que se representan en un modelo del dominio.

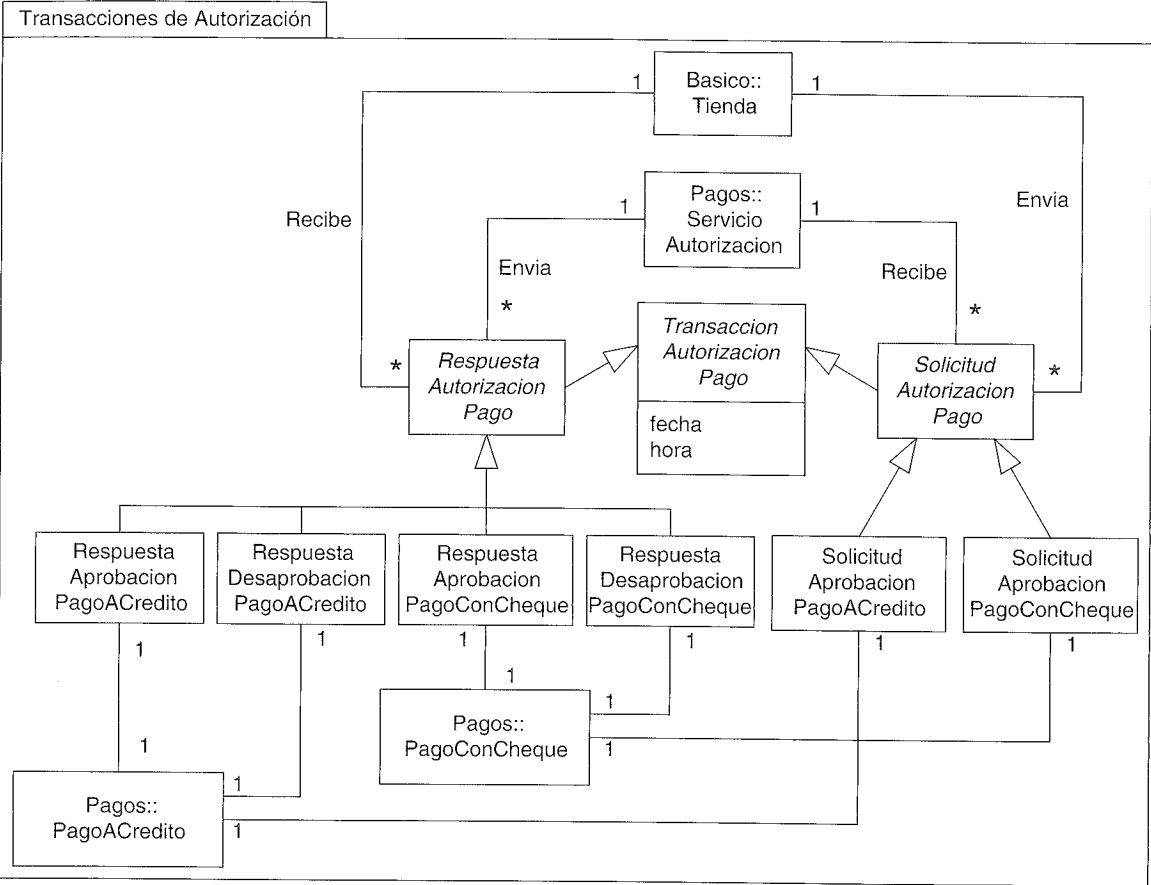


Figura 27.26. Paquete de transacciones de autorización.

AÑADIR NUEVOS DSSs Y CONTRATOS

La virtud es una tentación insuficiente.
George Bernard Shaw

Objetivos

- Definir los DSSs y los contratos de las operaciones del sistema para la iteración actual.

28.1. Nuevos diagramas de secuencia del sistema

En la iteración actual, los nuevos requisitos para la gestión de los pagos implican nuevas colaboraciones con sistemas externos. Recordemos que los DSSs utilizan la notación de los diagramas de secuencia para representar las colaboraciones entre sistemas, tratando cada sistema como una caja negra. Es conveniente ilustrar los nuevos eventos del sistema mediante DSSs para aclarar:

- Las nuevas operaciones del sistema a las que el sistema de PDV NuevaEra necesitará dar soporte.
- Las solicitudes a otros sistemas, y las respuestas esperadas de estas solicitudes.

Inicio común del escenario Procesar Venta

El DSS para la parte del inicio del escenario básico incluye los eventos del sistema *crearNuevaVenta*, *introducirArticulo*, y *finalizarVenta*; esta parte es común, independientemente del método de pago (ver Figura 28.1).

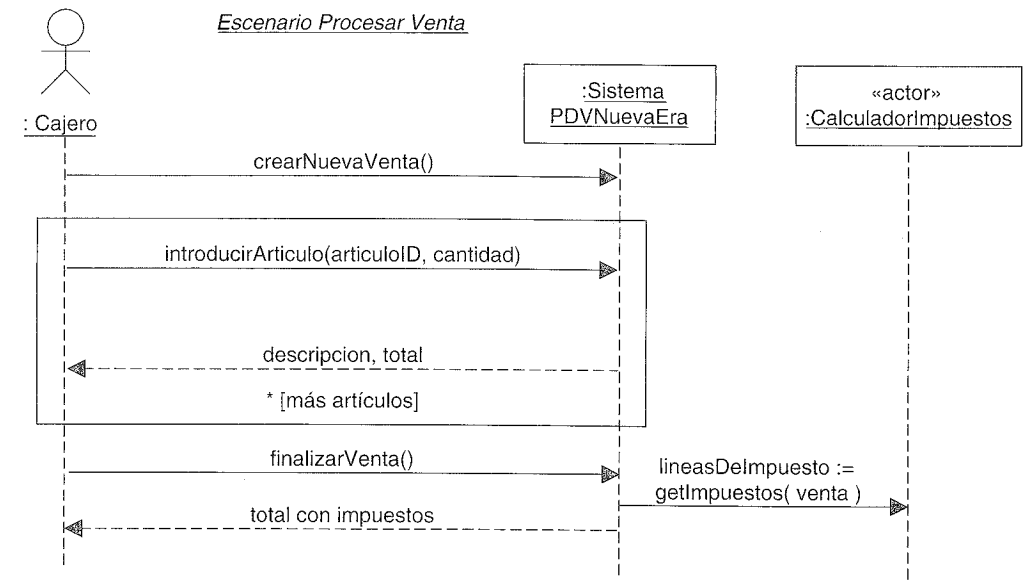


Figura 28.1. DSS del inicio común.

Pago a crédito

Este DSS del escenario de pago a crédito comienza después del inicio común (ver Figura 28.2).

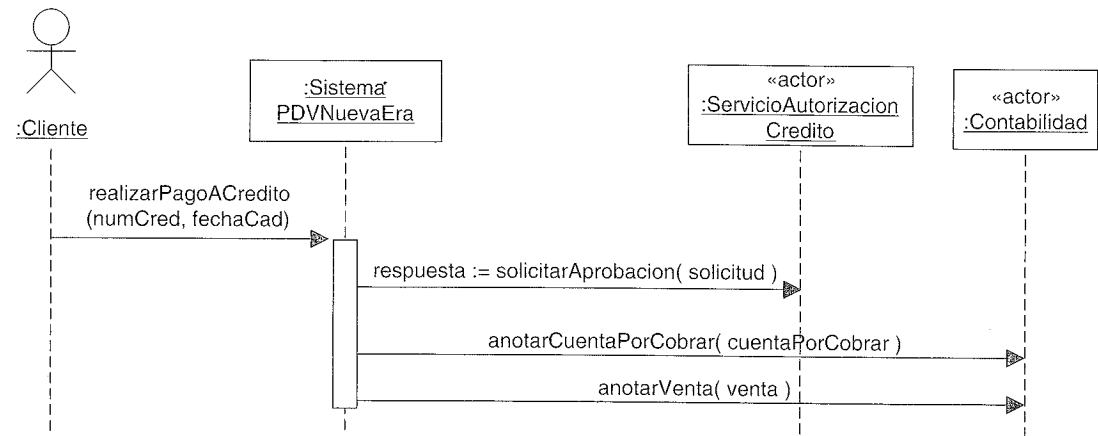


Figura 28.2. DSS del pago a crédito.

Tanto en el pago a crédito como en el pago con cheque, se asume por simplicidad (para esta iteración) que el pago es exactamente igual al total de la venta y, por tanto, no hay que pasar como parámetro una cantidad “entregada” diferente.

Nótese que la solicitud al *ServicioAutorizacionCredito* externo se modela como un mensaje síncrono ordinario con un valor de retorno. Esto es una abstracción; podría im-

plementarse con una solicitud SOAP sobre HTTPS seguros, o con cualquier mecanismo de comunicación remota. Los adaptadores de recursos que se definieron en la iteración anterior ocultarán el protocolo específico.

La operación del sistema *realizarPagoACredito* —y el caso de uso— asume que la información sobre el crédito del cliente procede de una tarjeta de crédito y, por tanto, se introduce en el sistema un número de cuenta de crédito y una fecha de caducidad (probablemente mediante un lector de tarjetas). Aunque admitimos que en el futuro surgirán mecanismos alternativos para comunicar la información acerca del crédito, la suposición del uso de tarjetas de crédito es muy estable.

Recordemos que cuando un servicio de autorización de crédito aprueba un pago a crédito, está en deuda con la tienda por el pago; por tanto, es necesario que se añada en el sistema de contabilidad una entrada de cuenta por cobrar.

Pago con cheque

El DSS para el escenario del pago con cheque se muestra en la Figura 28.3.

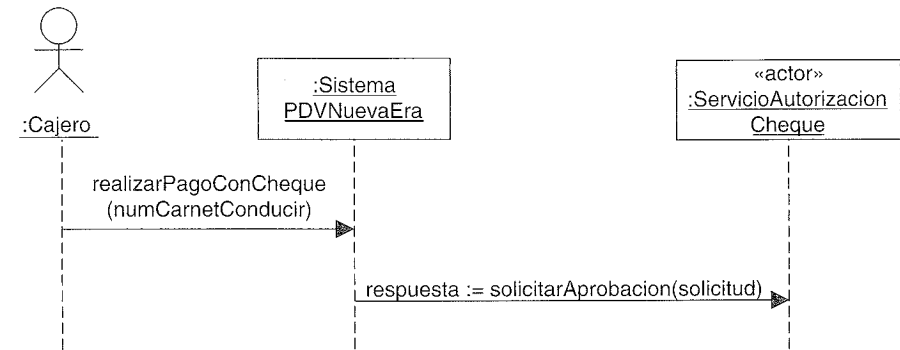


Figura 28.3. DSS del pago con cheque.

De acuerdo con el caso de uso, el cajero debe introducir el número del carnet de conducir para validarlo.

28.2. Nuevas operaciones del sistema

En esta iteración, las nuevas operaciones del sistema que debe gestionar el sistema son:

- *realizarPagoACredito*
- *realizarPagoConCheque*

En la primera iteración, el evento del sistema y la operación para el pago en efectivo era simplemente *realizarPago*. Ahora que los pagos son de diferentes tipos, se renombra como *realizarPagoEnEfectivo*.

28.3. Nuevos contratos de operaciones del sistema

Recordemos que los contratos de las operaciones del sistema son un artefacto de requisitos opcional (parte del Modelo de Casos de Uso) que añade detalles sutiles relativos a una operación del sistema. Algunas veces, el propio texto del caso de uso es suficiente, y no son necesarios estos contratos. Pero en ocasiones, resultan útiles por ser una manera precisa y detallada para identificar lo que ocurre cuando se invoca una operación compleja en el sistema, en términos de cambios de estado de los objetos definidos en el Modelo del Dominio.

A continuación presentamos los contratos de las nuevas operaciones del sistema:

Contrato CO5: realizarPagoACredito

Operación:	realizarPagoACredito(numCtaCredito, fechaCaducidad)
Referencias Cruzadas:	Casos de Uso: Procesar Venta
Precondiciones:	Existe una venta en curso y se han introducido todos los artículos.
Postcondiciones:	• Se creó un PagoACredito pg • pg se asoció con la Venta actual vta • se creó una TarjetaDeCredito tc; tc.numero = numCtaCredito, tc.fechaCaducidad = fechaCaducidad • tc se asoció con pg • se creó una SolicitudPagoACredito spc • pg se asoció con spc • se creó una EntradaCuentaPorCobrar ec • ec se asoció con el sistema externo de ContabilidadEntradasPorCobrar • vta se asoció con la Tienda como una venta completa

Obsérvese la postcondición que indica la asociación de una nueva entrada por cobrar en la contabilidad de cuentas por cobrar. Aunque esta responsabilidad está fuera de los límites del sistema NuevaEra, el sistema de contabilidad de cuentas por cobrar se encuentra dentro del control del negocio, por lo que se ha añadido la sentencia como comprobación de corrección.

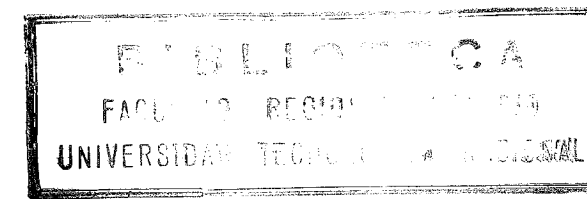
Por ejemplo, durante las pruebas, está claro a partir de esta postcondición que se debería comprobar que el sistema de contabilidad de cuentas por cobrar contiene una nueva entrada a cobrar.

Contrato CO6: realizarPagoConCheque

Operación:	realizarPagoConCheque(numCarnetConducir)
Referencias Cruzadas:	Casos de Uso: Procesar Venta
Precondiciones:	Existe una venta en curso y se han introducido todos los artículos.
Postcondiciones:	• Se creó un PagoConCheque pg • pg se asoció con la Venta actual vta • se creó un CarnetDeConducir cc; cc.numero = numCarnetConducir

Postcondiciones:
(continuación)

- cc se asocio con pg
- se creó una SolicitudPagoConCheque spc
- pg se asocio con spc
- vta se asocio con la Tienda como una venta completa



MODELADO DEL COMPORTAMIENTO CON DIAGRAMAS DE ESTADOS

La utilidad es como el oxígeno —nunca lo notas hasta que lo pierdes—.

Anónimo

Objetivos

- Crear diagramas de estados para las clases y los casos de uso.
-

Introducción

UML incluye la notación de los diagramas de estados para representar los eventos y los estados de las cosas —transacciones, casos de uso, personas, etcétera—. En esta introducción se presentan las características más importantes de la notación, pero hay otras que no se cubren.

Se destaca el uso de los diagramas de estados para mostrar los eventos del sistema en los casos de uso, pero podrían aplicarse adicionalmente a cualquier clase.

29.1. Eventos, estados y transiciones

Un **evento** es una ocurrencia significativa o relevante. Por ejemplo:

- Descolgar el teléfono.

Un **estado** es la condición de un objeto en un instante del tiempo —el tiempo entre eventos—. Por ejemplo:

- Un teléfono está en estado “inactivo” después de colgarlo y antes de descolgarlo.

Una **transición** es una relación entre dos estados que indica que cuando tiene lugar un evento, el objeto pasa del estado anterior al estado siguiente. Por ejemplo:

- Cuando tiene lugar el evento de “descolgar”, hay una transición del estado del teléfono de “inactivo” a “activo”.

29.2. Diagramas de estados

Un diagrama de estados UML, como se muestra en la Figura 29.1, representa los eventos y estados interesantes de un objeto, y el comportamiento de un objeto como reacción a un evento. Las transiciones se representan con flechas, etiquetadas con sus eventos. Los estados se representan en rectángulos de esquinas redondeadas. Es habitual incluir un pseudo-estado inicial, que pasa automáticamente a otro estado cuando se crea la instancia.

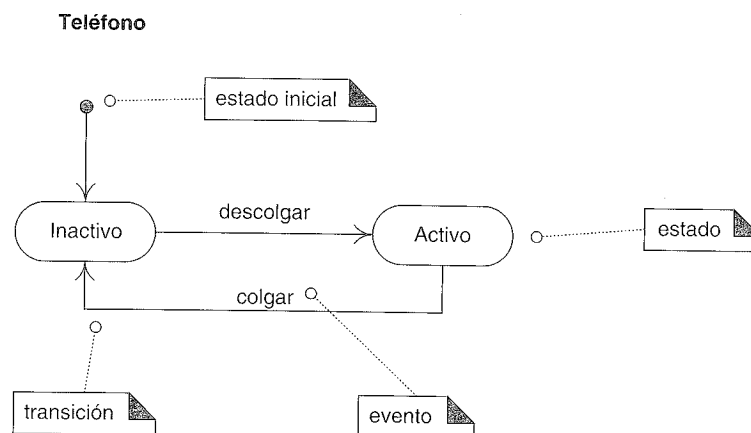


Figura 29.1. Diagrama de estados para un teléfono.

Un diagrama de estados muestra el ciclo de vida de un objeto: qué eventos experimenta, sus transiciones y los estados en los que se encuentra entre estos eventos. No es necesario ilustrar todos los posibles eventos; si surge un evento que no está representado en el diagrama, se ignora el evento por lo que al diagrama de estados se refiere. Por tanto, podemos crear un diagrama de estados que describa el ciclo de vida de un objeto a un nivel de detalle arbitrariamente simple o complejo, dependiendo de nuestras necesidades.

Aplicaciones de los diagramas de estados

Un diagrama de estados podría aplicarse a una variedad de elementos UML, entre los que se encuentran:

- Las clases (conceptuales o de software).
- Los casos de uso.

Puesto que un “sistema” completo se podría representar mediante una clase, también podría tener su propio diagrama de estados.

29.3. ¿Diagramas de estados en el UP?

No existe en el UP ningún modelo que se llame “modelo de estados”. Más bien, cualquier elemento de cualquier modelo (Modelo de Diseño, Modelo del Dominio, etcétera) podría tener una máquina de estados para entenderlo mejor o para comunicar su comportamiento dinámico como respuesta a los eventos. Por ejemplo, una máquina de estados asociada a la clase de diseño *Venta* del Modelo de Diseño también forma parte del Modelo de Diseño.

29.4. Diagramas de estados de casos de uso

Una aplicación útil de los diagramas de estados es la descripción de la secuencia legal de eventos del sistema externo que reconoce y maneja un sistema en el contexto de un caso de uso. Por ejemplo:

- Durante el caso de uso *Procesar Venta* en la aplicación del PDV NuevaEra, no es legal llevar a cabo la operación *realizarPagoACredito* hasta que haya tenido lugar el evento *finalizarVenta*.
- Durante el caso de uso de *Procesar Documento* en un procesador de texto, no es legal ejecutar la operación Guardar-Fichero hasta que haya tenido lugar el evento Nuevo-Fichero o Abrir-Fichero.

Un diagrama de estados que describe los eventos del sistema global y sus secuencias en un caso de uso es una especie de **diagrama de estados de casos de uso**. El diagrama de estados de casos de uso de la Figura 29.2 muestra una versión simplificada de los eventos del sistema para el caso de uso *Procesar Venta* en la aplicación del PDV. Ilustra que no es legal generar un evento *realizarPago* si previamente el evento *finalizarVenta* no ha causado la transición del sistema al estado *EsperandoPago*.

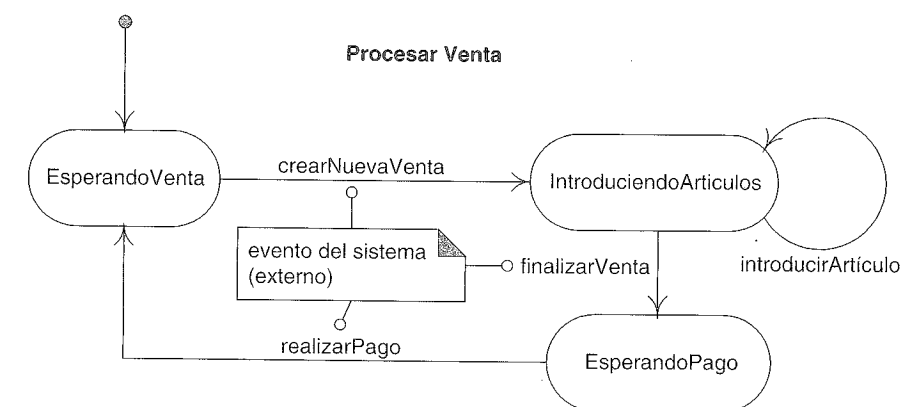


Figura 29.2. Diagrama de estados de caso de uso para *Procesar Venta*.

Utilidad de los diagramas de estados de casos de uso

El número de eventos del sistema y su orden legal para el caso de uso *Procesar Venta* son (hasta el momento) relativamente triviales; por tanto, podría no ser necesario el uso de un

diagrama de estados para mostrar la secuencia válida. Pero para un caso de uso complejo, con innumerables eventos del sistema —como cuando se utiliza un procesador de texto— resulta útil utilizar un diagrama de estados que ilustre el orden válido de los eventos externos.

Veamos cómo: durante el trabajo de diseño e implementación, es necesario crear e implementar un diseño que asegure que no ocurran eventos fuera de la secuencia establecida, de otra manera podría producirse una condición de error. Por ejemplo, no se debería permitir al sistema que reciba un pago a menos que se complete una venta; se debe escribir el código que garantice eso.

Proporcionando un conjunto de diagramas de estados, un diseñador puede desarrollar metódicamente un diseño que asegure el orden correcto de los eventos del sistema. Entre las posibles soluciones de diseño se encuentran:

- estructura condicional en el código para comprobar que los eventos ocurren en el orden correcto
- utilizar el patrón *Estado* (que se presentará en un capítulo posterior)
- deshabilitar los elementos gráficos de las ventanas activas para rechazar los eventos no válidos (un enfoque deseable)
- un intérprete de máquinas de estado que ejecuta una tabla de estados que representa un diagrama de estados de casos de uso.

En un dominio con muchos eventos del sistema, la concisión y minuciosidad de los diagramas de estados de casos de uso ayudan al diseñador a asegurar que no se ha omitido nada.

29.5. Diagramas de estados de casos de uso para la aplicación del PDV

Procesar Venta

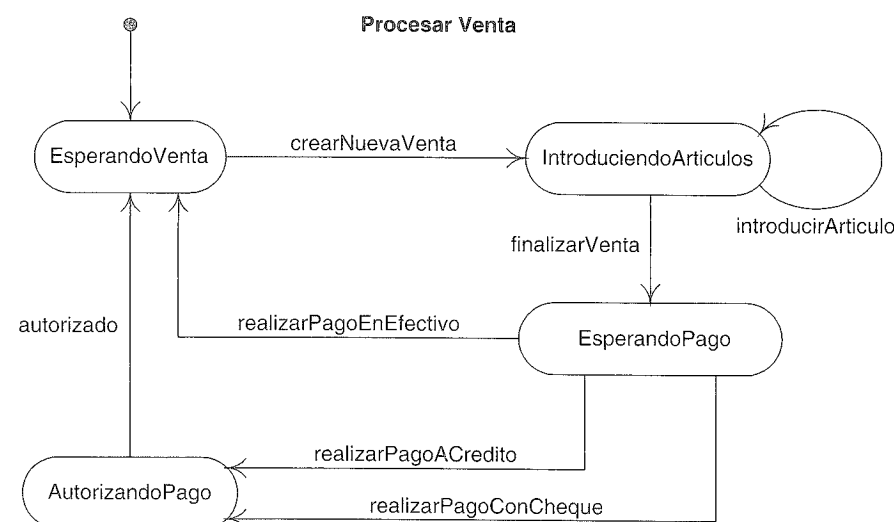


Figura 29.3. Un diagrama de estados de muestra.

29.6. Clases que se benefician de los diagramas de estados

Además de crearse los diagramas de estados para los casos de uso o el sistema global, podrían crearse prácticamente para cualquier tipo o clase.

Objetos dependientes e independientes del estado

Si un objeto siempre responde de la misma manera a un evento, entonces se considera **independiente-del-estado** (o sin modo, *modeless*) con respecto al evento. Por ejemplo, si un objeto recibe un mensaje, y el método que responde siempre hace lo mismo —el método no tendrá normalmente ninguna lógica condicional. El objeto es independiente del estado con respecto al mensaje. Si, para todos los eventos de interés, un objeto siempre reacciona de la misma manera, es un **objeto independiente-del-estado**. En cambio, los **objetos dependientes-del-estado** reaccionan de manera diferente a los eventos dependiendo de su estado.

Cree máquinas de estado para los objetos dependientes del estado con un comportamiento complejo.

En general, en los sistemas de información de gestión las clases realmente dependientes del estado son una minoría. En cambio, los dominios de control de procesos y telecomunicaciones con frecuencia tienen muchos objetos dependientes del estado.

Clases dependientes del estado comunes

A continuación presentamos una lista de objetos comunes que normalmente son dependientes del estado, y para los que podría ser útil crear un diagrama de estados:

- **Casos de uso**
 - Visto como una clase, el caso de uso *Procesar Venta* reacciona de manera diferente al evento *finalizarVenta* dependiendo de si la venta está en curso o no.
- **Sesiones con estado:** son objetos software del lado del servidor que representan sesiones en marcha o conversaciones con un cliente; por ejemplo, los objetos sesión con estado (*stateful*) EJB.
 - Otro ejemplo muy común es la gestión del lado del servidor de aplicaciones web del cliente y la lógica del flujo de la presentación; por ejemplo, un servlet de Java *helper* o “controlador” que recuerda el estado de la sesión con un cliente Web, y controla las transiciones a nuevas páginas web, o las modificaciones en la información que muestra la página web actual, basado en el estado de la sesión o la conversación.
 - Una sesión con estado normalmente se suele ver como una clase software que representa un caso de uso. Recordemos que uno de las variantes del patrón GRASP Controlador es un controlador de casos de uso; que es un objeto de sesión con estado de caso de uso.

- **Sistemas:** Es una clase que representa la aplicación o sistema global.
 - El “*sistema de PDV*.”
- **Ventanas**
 - La acción Editar-Pegar sólo es válida si hay algo en el “portapapeles” para pegar.
- **Controladores:** Son objetos controlador GRASP.
 - La clase *Registro*, que maneja los eventos del sistema *introducirArticulo* y *finalizarVenta*.
- **Transacciones:** Las formas en las que reacciona una transacción (una venta, pedido, pago) a un evento a menudo dependen del estado actual en el que se encuentra dentro del ciclo de vida global.
 - Si una *Venta* recibe el mensaje *crearLineaDeVenta* después del evento *finalizarVenta*, debería dar lugar a una condición de error o ignorarlo.
- **Dispositivos**
 - La TV, el microondas: reaccionan de manera diferente a un evento particular dependiendo de su estado actual.
- **Cambiador de Rol:** Son clases que cambian de rol.
 - Una *Persona* que cambia de rol pasa de ser un civil a ser un militar.

29.7. Representación de eventos externos e internos

Tipos de eventos

Es útil clasificar los eventos como sigue:

- **Evento externo:** También conocido como evento del sistema, lo origina algo fuera de los límites del sistema (por ejemplo, un actor). Los DSSs muestran los eventos externos. Los eventos externos relevantes originan la invocación de las operaciones del sistema para responder a ellos.
 - Cuando un cajero presiona el botón “introducir artículo” en un terminal de PDV, ha ocurrido un evento externo.
- **Evento interno:** Causado por algo de dentro de los límites de nuestro sistema. Por lo que se refiere al software, un evento interno surge cuando se invoca a un método por medio de un mensaje o señal que fue enviada desde otro objeto interno. Los mensajes de los diagramas de interacción sugieren eventos internos.
 - Cuando una *Venta* recibe un mensaje *crearLineaDeVenta*, ha ocurrido un evento interno.
- **Evento de tiempo:** Causado por la ocurrencia de una fecha y hora específicas o el paso del tiempo. En cuanto al software, un evento temporal lo dirige un reloj de tiempo real o de tiempo simulado.
 - Suponga que después de que tenga lugar la operación *finalizarVenta*, debe ocurrir la operación *realizarPago* en cinco minutos, en otro caso la venta actual se elimina automáticamente.

Diagramas de estados para eventos internos

Un diagrama de estados puede mostrar eventos *internos* que representan generalmente los mensajes que recibe desde otros objetos. Puesto que los diagramas de interacción también muestran los mensajes y sus reacciones (en función de otros mensajes), ¿por qué utilizar un diagrama de estados para ilustrar eventos internos y el diseño de objetos? El paradigma del diseño de objetos es aquel en el que los objetos colaboran mediante mensajes para llevar a cabo las tareas; los diagramas de interacción UML ilustran de una forma evidente ese paradigma. Es algo incoherente utilizar diagramas de estados para mostrar un diseño del paso de mensajes e interacciones entre objetos¹.

En consecuencia, tengo mis reservas sobre recomendar el uso de diagramas de estados para mostrar los eventos internos con el fin de obtener un diseño de objetos creativo². Sin embargo, podrían ser útiles para resumir los resultados de un diseño, después de que se complete.

En cambio, como explicó la discusión anterior sobre los diagramas de estados de casos de uso, un diagrama de estados podría ser una herramienta útil y concisa para los eventos *externos*.

Es preferible utilizar los diagramas de estados para ilustrar los eventos externos y de tiempo, y las reacciones a ellos, en lugar de utilizarlas para diseñar el comportamiento de los objetos basado en los eventos internos.

29.8. Notación adicional de los diagramas de estados

La notación UML para los diagramas de estados contiene un conjunto rico de características que no se han utilizado en esta introducción. Tres características significativas son:

- Acciones de la transición
- Condiciones de guarda de la transición
- Estados anidados

Acciones y condiciones de guarda de la transición

Una transición puede provocar que se dispare una acción. En una implementación software, esto podría representar la invocación de un método de la clase del diagrama de estados.

¹ Un lector de literatura sobre el A/DOO encontrará ejemplos en publicaciones y libros de texto de diagramas de estados complejos que se dedican a los eventos *internos* y las reacciones de los objetos a ellos. Esencialmente, sus creadores han reemplazado el paradigma de interacción y colaboración de objetos mediante mensajes por el paradigma de objetos como máquinas de estados, y han utilizado diagramas de estados para diseñar el comportamiento de los objetos, en lugar de utilizar diagramas de colaboración. De manera abstracta, las dos visiones son equivalentes.

² Es razonable utilizar los diagramas de estados para mostrar el diseño de objetos basado en los eventos internos, cuando se va a obtener el código con un generador de código que está dirigido por los diagramas de estados, o cuando se utiliza un intérprete de máquinas de estados para ejecutar el sistema software.

Una transición podría tener también una condición de guarda —o condición booleana—. Sólo ocurre la transición si se cumple la condición.

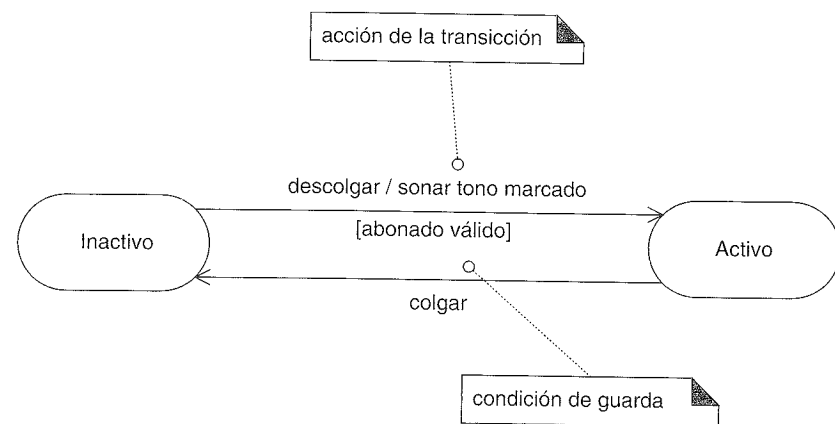


Figura 29.4. Notación para una acción y condición de guarda de una transición.

Estados anidados

Un estado permite el anidamiento para contener subestados; un subestado hereda la transición de su superestado (el estado que lo incluye). Ésta es una contribución clave de la notación de los diagramas de estados de Harel en la que se basa UML, lo que nos lleva a diagramas de estados concisos. Los subestados se podrían mostrar gráficamente anidándolos en una caja que representa el superestado.

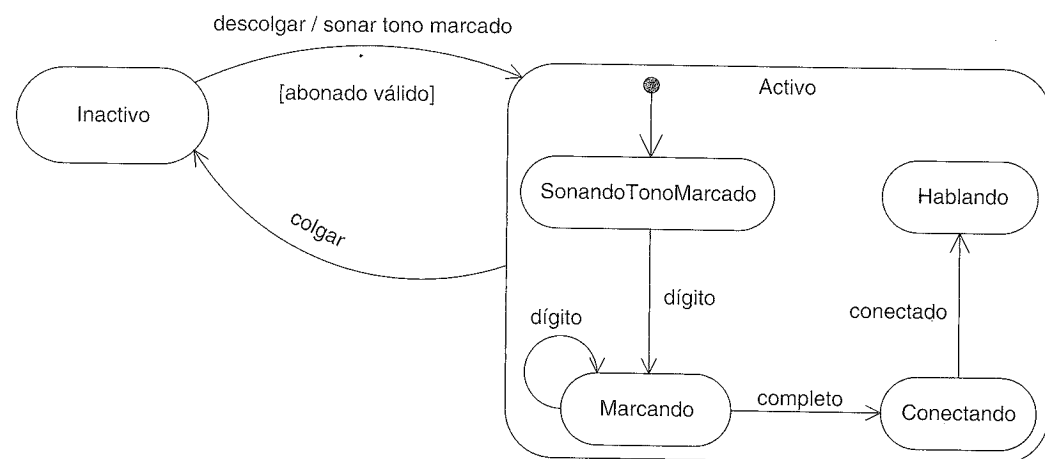


Figura 29.5. Estados anidados.

Por ejemplo, cuando ocurre una transición al estado *Activo*, tiene lugar la creación y transición al subestado *SonandoTonoMarcado*. No importa el subestado en el que se encuentre el objeto, si tiene lugar el evento *colgar* relacionado con el superestado *Activo*, tiene lugar una transición al estado *Inactivo*.

29.9. Lecturas adicionales

La aplicación de los modelos de estados en el A/DOO se cubre bien en *Designing Object Systems* de Cook y Daniels. También *Doing Hard Time* de Douglas proporciona una discusión excelente acerca del modelado de estados; el contenido se centra en los sistemas en tiempo real, pero se puede aplicar en general.

DISEÑO DE LA ARQUITECTURA LÓGICA CON PATRONES

0x2B \ ~0x2B

Hamlet

Objetivos

- Diseñar una arquitectura lógica en función de capas y particiones con el patrón Capas (*Layers*).
 - Ilustrar la arquitectura lógica utilizando los diagramas de paquetes de UML.
 - Aplicar los patrones Fachada, Observador y Controlador.
-

Introducción

Lo primero de todo, para fijar las expectativas, decir que este capítulo es una *introducción* al tema de la arquitectura lógica, un tema realmente extenso.

Las iteraciones anteriores se centraron en un grupo de objetos software del “dominio” estrechamente relacionados, en el Modelo del Diseño (como *Venta* y *Pago*). No se prestó atención al interfaz de usuario o al acceso a recursos como una base de datos. El motivo era mantener las cosas simples y centrarse en las técnicas fundamentales del diseño de objetos.

Sin embargo, un sistema típico está compuesto de muchos paquetes lógicos, como un paquete de interfaz de usuario, un paquete de acceso a bases de datos, etcétera. Cada paquete agrupa un conjunto cohesivo de responsabilidades (ej. el acceso a bases de datos). Esta es la práctica básica de aplicar la modularidad para dar soporte a la separación de intereses.

Este capítulo presenta brevemente las arquitecturas lógicas, y la comunicación y acoplamiento entre los paquetes.

30.1. Arquitectura del software

Una definición de **arquitectura del software** es:

Una arquitectura es el conjunto de decisiones significativas sobre la organización del sistema software, la selección de los elementos estructurales y sus interfaces, con los que se compone el sistema, junto con su comportamiento tal como se especifica en las colaboraciones entre esos elementos, la composición de esos elementos estructurales y de comportamiento en subsistemas progresivamente más amplios, y el estilo de arquitectura que guía esta organización —estos elementos y sus interfaces, sus colaboraciones, y su composición—. [BRJ99]

Independientemente de la definición (y hay muchas) el tema común en todas las definiciones de arquitectura del software es que tiene que ver con la gran escala —las Grandes Ideas en las influencias, organización, estilos, patrones, responsabilidades, colaboraciones, conexiones y motivaciones de un sistema (o un sistema de sistemas), y los subsistemas importantes—.

En el desarrollo de software, arquitectura se considera tanto un nombre como un verbo.

Como nombre, la arquitectura comprende —como indica la definición anterior— la organización y estructura de los elementos importantes del sistema. Más allá de esta definición estática, incluye el comportamiento del sistema, especialmente en función de responsabilidades de gran escala de los sistemas y subsistemas, y sus colaboraciones. En cuanto a una descripción, la arquitectura comprende las *motivaciones* o fundamentos de por qué el sistema está diseñado de la forma que está.

Como verbo, la arquitectura es parte investigación y parte trabajo de diseño; por claridad, el término es mejor que se califique como en investigación arquitectural o diseño arquitectural.

La **investigación arquitectural** implica la identificación de aquellos requisitos funcionales y no-funcionales que influyen (o deberían influir) de manera significativa en el diseño del sistema, como tendencias del mercado, rendimiento, coste, mantenimiento, y puntos de evolución. Ampliamente, se trata del análisis de requisitos centrado en aquellos requisitos que tienen una influencia especial en las decisiones de diseño del sistema más importantes.

El **diseño arquitectural** es la resolución de estas influencias y requisitos en el diseño del software, el hardware y las redes, operaciones, políticas, etcétera.

En el UP, el diseño y la investigación de la arquitectura se llaman conjuntamente **análisis arquitectural**, cuyo proceso se introducirán brevemente en el Capítulo 32.

Dimensiones y vistas de la arquitectura en el Proceso Unificado

La arquitectura de un sistema abarca varias dimensiones. Por ejemplo:

- La arquitectura lógica, que describe el sistema en términos de su organización conceptual en capas, paquetes, frameworks importantes, clases, interfaces y subsistemas.

- El despliegue de la arquitectura, que describe el sistema en términos de la asignación de los procesos a unidades de proceso, y la configuración de la red.

El Proceso Unificado sugiere seis vistas de la arquitectura (lógica, despliegue, etcétera), todas ellas se definirán en el Capítulo 32.

Este capítulo se centra en la vista lógica de la arquitectura.

Patrones de arquitectura y categorías de patrones

Existen buenas prácticas bien conocidas en el diseño arquitectural, especialmente en cuanto a la arquitectura lógica a gran escala, y estas prácticas se han escrito en forma de patrones, como el patrón Capas (*Layers*). El primer libro que se dedicó al tema de los patrones de arquitectura fue *Pattern-Oriented Software Architecture* (POSA) [BMRSS96].

El libro POSA también ofrece una clasificación de los patrones sencilla y útil, a diferentes niveles:

1. **Patrones de arquitectura:** Relacionados con el diseño a gran escala y de grano grueso, y que se aplican típicamente durante las primeras iteraciones (la fase de elaboración) cuando se establecen las estructuras y conexiones más importantes.
 - Los patrones Capas, que estructuran el sistema en las principales capas.
2. **Patrones de diseño:** Relacionados con el diseño de los objetos y frameworks de pequeña y mediana escala. Aplicables al diseño de una solución para conectar los elementos de gran escala que se definen mediante los patrones de arquitectura, y durante el trabajo de diseño detallado para cualquier aspecto del diseño local. También se conocen como patrones de micro-arquitectura.
 - El patrón Fachada, que se puede utilizar para proporcionar la interfaz de una capa a la siguiente.
 - El patrón Estrategia, para permitir algoritmos conectables.
3. **Estilos:** Soluciones de diseño de bajo nivel orientadas a la implementación o al lenguaje.
 - El patrón Singleton, para asegurar el acceso global a una única instancia de una clase.

Este capítulo se centra en los patrones de arquitectura y la aplicación de los patrones de diseño para realizar las conexiones entre las estructuras a gran escala.

Existen otras categorías de patrones. Las categorías del POSA forman una nítida tríada, y son útiles para muchos patrones, pero no cubre la gama completa de patrones

que se han publicado. Aun con el riesgo de simplificar demasiado, un patrón es la repetición de las mejores prácticas de lo que funciona —en cualquier dominio—. Otras categorías de patrones publicadas comprenden:

- Patrones del proceso del desarrollo de software y organizacionales.
- Patrones de interfaz de usuario.
- Patrones de pruebas.

30.2. Patrón de arquitectura: Capas (Layers)

Solución Las ideas esenciales del patrón Capas [BMRSS96] son simples:

- Organizar la estructura lógica de gran escala de un sistema en capas separadas de responsabilidades distintas y relacionadas, con una separación clara y cohesiva de intereses como que las capas “más bajas” son servicios generales de bajo nivel, y las capas más altas son más específicas de la aplicación.
- La colaboración y el acoplamiento es desde las capas más altas hacia las más bajas; se evita el acoplamiento de las capas más bajas a las más altas.

Una capa es un elemento de gran escala, a menudo compuesto de varios paquetes o subsistemas.

El patrón Capas se relaciona con la arquitectura lógica; es decir, describe la organización conceptual de los elementos del diseño en grupos, independiente de su empaquetamiento o despliegue físico.

Las Capas definen un modelo general de N-niveles para la arquitectura lógica; produce una **arquitectura en capas**. Se lleva tanto tiempo aplicando y escribiendo sobre esto como un patrón que en *Pattern Almanac 2000* [Rising00] se listan alrededor de 100 patrones que son variantes o están relacionados con el patrón Capas.

- Problemas**
- Los cambios del código fuente se propagan a lo largo de todo el sistema —muchas partes del sistema están altamente acopladas—.
 - La lógica de la aplicación se entrelaza con la interfaz de usuario, entonces no se puede reutilizar con una interfaz diferente, ni distribuirse a otro nodo de proceso.
 - La lógica más específica de la aplicación se entrelaza con los servicios técnicos o la lógica del negocio potencialmente generales, entonces no se puede reutilizar, distribuir a otro nodo o reemplazar fácilmente con una implementación diferente.
 - Existe un alto acoplamiento entre diferentes áreas de interés. Esto es por lo que es difícil dividir el trabajo para diferentes desarrolladores mediante límites claros.
 - Debido al alto acoplamiento y la mezcla de intereses, es difícil que la funcionalidad evolucione, que el sistema crezca de forma proporcionada o que se actualice para utilizar nuevas tecnologías.

Ejemplo El objetivo y número de las capas varía de una aplicación a otra y entre dominios de aplicación (sistemas de información, sistemas operativos, etcétera). Aplicado a los sistemas de información, las capas típicas se ilustran y explican en la Figura 30.1.

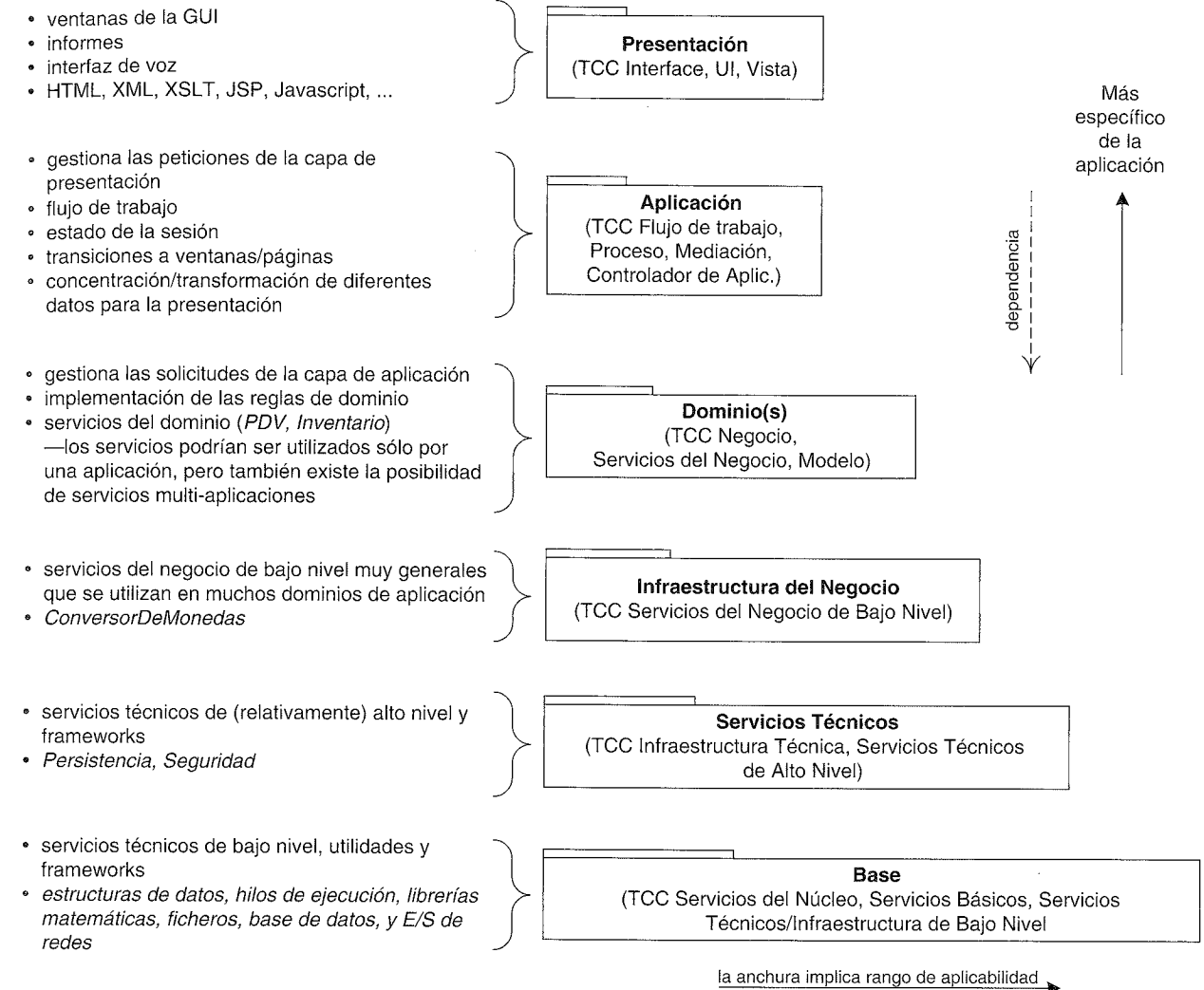


Figura 30.1. Capas comunes en una arquitectura lógica de un sistema de información¹.

Basado en estos arquetipos, la Figura 30.2 ilustra una arquitectura lógica en capas parcial de la aplicación NuevaEra.

Notación UML: Los diagramas de paquetes se utilizan para representar las capas. En UML una capa es simplemente un paquete.

Obsérvese que para esta iteración del diseño no existe una capa de Aplicación; como se discutirá después, no siempre es necesario.

Puesto que esto es un desarrollo iterativo, es normal crear un diseño de capas que comience siendo simple y evolucione a lo largo de las iteraciones de la fase de elabo-

¹ La anchura del paquete se utiliza para comunicar el rango de aplicabilidad en este diagrama, pero no es una práctica general en UML. TCC significa “también conocido como”.

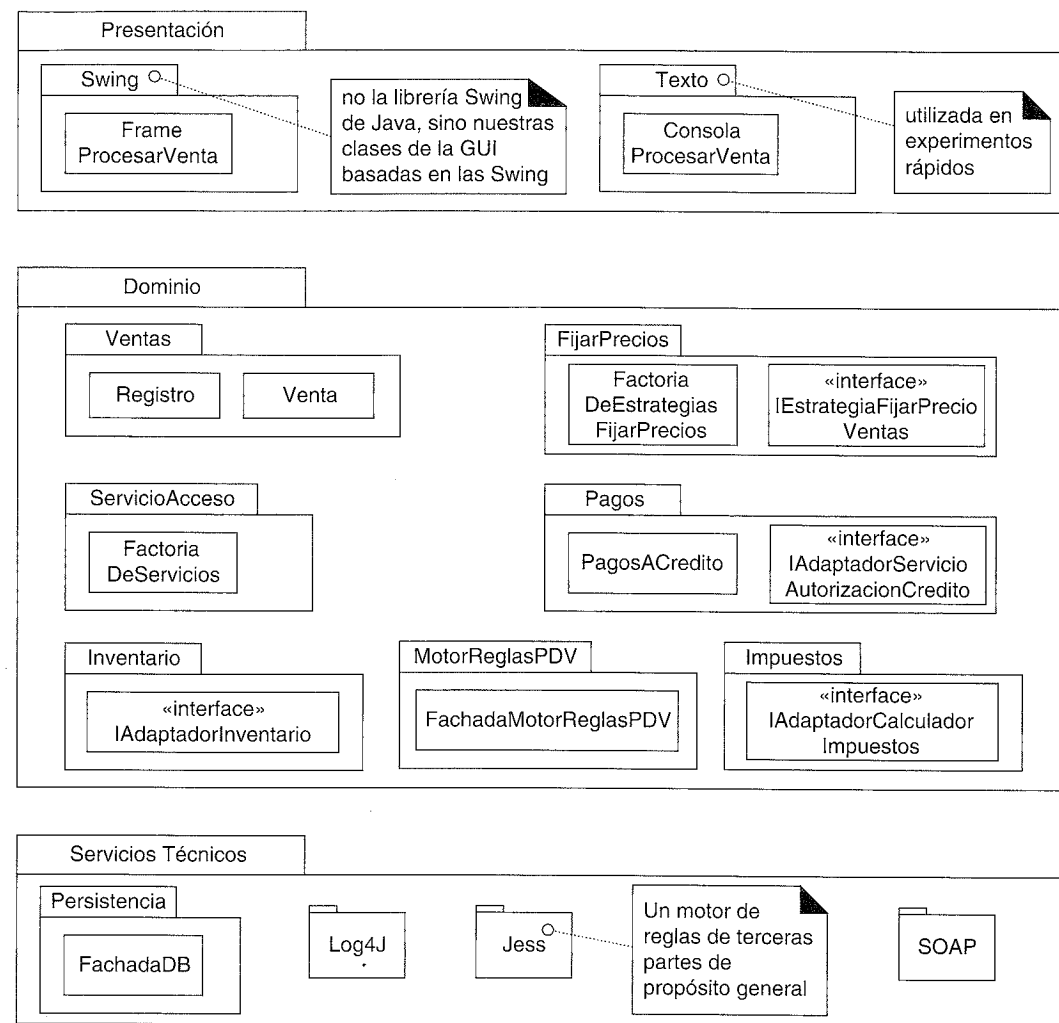


Figura 30.2. Vista lógica parcial de las capas en la aplicación NuevaEra.

ración. Uno de los objetivos de esta fase es establecer la arquitectura básica (diseñarla e implementarla) al final de las iteraciones de la elaboración, pero esto no significa realizar un diseño especulativo detallado de la arquitectura por adelantado, antes de empezar a programar. Más bien, se diseña una arquitectura lógica tentativa en las primeras iteraciones, que evolucionará incrementalmente a lo largo de la fase de elaboración.

Obsérvese que en este diagrama de paquetes sólo se presentan unos pocos tipos de elementos como muestra; esto no sólo está motivado por las limitaciones de espacio al dar formato al libro, sino que es una señal de calidad de un diagrama de la **vista de la arquitectura** —sólo muestra unos pocos elementos relevantes para transmitir de manera concisa las ideas importantes de los aspectos más significativos de la arquitectura—. La idea de un documento de la vista de la arquitectura del UP es decirle al lector, “He elegido este pequeño conjunto de elementos instructivos para transmitir las grandes ideas”.

Comentarios acerca del diagrama:

- Existen otros tipos en estos paquetes; sólo se muestran unos pocos para indicar los aspectos relevantes.
- No se mostró en esta vista la capa Base; el arquitecto (yo) decidió que no añadía información interesante, aunque el equipo de desarrollo, con seguridad, añadirá algunas clases Base, como utilidades avanzadas para la manipulación de *Strings*.
- Hasta el momento, no se utiliza una capa de Aplicación separada. Las responsabilidades de control o los objetos de sesión de la capa de aplicación las maneja el objeto *Registro*. El arquitecto añadirá una capa de Aplicación en una iteración posterior cuando crezca la complejidad del comportamiento y se introduzcan interfaces alternativas para los clientes (como un navegador web y un PDA portátil de red sin cable).

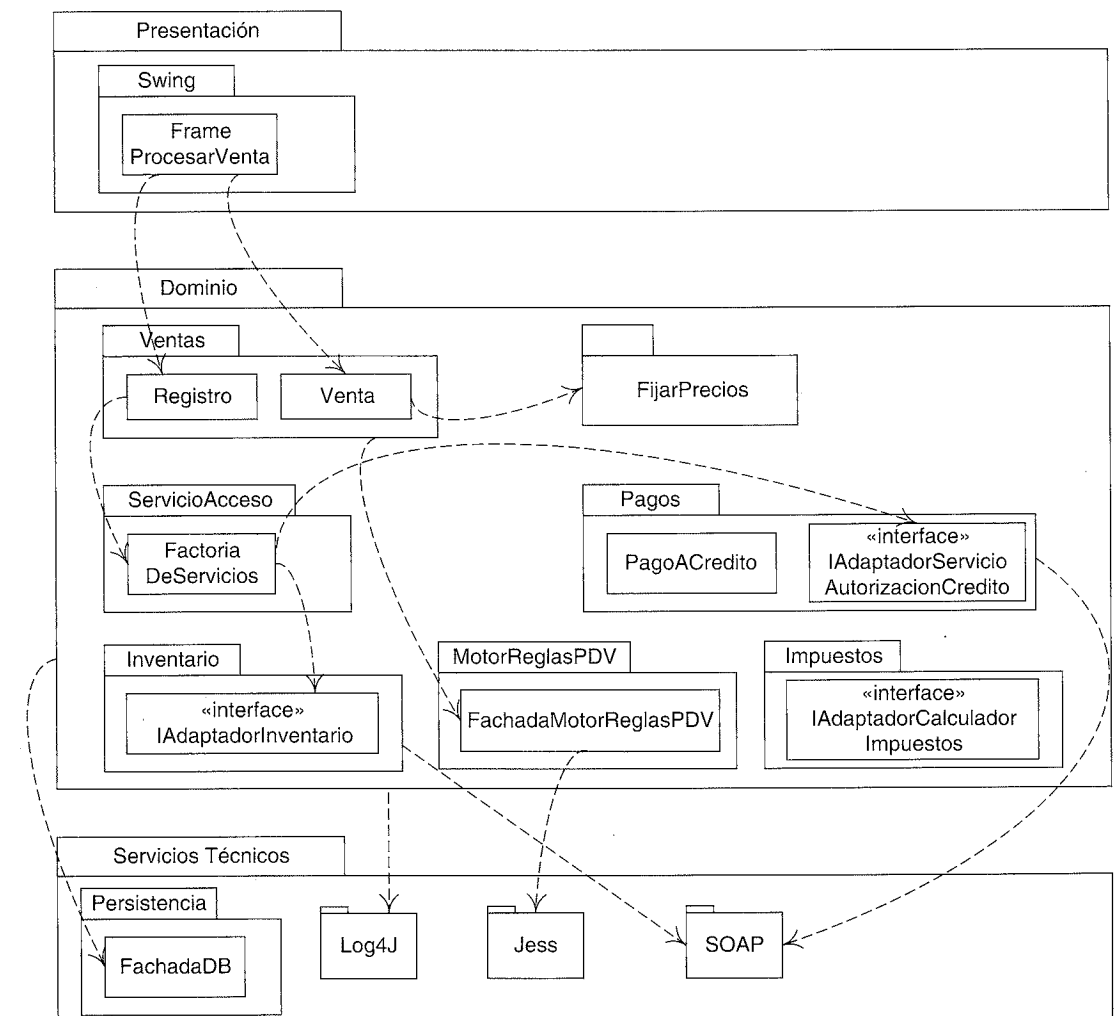


Figura 30.3. Acoplamiento parcial entre paquetes.

Acoplamiento entre capas y entre paquetes

También proporciona información la inclusión de un diagrama en la vista lógica que muestre el acoplamiento relevante entre las capas y los paquetes. La Figura 30.3 ilustra un ejemplo parcial.

Notación UML:

- Obsérvese que se pueden utilizar líneas de dependencia para mostrar el acoplamiento entre los paquetes o los tipos de los paquetes. Es conveniente utilizar simples líneas de dependencia cuando al comunicador no le preocupa especificar la dependencia exacta (visibilidad de atributo, subclase,...), sino simplemente quiere resaltar las dependencias.
- Nótese también el uso de una línea de dependencia que sale de un paquete en lugar de desde un tipo específico, como desde el paquete *Ventas* a la clase *FachadaMotorReglasPDV*, y del paquete del *Dominio* al paquete *Log4J*. Esto es útil cuando o bien no es interesante el tipo concreto del que depende, o bien el comunicador quiere dar a entender que podrían compartir la dependencia muchos elementos del paquete.

Otro uso común del diagrama de paquetes es ocultar los tipos específicos, y centrarse en ilustrar el acoplamiento paquete-paquete, como en el diagrama parcial de la Figura 30.4.

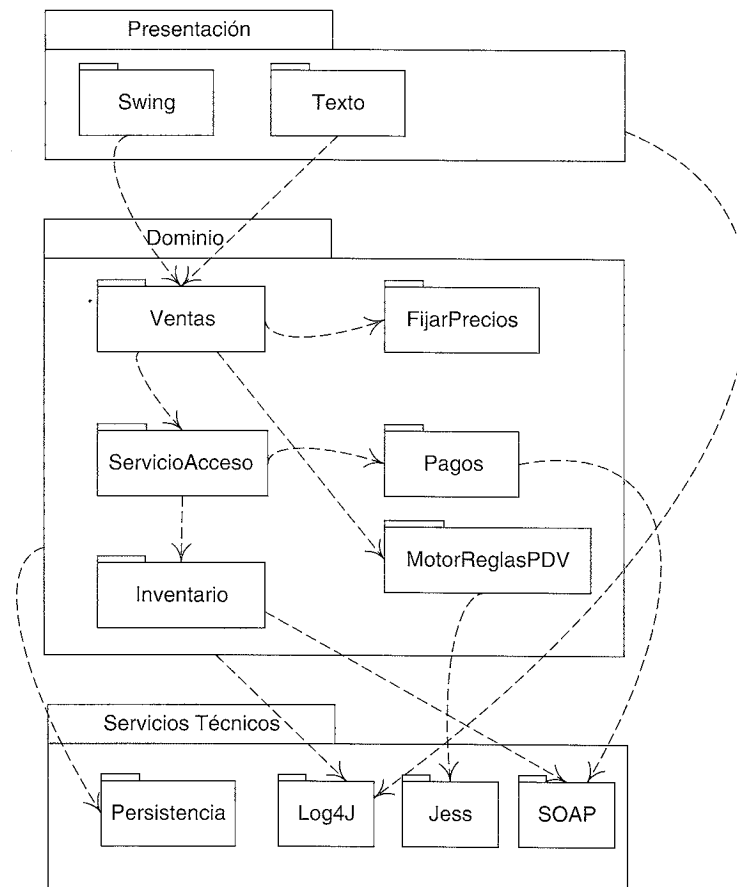


Figura 30.4. Acoplamiento parcial entre los paquetes.

De hecho, la Figura 30.4 ilustra probablemente el estilo más común del diagrama de la arquitectura lógica en UML —un diagrama de paquetes que normalmente muestra de 5 a 20 paquetes importantes, y sus dependencias—.

Escenarios de interacción entre capas y entre paquetes

Los diagramas de paquetes muestran información estática. Un diagrama de interacción proporciona la información para entender la dinámica del modo en el que se conectan y comunican los objetos entre las capas. En el espíritu de una “vista de la arquitectura” que oculta los detalles irrelevantes, y destaca lo que la arquitectura quiere transmitir, un diagrama de interacción en la vista lógica de la arquitectura se centra en las colaboraciones que cruzan los límites de las capas y los paquetes. Por tanto, es útil contar con un conjunto de diagramas de interacción que ilustren los **escenarios más significativos desde el punto de vista de la arquitectura** (en el sentido de que ilustran muchos aspectos de gran escala o grandes ideas del diseño).

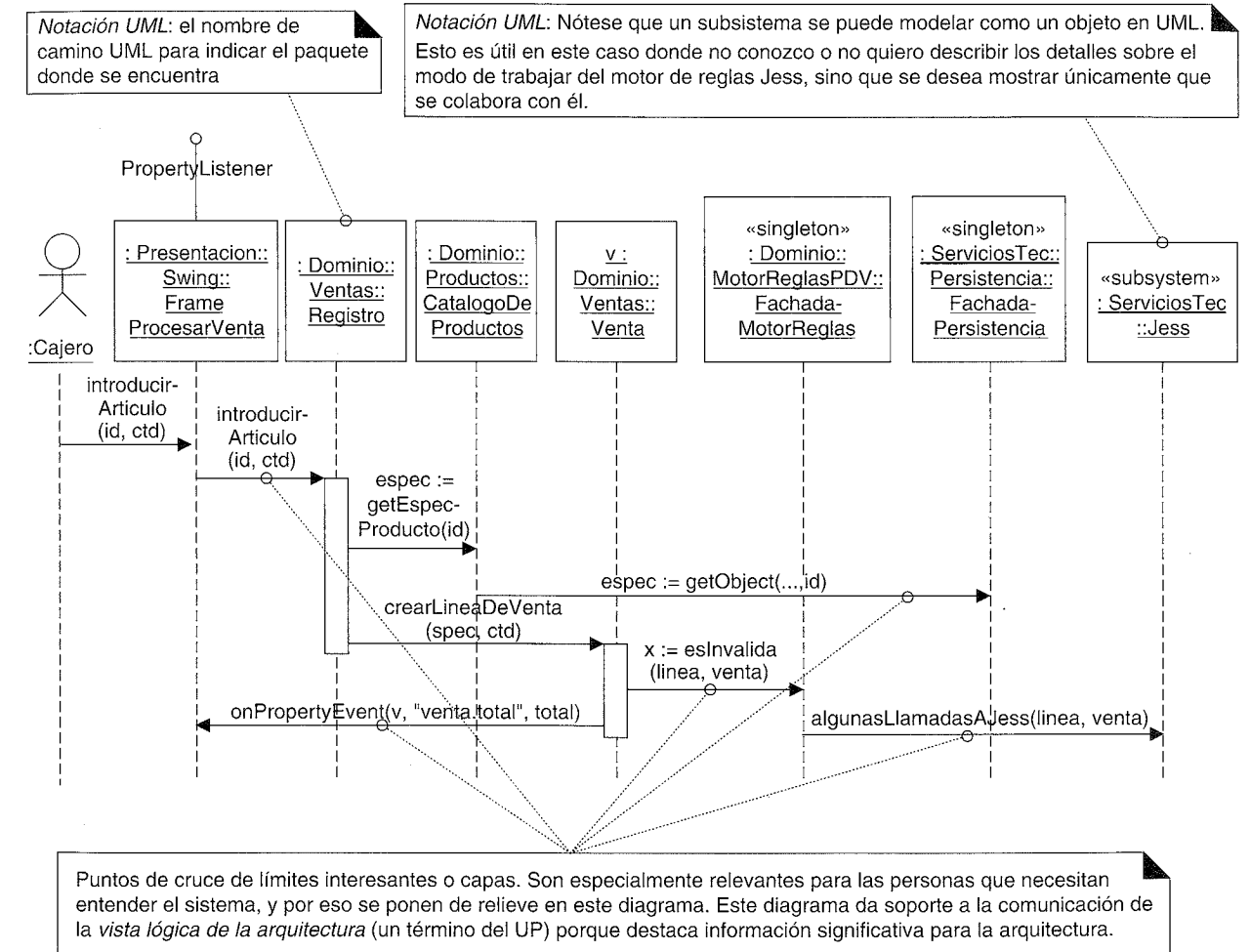


Figura 30.5. Un diagrama de interacción significativo desde el punto de vista de la arquitectura que pone de relieve las conexiones que cruzan los límites.

Por ejemplo, la Figura 30.5 ilustra parte del escenario *Procesar Venta* que pone de relieve los puntos de conexión entre las capas y los paquetes.

Notación UML:

- El paquete al que pertenece un tipo se puede mostrar opcionalmente calificando el tipo con la expresión del **nombre de camino** de UML `<NombrePaquete>::<NombreTipo>`. Por ejemplo, *Dominio::Ventas::Registro*. Esto se puede aprovechar para destacarle al lector las conexiones entre los paquetes y entre las capas en el diagrama de interacción.
- Nótese también el uso del estereotipo «subsystem». En UML, un subsistema es una entidad discreta que tiene comportamiento e interfaces. Se puede modelar un subsistema como un tipo especial de paquete, o —como se muestra aquí— como un objeto, que es útil cuando uno quiere mostrar las colaboraciones entre subsistemas (o sistemas). En UML, el sistema entero es también un “subsistema” (la raíz) y, por tanto, también se puede mostrar como un objeto en un diagrama de interacción (como un DSS).

Obsérvese que el diagrama no muestra algunos mensajes, como ciertas colaboraciones de la *Venta*, para poner de relieve las interacciones significativas para la arquitectura.

Colaboraciones Dos decisiones de diseño al nivel de la arquitectura son:

1. ¿Cuáles son las partes importantes?
2. ¿Cómo se conectan?

Mientras el patrón de arquitectura Capas guía en la definición de las partes importantes, los patrones de micro-arquitectura como Fachada, Controlador y Observador se utilizan comúnmente para el diseño de las conexiones entre las capas y los paquetes. Esta sección estudia los patrones para la conexión y comunicación entre las capas y los paquetes.

Paquetes simples vs. subsistemas

Algunos paquetes o capas no son simplemente grupos de cosas conceptuales, sino que son verdaderos subsistemas con comportamiento e interfaces. Para contrastar:

- El paquete *FijarPrecios* no es un subsistema; agrupa simplemente la factoría y las estrategias que se utilizan para fijar los precios. De igual modo que con los paquetes Base como *java.util*.
- Por otro lado, los paquetes *Persistencia*, *MotorReglasPDV* y *Jess* son subsistemas. Son motores discretos con responsabilidades cohesivas que realizan un trabajo.

En UML, un subsistema se puede identificar con un estereotipo como en la Figura 30.6.

Fachada

Para los paquetes que representan subsistemas, el patrón más común de acceso es el Fachada, un patrón de diseño GoF. Esto es, un objeto fachada público define los ser-

vicios para el subsistema, y los clientes colaboran con la fachada, no con componentes internos del subsistema. Esto es cierto en lo que se refiere a *FachadaMotorReglasPDV* y *FachadaPersistencia* para acceder a los subsistemas de motor de reglas y de persistencia.

Normalmente, la fachada no debería incluir muchas operaciones de bajo nivel. Más bien, es deseable que la fachada incluyera un pequeño número de operaciones de alto nivel —los servicios de grano grueso—. Cuando una fachada da a conocer muchas operaciones de bajo nivel, tiende a perder la cohesión. Además, si la fachada será, o podría llegar a ser, un objeto distribuido o remoto (como un *bean* de sesión EJB, o un objeto servidor RMI), los servicios de grano fino dan lugar a problemas de rendimiento en las comunicaciones —muchas llamadas remotas pequeñas constituyen un cuello de botella en cuanto al rendimiento en los sistemas distribuidos—.

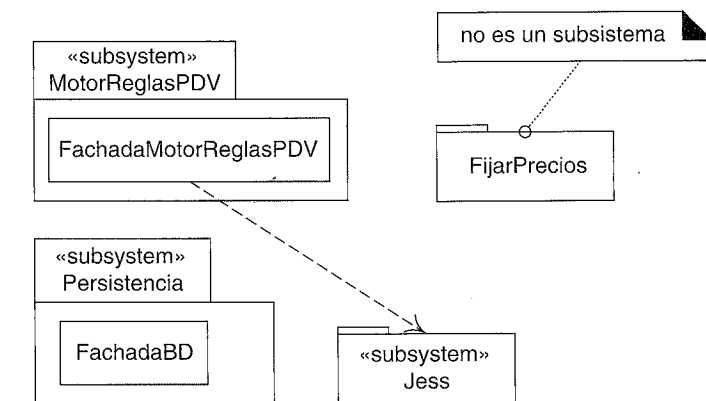


Figura 30.6. Estereotipos de subsistemas.

Por otro lado, normalmente una fachada no realiza su propio trabajo. Más bien, actúa para concentrar como mediador con los objetos del subsistema subyacente, que son los que hacen el trabajo.

Por ejemplo, la *FachadaMotorReglasPDV* es el envoltorio (*wrapper*) y único punto de acceso al motor de reglas para la aplicación PDV. Los otros paquetes no ven la implementación de este subsistema, puesto que está oculta detrás de la fachada. Suponga (ésta es sólo una de las muchas implementaciones) que el subsistema del motor de reglas del PDV se implementa colaborando con el motor de reglas Jess. Jess es un subsistema que expone en su interfaz muchas operaciones de grano fino (esto es común en los subsistemas de terceras partes muy generales). Pero la *FachadaMotorReglasPDV* no pone al descubierto en su interfaz las operaciones de bajo nivel de Jess, sino que proporciona sólo unas pocas operaciones de alto nivel como *esInvalida(linea, venta)*.

Si la aplicación tiene sólo un número “pequeño” de operaciones del sistema, entonces es habitual que la capa del Dominio o de la Aplicación sólo exponga un objeto a una capa superior. Por otro lado, la capa de Servicios Técnicos, que contiene varios subsistemas, expone al menos una fachada (o varios objetos públicos, si no se utilizan las fachadas) a las clases superiores para cada subsistema. Véase la Figura 30.7.

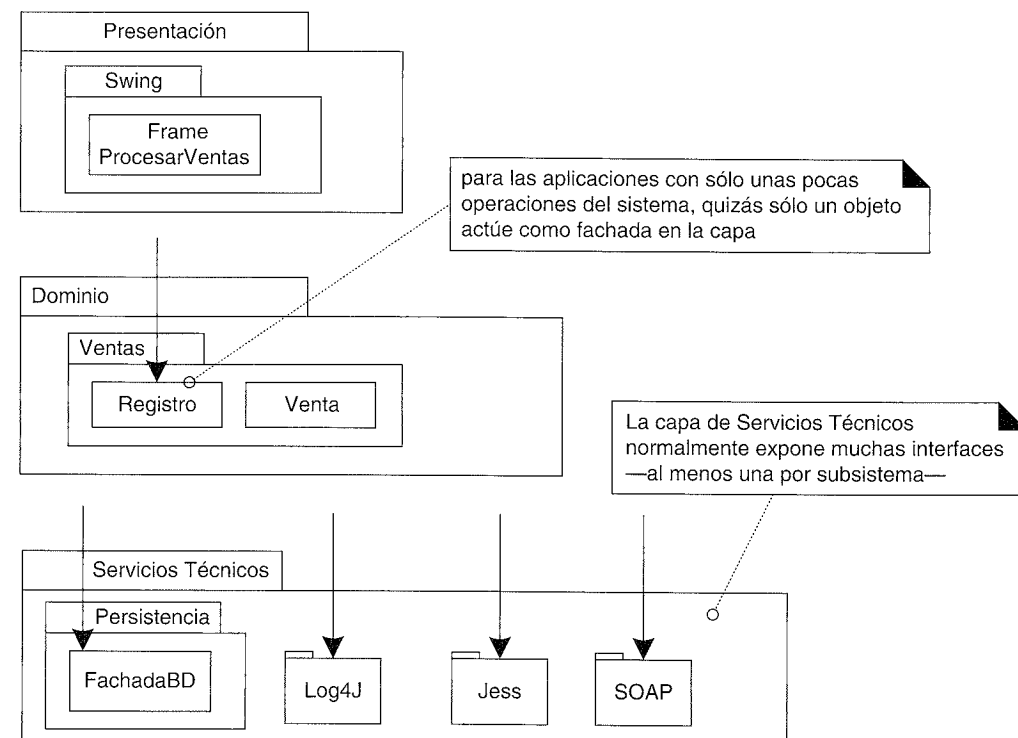


Figura 30.7. Número de interfaces que se muestran a las capas superiores.

Fachadas de Sesión y la capa de Aplicación

A diferencia de la Figura 30.7, cuando una aplicación tiene muchas operaciones del sistema y da soporte a muchos casos de uso, es común que haya más de un objeto mediando entre la capas de Presentación y del Dominio.

En la versión actual del sistema NuevaEra, existe un diseño simple de un único objeto *Registro* que actúa como fachada en la capa del Dominio (en virtud del patrón GRASP Controlador).

Sin embargo, cuando el sistema crece para manejar muchos casos de uso y operaciones del sistema, no es raro que se introduzca una capa de Aplicación de objetos que mantienen el estado de la sesión para las operaciones de un caso de uso, donde cada instancia de sesión representa una sesión con un cliente. Estos objetos se conocen como Fachadas de Sesión, y su uso es otra recomendación del patrón GRASP Controlador, como en el controlador fachada de sesión de caso de uso que es una variante del patrón. En la Figura 30.8 se presenta un ejemplo del modo en el que podría evolucionar la arquitectura del NuevaEra con una capa de Aplicación y fachadas de sesión.

Controlador

El patrón GRASP Controlador describe alternativas comunes en el manejo del lado del cliente (o controladores, como se les ha denominado) de las peticiones de las operaciones del sistema que se emiten desde la capa de Presentación. La Figura 30.9 lo ilustra.

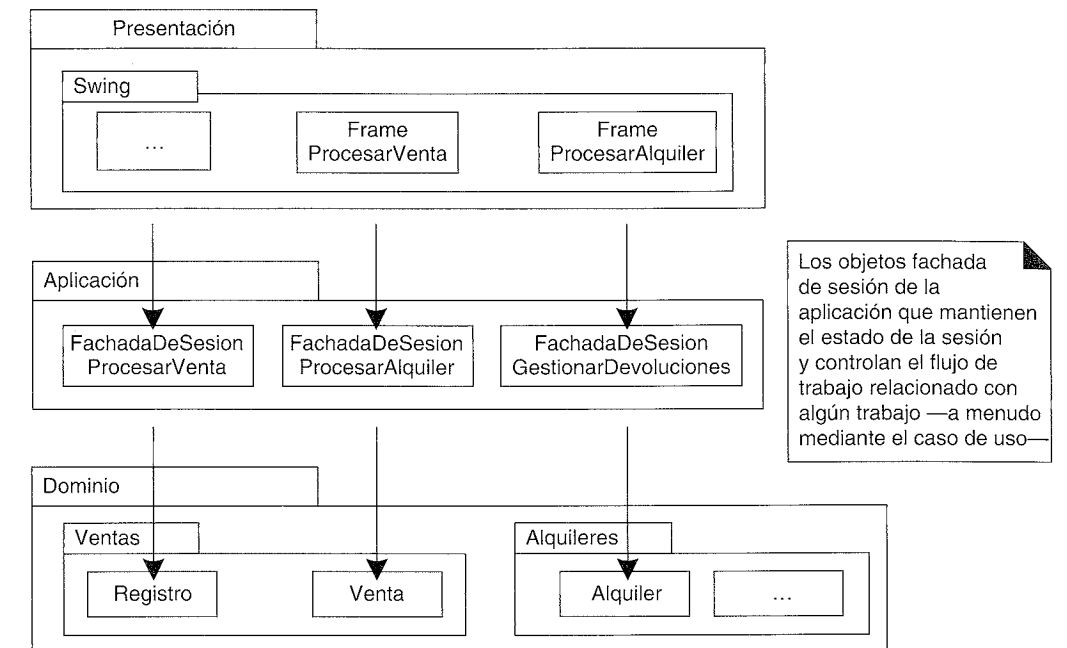


Figura 30.8. Fachadas de sesión y una Capa de Aplicación.

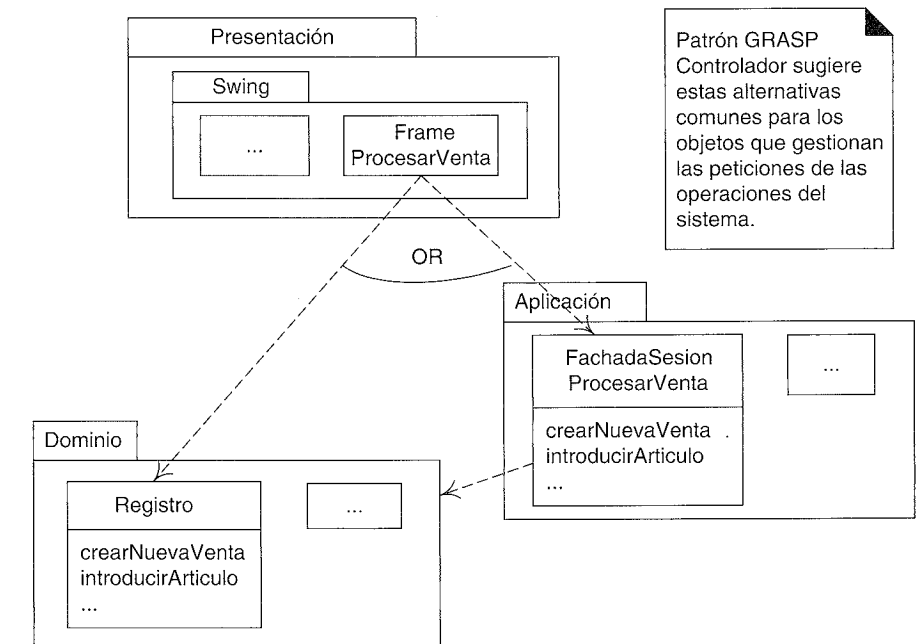


Figura 30.9. Alternativas del Controlador.

Operaciones del sistema y las capas

Los DSS ilustran las operaciones del sistema, ocultando los objetos de presentación del diagrama. Las operaciones del sistema que se invocan sobre el sistema en la Figura 30.10

son peticiones que genera un actor por medio de la capa de Presentación, en la capa de Aplicación o del Dominio.

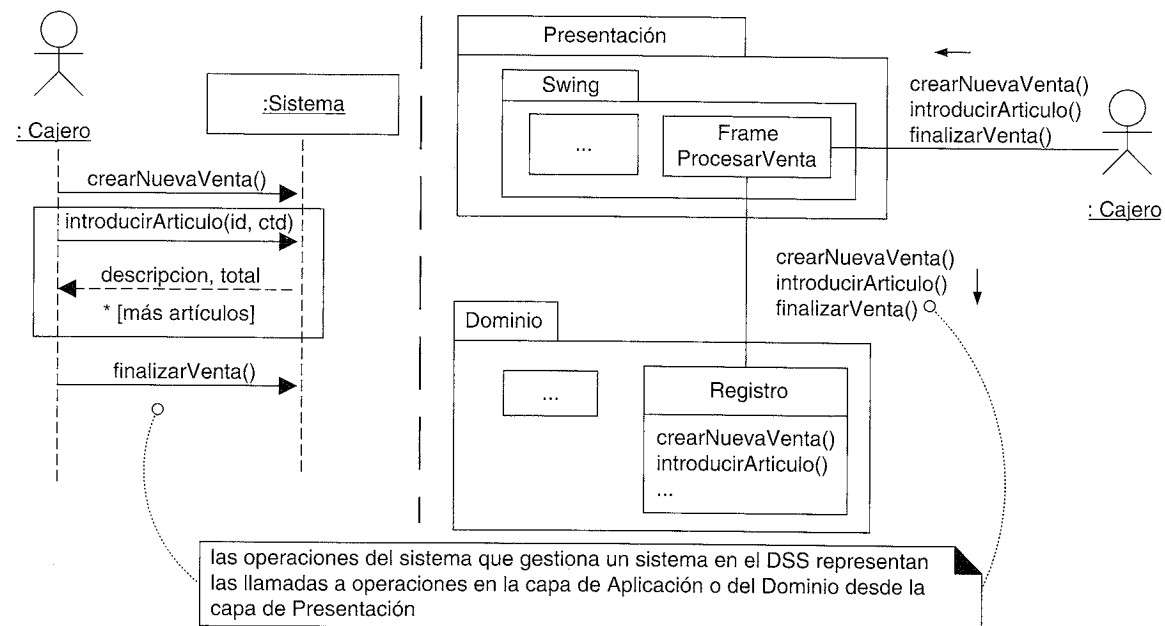


Figura 30.10. Operaciones del sistema en los DSS y en función de las capas.

Colaboraciones ascendentes con el Observador

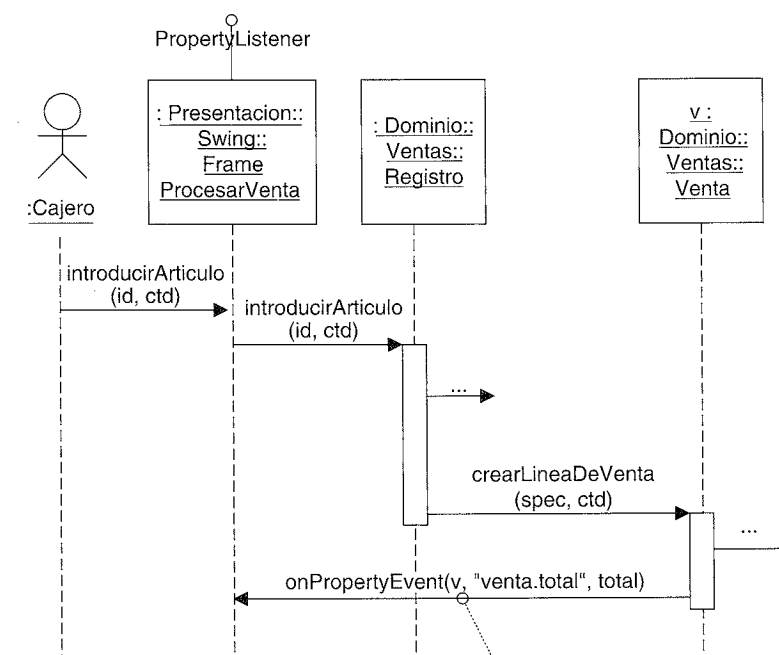
El patrón Fachada se utiliza normalmente para la colaboración “descendente” desde una capa más alta a una más baja, o para el acceso a los servicios de otros subsistemas de la misma capa. Cuando las capas más bajas Aplicación o Dominio necesitan comunicarse hacia arriba con la capa de Presentación, por lo general se hace por medio del patrón Observador. Es decir, los objetos UI en la capa de Presentación más alta implementan una interfaz como *PropertyListener* o *AlarmaListener*, y se suscriben o escuchan los eventos (como los eventos sobre la propiedad o de la alarma) que proceden de los objetos de las capas más bajas. Los objetos de las capas más bajas envían mensajes directamente a los objetos superiores de la capa de UI, pero sólo se acopla con los objetos vistos como cosas que implementan una interfaz como *PropertyListener*, no vistos como una ventana concreta de la GUI.

Esto se estudió cuando se presentó el patrón Observador. La Figura 30.11 resume la idea en relación con las capas.

Acoplamiento relajado entre las capas

Las capas en la mayoría de las arquitecturas en capas *no* están acopladas en el mismo sentido limitado que un protocolo de red basado en el Modelo OSI de 7 Capas. En el modelo del protocolo, existe una restricción estricta de que los elementos de la capa N sólo acceden a los servicios de la capa inmediatamente inferior N-1.

Esto raramente se sigue en las arquitecturas de los sistemas de información. Más bien, el estándar es una arquitectura de “capas relajadas” o “de capas transparentes”



La colaboración desde las capas más bajas hacia la capa de Presentación normalmente se lleva a cabo mediante el patrón Observador (Publicar-Suscribir). El objeto Venta ha registrado a los suscriptores que son objetos *PropertyListener*. Resulta que uno de ellos es una *JFrame Swing* de la GUI, pero la Venta no conoce a este objeto como un *JFrame* de la GUI sino como un *PropertyListener*.

Figura 30.11. Observador para la comunicación “ascendente” con la capa de Presentación.

[BMRSS96], en el que los elementos de una capa colaboran o se acoplan con varias de las otras capas.

Comentarios acerca del acoplamiento típico entre las capas:

- Todas las capas más altas dependen de los Servicios Técnicos y la capa Base.
 - Por ejemplo, en Java todas las capas dependen de los elementos del paquete *java.util*.
- Es sobre todo la capa del Dominio la que depende de la capa de Infraestructura del Negocio.
- La capa de Presentación realiza llamadas a la capa de Aplicación, que solicita los servicios de la capa del Dominio; la capa de Presentación no invoca al Dominio, a menos que no exista la capa de Aplicación.
- Si se trata de una aplicación “desktop” de un solo proceso, los objetos software en la capa del Dominio son visibles directamente, o se pasan entre, Presentación, Aplicación, y en menor extensión con, Servicios Técnicos.
 - Por ejemplo, asumiendo que el sistema NuevaEra es de este tipo, un objeto *Venta* y un *Pago* podrían ser visibles directamente en la Capa de Presentación de GUI, y pasarse también al subsistema de Persistencia en la capa de Servicios Técnicos.

- Por otro lado, si se trata de un sistema distribuido, en general se pasa a la capa de Presentación las **réplicas** serializables (también conocidas como **objetos data holder** u **objetos valor**) de los objetos de la capa del Dominio. En este caso, la capa del Dominio se despliega en un ordenador servidor, y los nodos clientes obtienen copias de los datos del servidor.

¿No es peligroso el acoplamiento entre los servicios técnicos y las capas básicas?

Como se presentó en la discusión acerca de los patrones GRASP Variaciones Protegidas y Bajo Acoplamiento, no es el acoplamiento en sí lo que constituye un problema, sino el acoplamiento innecesario en los puntos de variación y evolución que son inestables y costosos de arreglar. Es poco justificable el malgastar el tiempo y el dinero intentando abstraer u ocultar algo que no es probable que cambie, o si lo hace, el coste del impacto sería insignificante. Por ejemplo, si construimos una aplicación con las tecnologías Java, ¿cuál es el beneficio de ocultar en la aplicación el acceso a las librerías de Java? El alto acoplamiento en muchos puntos de las librerías es un problema poco probable, puesto que son (relativamente) estables y ubicuas.

Discusión Además de las cuestiones estructurales y de colaboración de este patrón discutidas anteriormente, entre otras cuestiones encontramos las siguientes.

¿Recursos externos o capa de base de datos externa abajo?

La mayoría de los sistemas confían en recursos o servicios externos, como una base de datos Oracle o un servicio de nombres y de directorio LDAP Novell. Éstos son componentes de implementación *físicos*, no una capa en la vista *lógica* de la arquitectura.

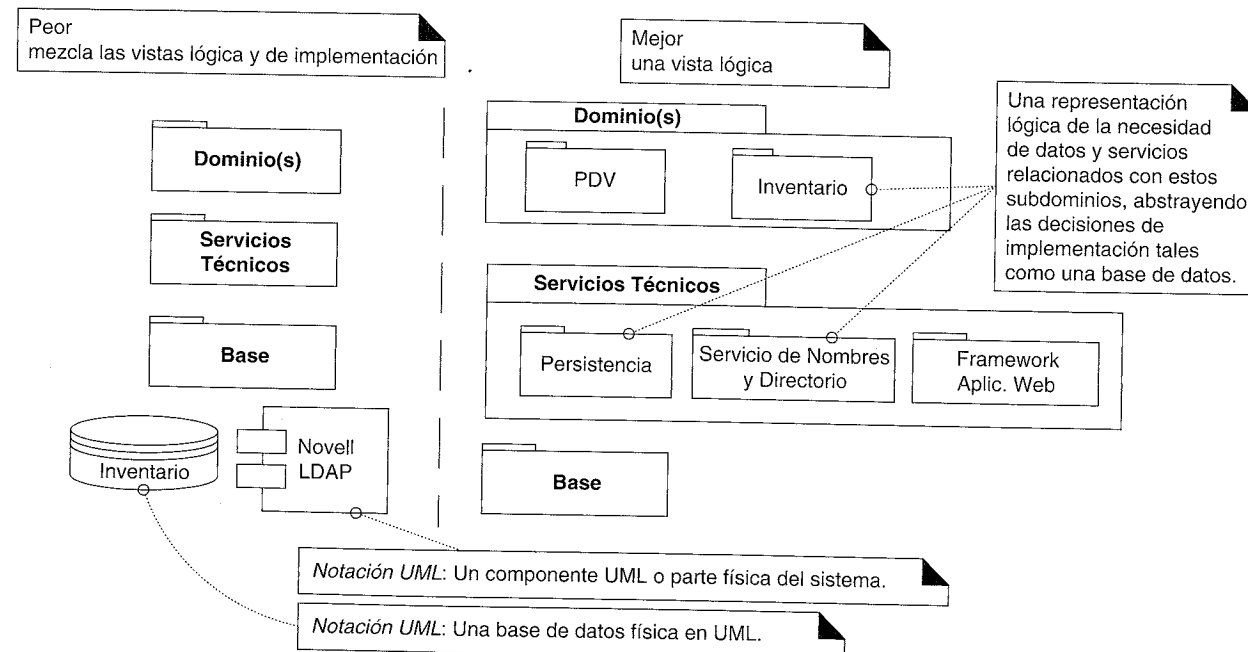


Figura 30.12. Mezcla de vistas de la arquitectura.

Mostrando los recursos externos, como una base de datos específica, en una capa “por debajo de” la capa Base (por ejemplo) mezcla la vista lógica y las vistas de despliegue o implementación de la arquitectura.

Más bien, en función de la vista lógica de la arquitectura y sus capas, se puede ver el acceso a un conjunto de datos persistentes (como los datos del inventario) como un subdominio de la Capa del Dominio —el subdominio del Inventario—. Y los servicios generales que proporcionan el acceso a las bases de datos se podrían ver como una partición del Servicio Técnico —el servicio de Persistencia—. Véase la Figura 30.12.

Vista lógica vs. las vistas de proceso y despliegue de la arquitectura

Las capas de la arquitectura son una vista lógica de la arquitectura, no una vista de despliegue de los elementos en procesos y nodos de procesos. Dependiendo de la plataforma, *todas* las capas podrían desplegarse en el mismo proceso en el mismo nodo, como una aplicación en un PDA portátil, o dispersas por muchos ordenadores y procesos para una aplicación web de gran escala.

En el Modelo de Despliegue del UP que establece la correspondencia entre la arquitectura lógica y los procesos y nodos, influye fuertemente la elección de la plataforma software y hardware y los *frameworks* de aplicación asociados. Por ejemplo, la elección entre J2EE o .NET influye en la arquitectura de despliegue.

Existen muchas formas de organizar estas capas lógicas para el despliegue, y en general el tema de la arquitectura de despliegue sólo se introducirá ligeramente, puesto que no es trivial, queda claramente fuera del alcance de este libro y depende de una discusión detallada de la plataforma software elegida, tal como J2EE.

¿Es opcional la capa de Aplicación?

De existir, la capa de Aplicación contiene los objetos responsables de conocer el estado de la sesión de los clientes, de mediar entre las capas de Presentación y del Dominio, y de controlar el flujo de trabajo.

El flujo podría organizarse controlando el orden de las ventanas o las páginas web, por ejemplo.

En cuanto a los patrones GRASP, forman parte de esta capa los objetos Controlador GRASP como el controlador de fachada de caso de uso. En sistemas distribuidos, forman parte de esta capa componentes tales como los *bean* sesión EJB (y los objetos sesión con estado en general).

En algunas aplicaciones, no es necesaria esta capa. Es útil (ésta no es una lista exhaustiva) cuando se cumple uno o más de los siguientes criterios:

- Se utilizarán diversas interfaces de usuario (por ejemplo, páginas web y una GUI Swing) en el sistema. Los objetos de la capa de Aplicación pueden actuar como Adaptadores que recopilan y reúnen los datos como se necesitan para diferentes UIs, y pueden actuar como Fachadas que envuelven y ocultan el acceso a la capa del Dominio.
- Es un sistema distribuido y la capa del Dominio está en un nodo diferente al de la capa de Presentación, y la comparten múltiples clientes. Normalmente se necesita

mantener la traza del estado de la sesión, y los objetos de la capa de Aplicación son una opción conveniente para esta responsabilidad.

- La Capa del Dominio no puede o no debe mantener el estado de la sesión.
- Existe un flujo de trabajo definido en función del orden controlado de las ventanas o páginas web que se deben presentar.

Pertenencia a un conjunto difuso en capas diferentes

Algunos elementos —son sin ninguna duda— miembros de una capa; una clase *Math*² forma parte de la capa Base. Sin embargo, especialmente entre las capas de Servicios Técnicos y Base, y Dominio e Infraestructura del Negocio, es difícil clasificar algunos elementos, porque la diferencia entre estas capas es, a grandes rasgos, “alto” frente a “bajo”, o “específico” frente “general”, que son un conjunto de términos difusos. Esto es normal, y rara vez es necesario optar por una clasificación definitiva —el equipo de desarrollo podría considerar que un elemento a grandes rasgos forma parte de los Servicios Técnicos y/o la capa Base que se consideran como un grupo, en general conocido como capa de Infraestructura.³

Por ejemplo:

- Suponga que se trata de un proyecto que utiliza las tecnologías de Java, y se ha elegido el framework de *logging*⁴ de libre distribución *Log4J* (parte del proyecto Jakarta). ¿Logging forma parte del Servicio Técnico o de la capa Base? Log4J es un framework general, pequeño, de bajo nivel. Es razonablemente miembro de los dos conjuntos difusos Servicios Técnicos y Base.
- Suponga que se trata de una aplicación web, y se ha elegido el framework para aplicaciones web *Struts* de Jakarta. Struts es un framework técnico, específico, relativamente de alto nivel y grande. Se puede justificar fuertemente que es miembro del conjunto de Servicios Técnicos, y débilmente que es miembro del conjunto Base.

Pero, lo que para una persona es el Servicio Técnico de Alto Nivel, para otras es la Base...

Finalmente, no es el caso de que las librerías que proporcionan una plataforma software sólo representan servicios Básicos de bajo nivel. Por ejemplo, tanto en .NET como J2SE+J2EE, los servicios incluyen funciones de relativamente alto nivel como los servicios de nombres y directorio.

Terminología: niveles, capas y particiones

La noción original de **nivel** (*tier*) en arquitectura era una capa lógica, no un nodo físico, pero la palabra ha pasado a utilizarse ampliamente para referirse a un nodo de procesamiento físico (o una agrupación de nodos), como el “nivel del cliente” (el ordenador del

² N. del T. Se refiere a la clase *Math* de Java que reúne funciones matemáticas.

³ Nótese que no existe una convención de nombres bien establecida para las capas, y es común que aparezca en la literatura sobre la arquitectura sobrecarga y contradicciones en los nombres.

⁴ N. del T. Permite al programador insertar sentencias log (control de errores) en programas Java sin incurrir en una penalización del rendimiento.

cliente). Esta presentación evitará el término para ser más claros, pero téngalo presente cuando lea literatura sobre arquitectura.

Las **capas** de una arquitectura se dice que representan los cortes verticales, mientras que las **particiones** representan una división horizontal de subsistemas relativamente paralelos de una capa. Por ejemplo, la capa de *Servicios* podría dividirse en particiones tales como *Seguridad* e *Informes* (Figura 30.13).

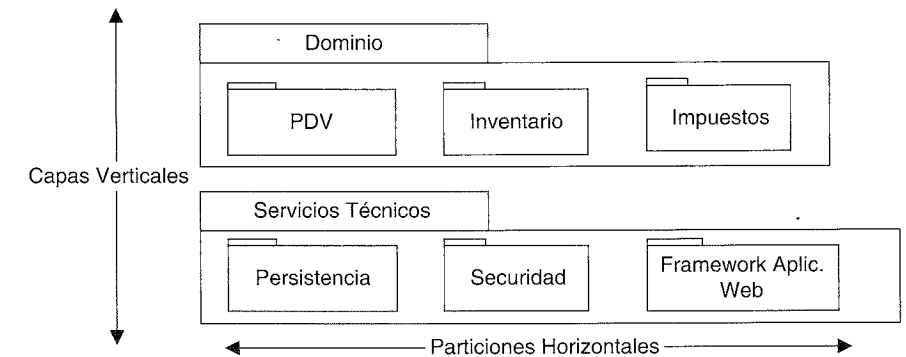


Figura 30.13. Capas y particiones.

Contraindicaciones y Compromisos

- En algunos contextos, añadir capas introduce problemas de rendimiento. Por ejemplo, en un juego en el que se utilizan muchos gráficos y que exige alto rendimiento añadir capas de abstracción e indirección sobre el acceso a los componentes de la tarjeta gráfica podría introducir problemas de rendimiento.

Beneficios

- El patrón Capas es uno entre varios patrones de arquitectura básicos; no es aplicable a todos los problemas. Por ejemplo, un sustituto es Tuberías y Filtros (*Pipes and Filters*) [BMRSS96]. Éste es útil cuando el problema principal de la aplicación implica el procesamiento de alguna cosa mediante una serie de transformaciones, como transformaciones de imagen, y el orden de las transformaciones puede cambiar. Pero incluso en el caso en el que el patrón de la arquitectura de más alto nivel sea Tuberías y Filtros, las tuberías y filtros particulares se pueden diseñar con Capas.
- En general, existe una separación de intereses, una separación entre los servicios de alto y bajo nivel, y de servicios específicos y generales de la aplicación. Esto reduce el acoplamiento y las dependencias, mejora la cohesión, incrementa el potencial para reutilizar e incrementa la claridad.
- La complejidad relacionada se encapsula y se descompone.
- Algunas capas se pueden reemplazar por implementaciones nuevas. Esto generalmente no es posible en las capas de más bajo nivel de Servicios Técnicos o Base (ej. *java.util*), pero podría ser posible en las capas de Presentación, Aplicación y Dominio.
- Las capas más bajas contienen funciones reutilizables.
- Algunas capas (sobre todo de Dominio y Servicios Técnicos) pueden ser distribuidas.
- Se ayuda al desarrollo en equipo debido a la segmentación lógica.

Implementación Implementación de las capas: personas y procesos

Es común y recomendable, en una iteración, contar con un desarrollador especializado en una capa o en un servicio.

Pero, no es el caso de que el equipo completo del proyecto se centre en una capa o servicio en una iteración. Sino más bien, es más habitual implementar cortes verticales entre las capas. Éste es el enfoque del UP en la fase de elaboración: elegir escenarios y requisitos que fueren, en cada iteración, a cubrir ampliamente muchos paquetes/capas/subsistemas significativos para la arquitectura, para descubrir y estabilizar los elementos importantes de la arquitectura en las primeras iteraciones.

Sin embargo, en este libro, no se ilustra este enfoque en el caso de estudio NuevaEra, porque hacerlo así requeriría una discusión previa de muchos y extensos temas —desde la programación de GUI a la correspondencia objeto-relacional y la optimización de sentencias SQL—. El caso de estudio del libro se ha centrado en el diseño de los objetos de la capa del Dominio, mientras que se reconoce que en realidad se llevaría a cabo un trabajo paralelo para desarrollar otras capas y subsistemas.

Los principios de diseño que se ilustran para el caso de estudio se pueden aplicar prácticamente en todas las capas del diseño.

Vista de implementación: correspondencia de la organización del código fuente con las capas y paquetes

La organización del código fuente forma parte del Modelo de Implementación del UP. Para lenguajes como Java o C#, que proporcionan un soporte sencillo para los paquetes (espacio de nombres, *namespace*), la correspondencia entre los paquetes lógicos y los paquetes de implementación es similar, con notables excepciones cuando se utilizan librerías de terceras partes⁵. De hecho, sólo es en las primeras etapas del desarrollo, cuando los paquetes se dibujan de manera especulativa, pero no se implementan, que hay diferencias significativas.

A lo largo del tiempo, cuando el código base crece, es normal abandonar los primeros dibujos especulativos (como los que acabamos de ver), y en lugar de eso utilizar una herramienta CASE UML para llevar a cabo un proceso de ingeniería inversa que lea el código fuente y genere un diagrama de paquetes. Entonces, estos diagramas de paquetes generados automáticamente, que reflejan de manera precisa el código (el diseño real) se convierten en la base de la vista lógica de la arquitectura.

Utilizando Java como ejemplo para mostrar la correspondencia con los paquetes de la implementación, las capas y paquetes que se ilustraban en la Figura 30.4 podrían corresponderse con los siguientes nombres de paquetes Java:

```
// --- PRESENTACIÓN
```

```
com.foo.nuevaera.ui.swing
com.foo.nuevaera.ui.texto
```

⁵ C++ también da soporte a los espacios de nombres, pero resulta difícil utilizar el lenguaje con docenas o cientos de espacios de nombres de grano fino; no así para Java o C#.

```
// --- DOMINIO
```

```
//paquetes relativamente específicos del proyecto NuevaEra
com.foo.nuevaera.dominio.ventas
com.foo.nuevaera.dominio.fijarprecios
com.foo.nuevaera.dominio.servicioacceso
com.foo.nuevaera.dominio.motorreglaspdv
```

```
//paquetes que se pueden diseñar fácilmente como
//servicios del negocio comunes a múltiples aplicaciones
com.foo.dominio.inventario
com.foo.dominio.pagoacredito
```

```
// --- SERVICIOS TÉCNICOS
```

```
//nuestro equipo crea
com.foo.servicios.persistencia
```

```
//de terceras partes
org.apache.log4j
org.apache.soap.rpc
jess
```

```
// --- BASE
```

```
//nuestro equipo crea
com.foo.utilidades
com.foo.utilstring
```

Nótese que se ha hecho un esfuerzo por evitar la utilización de los calificadores de los nombres de los paquetes dependientes de la aplicación (“nuevaera”) a menos que sea necesario. Por ejemplo, los paquetes de UI están relacionados con la aplicación NuevaEra, y de ahí que se califiquen con el nombre de la aplicación *com.foo.nuevaera.ui.**.

Para dar soporte a la reutilización, una práctica es nombrar los elementos de manera independiente de la aplicación, cuando sea apropiado. Un ejemplo sencillo, la utilidades de propósito general para los *String* creadas por el equipo NuevaEra, se colocan en *com.foo.utilstring*, en lugar de en *com.foo.nuevaera.utilstring*. Además, *com.foo.utilstring* debería colocarse en el repositorio de código fuente de la compañía al nivel de la compañía, en lugar de enterrarse en las carpetas de código fuente del proyecto NuevaEra. No puede reutilizar lo que no puede ver.

Otro ejemplo, considere los servicios para acceder a los sistemas de terceras partes del inventario y autorización de pago a crédito. Aunque fue el equipo NuevaEra el que los creó al servicio del proyecto NuevaEra, son servicios del negocio generales —uno podría imaginar el acceso a un sistema de inventario desde el interior de otras aplicaciones—; lo mismo para la autorización de pagos a crédito. De ahí que sea preferible *com.foo.dominio.inventario* en lugar de *com.foo.nuevaera.dominio.inventario*.

Por otro lado, el paquete MotorReglasPDV está completamente relacionado con el proyecto del PDV NuevaEra. Así, *com.foo.nuevaera.dominio.motorreglaspdv*.

En caso de duda, califique el paquete con el nombre del proyecto. Siempre se puede cambiar más adelante.

Usos Conocidos Un gran número de sistemas orientados a objetos modernos (desde aplicaciones desktop a sistemas web distribuidos J2EE) se desarrollan con Capas; sería más difícil encontrar uno que no esté desarrollado así, que uno que sí. Retrocediendo más lejos en la historia:

Máquinas virtuales y sistemas operativos

Comenzando en los años sesenta, los arquitectos de sistemas operativos abogaban por el diseño de sistemas operativos en función de capas claramente definidas, donde la capa "más baja" encapsulaba el acceso a los recursos físicos y proporcionaba procesos y servicios de E/S, y las capas más altas invocaban estos servicios. Entre éstos se encuentran Multics [CV65] y el sistema THE [Dijkstra68].

Todavía antes —en los cincuenta— los investigadores sugirieron la idea de una máquina virtual (MV) con un lenguaje máquina universal de bytecodes (por ejemplo, UNCOL [Conway1958]), de manera que las aplicaciones pudieran escribirse en las capas más altas de la arquitectura (y ejecutarse sin tener que recompilarse en diferentes plataformas), sobre la capa de la máquina virtual, que por su parte se asentaría por encima del sistema operativo y los recursos de la máquina. Alan Kay aplicó la arquitectura de capas con MV en Flex un sistema de computación personal basado en la orientación a objetos que marcó un hito [Kay68] y después (1972) Kay y Dan Ingalls en la influyente máquina virtual de Smalltalk [GK76] —la progenitora de las MVs más recientes como la Máquina Virtual de Java—.

Sistemas de información: la arquitectura clásica de tres-niveles

Una primera descripción de la arquitectura de capas para sistemas de información que ha tenido mucha influencia, que incluía una interfaz de usuario y el almacenamiento persistente de los datos, era conocida como **arquitectura de tres niveles** (Figura 30.14), fue descrita en los años setenta en [TK78]. El término no alcanzó popularidad hasta mediados de los noventa, en parte debido a que en [Gartner95] se presentaba como una solución a los problemas asociados con el uso extendido de las arquitecturas de dos niveles.

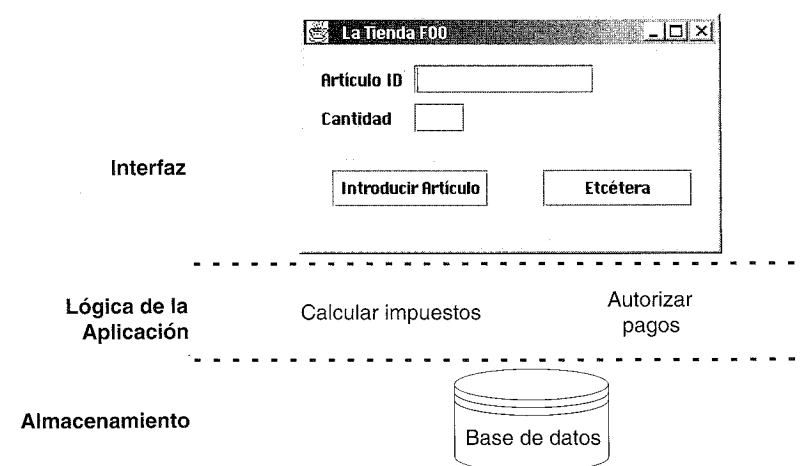


Figura 30.14. Vista clásica de una arquitectura de tres niveles.

El término original ahora es menos común, pero su motivación aún es relevante.

Una descripción clásica de los niveles verticales en la arquitectura de tres niveles es:

1. **Interfaz:** ventanas, informes, etcétera.
2. **Lógica de la Aplicación:** tareas y reglas que dirigen el proceso.
3. **Almacenamiento:** mecanismos de almacenamiento persistente.

La cualidad singular de una arquitectura de tres niveles es la separación de la lógica de la aplicación en un nivel de software intermedio distinto para la lógica. El nivel de la interfaz se encuentra relativamente liberado del procesamiento de la aplicación; las ventanas o las páginas web remiten las peticiones de las tareas al nivel intermedio. El nivel intermedio se comunica con la capa de almacenamiento back-end.

Hubo algún malentendido, se creía que la descripción original implicaba o requería un despliegue físico en tres ordenadores, pero la descripción dada era puramente lógica; la asignación de los niveles a los nodos de ordenadores podría variar de uno a tres. Véase la Figura 30.15.

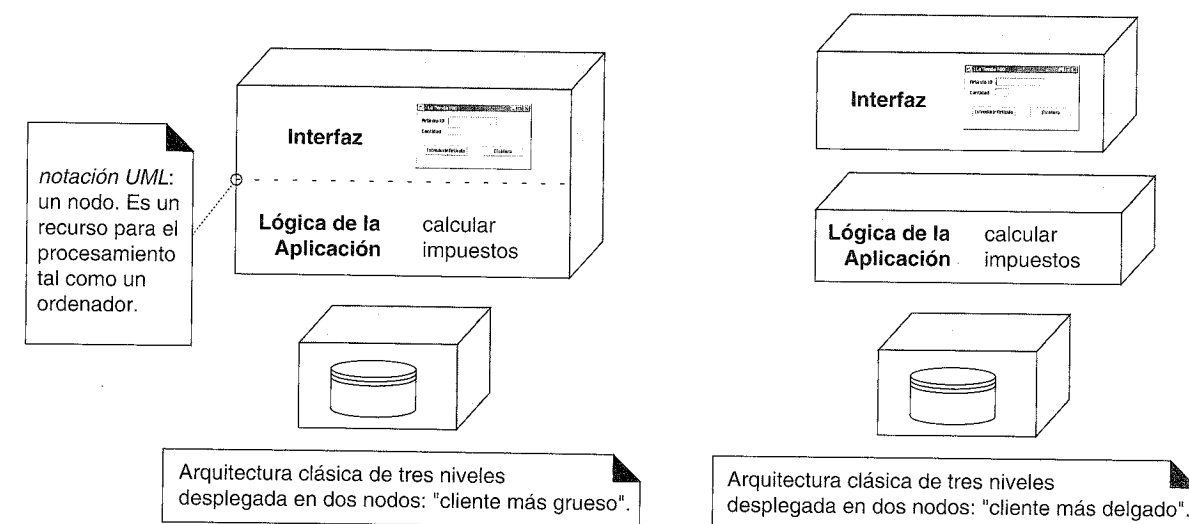


Figura 30.15. Una división lógica en tres niveles desplegada en dos arquitecturas físicas.

El Grupo Gartner contrastó la arquitectura de tres niveles con un diseño **en dos niveles**, en el cual, por ejemplo, la lógica de la aplicación se coloca en las definiciones de las ventanas, que lee y escribe directamente en una base de datos; no existe nivel intermedio que separe la lógica de la aplicación. Las arquitecturas cliente-servidor de dos niveles se hicieron populares especialmente al surgir herramientas como Visual Basic y PowerBuilder.

Los diseños en dos niveles tienen (en algunos casos) la ventaja de un desarrollo rápido inicial, pero puede sufrir los inconvenientes que se presentaron en la sección de *Problemas*. No obstante, hay aplicaciones que son ante todo simples sistemas de uso intensivo de datos CRUD (creación, recuperación, actualización y borrado), para los que esto es una opción adecuada.

Patrones Relacionados

- Indirección: las capas pueden añadir un nivel de indirección a los servicios de un nivel inferior.
- Variaciones Protegidas: las capas pueden proteger contra el impacto de implementaciones que varían.
- Bajo Acoplamiento y Alta Cohesión: las capas dan soporte efectivo a estos objetivos.
- La aplicación específica a los sistemas de información orientados a objetos se describe en [Fowler96].

También Conocido Como

Arquitectura en Capas [Shaw96, Gemstone00].

30.3. Principio de Separación Modelo-Vista

Este principio se ha discutido varias veces; esta sección lo resume.

¿Qué tipo de visibilidad tendrían que tener otros paquetes de la capa de Presentación? ¿Cómo deberían comunicarse las clases que no son ventanas con las ventanas? Es conveniente que no exista un acoplamiento directo de otros componentes con los objetos ventana puesto que las ventanas están relacionadas con una aplicación específica, mientras que (idealmente) los componentes que no son ventanas podrían reutilizarse en nuevas aplicaciones o conectarse a una nueva interfaz. Éste es el principio de Separación Modelo-Vista.

En este contexto, el **modelo** es un sinónimo de la capa del Dominio de los objetos. La **Vista** es un sinónimo para los objetos de la presentación, como ventanas, applets e informes.

El principio de **Separación Modelo-Vista**⁶ establece que los objetos del modelo (dominio) no deberían conocer *directamente* a los objetos de la vista (presentación), al menos como objetos de la vista. Así, por ejemplo, un objeto *Registro* o *Venta* no debería enviar un mensaje directamente a un objeto ventana de la GUI *FrameProcesarVenta*, pidiéndole que muestre algo, cambie de color, se cierre, etcétera.

Como se discutió anteriormente, una relajación legítima de este principio es el patrón Observador, donde los objetos del dominio envían mensajes a los objetos de la UI vistos sólo en términos de una interfaz como *PropertyListener* o *AlarmaListener*.

Una parte adicional de este principio es que las clases del dominio encapsulan la información y el comportamiento relacionado con la lógica de la aplicación. Las clases de las ventanas son relativamente delgadas; son responsables de la entrada y salida, y capturar los eventos del GUI, pero no mantienen datos ni proporcionan directamente ninguna funcionalidad de la aplicación.

⁶ Éste es un principio clave en el patrón *Modelo-Vista-Controlador* (MVC). El MVC fue originalmente un patrón de pequeña escala de Smalltalk-80, y relacionaba los objetos de datos (modelos), los elementos gráficos de la GUI (vistas), y el manejo de los eventos del ratón y el teclado (controladores). Más recientemente, el término “MVC” también se ha adoptado por la comunidad de diseño distribuido para aplicarlo en los niveles de la arquitectura a gran escala. El Modelo es la Capa del Dominio, la Vista es la Capa de Presentación, y el Controlador son los objetos del flujo de trabajo en la capa de Aplicación.

Los motivos para la Separación Modelo-Vista son los siguientes:

- Dar soporte a definiciones de modelos cohesivos que se centren en los procesos del dominio, en lugar de preocuparse de las interfaces de usuario.
- Permitir separar el desarrollo de las capas del modelo y la interfaz de usuario.
- Minimizar el impacto de los cambios de los requisitos de la interfaz sobre la capa del dominio.
- Permitir que se conecten fácilmente otras vistas a una capa de dominio existente, sin afectar a la capa del dominio.
- Permitir múltiples vistas simultáneas sobre el mismo modelo del dominio, como una vista de la información sobre las ventas en formato tabular o mediante un diagrama de barras.
- Permitir la ejecución de la capa del modelo de manera independiente de la capa de interfaz de usuario, como en un sistema de procesamiento de mensajes o en modo de procesamiento por lotes.
- Permitir trasladar fácilmente la capa del modelo a otro framework de interfaz de usuario.

Separación Modelo-Vista y comunicación “ascendente”

¿Cómo pueden obtener las ventanas la información que tiene que mostrar? Normalmente, es suficiente que envíen mensajes a los objetos del dominio, preguntando sobre la información que luego mostrarán en elementos gráficos —un modelo de **escrutinio** (*polling*) o **tirar-desde-arriba** (*pull-from-above*) para mostrar las actualizaciones—.

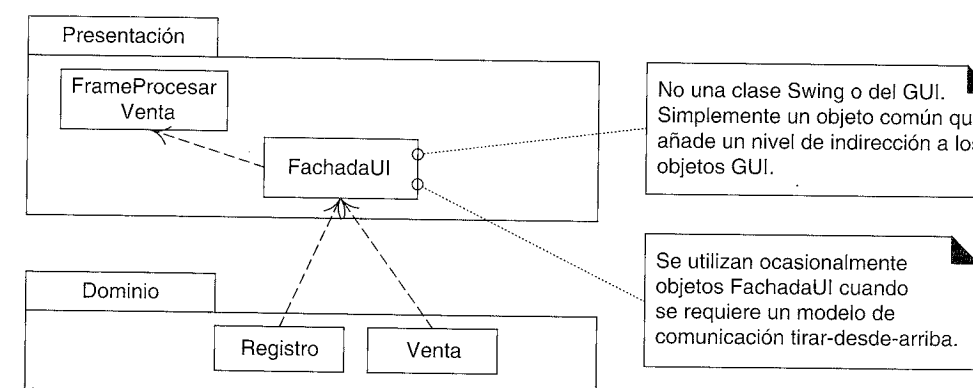


Figura 30.16. Ocasionalmente se utiliza una *FachadaUI* en la capa de Presentación para los diseños tirar-desde-arriba.

Sin embargo, un modelo de escrutinio a veces es insuficiente. Por ejemplo, comprobar cada segundo cientos de objetos para descubrir que sólo han cambiado uno o dos, lo cual se utiliza entonces para actualizar la información que se muestra en la GUI, no es

eficiente. En este caso es más eficiente que los pocos objetos del dominio que cambian se comuniquen con las ventanas para que provoque que se actualice la información que muestran cuando el estado del objeto del dominio cambia. Las situaciones típicas de este caso son:

- Aplicaciones de vigilancia, como la gestión de redes de telecomunicaciones.
- Aplicaciones de simulación que requieren visualización, como el modelado aerodinámico.

En estas situaciones, se requiere un modelo **empujar-desde-abajo** (*push-from-below*) para mostrar las actualizaciones. Debido a la restricción del patrón Separación Modelo-Vista, necesitamos establecer una comunicación “indirecta” desde los objetos inferiores hacia las ventanas —hacen subir la notificación de actualización desde abajo—.

Existen dos soluciones comunes:

1. El patrón Observador, haciendo que los objetos de la GUI simplemente parezcan objetos que implementan una interfaz como *PropertyListener*.
2. Un objeto fachada de Presentación. Es decir, añadir una fachada en la capa de Presentación que recibe las peticiones desde abajo. Es un ejemplo de la inclusión de Indirección para proporcionar Variaciones Protegidas si cambia la GUI. Por ejemplo, véase la Figura 30.16.

30.4. Lecturas adicionales

Existe abundante literatura sobre las arquitecturas en capas, tanto impresas como en la Web. Una serie de patrones en *Pattern Languages of Program Design*, volumen 1 [CS95] abordó por primer vez el tema en forma de patrones, aunque las arquitecturas en capas se usan y se escribe sobre ellas desde al menos los años sesenta; el volumen 2 continúa con patrones relacionados con las capas adicionales. *Pattern-Oriented Software Architecture* volumen 1 [BMRSS96] trata de manera adecuada el patrón Capas.

ORGANIZACIÓN DE
LOS PAQUETES DE LOS MODELOS
DE DISEÑO E IMPLEMENTACIÓN

*Si estuvieses arando un campo,
¿qué preferirías utilizar? ¿Dos bueyes fuertes o 1.024 gallinas?*
Seymour Cray

Objetivos

- Organizar los paquetes para reducir el impacto de los cambios.
- Conocer notación UML alternativa para la estructura de los paquetes.

Introducción

Si el equipo de desarrollo depende en gran medida de un paquete X, no es conveniente que X sea muy inestable (pasando por muchas versiones nuevas), puesto que incrementa el impacto en el equipo en cuanto a constantes re-sincronizaciones de las versiones y arreglos en el software dependiente que deja de funcionar en respuesta a los cambios en X (**destrazo de versiones**).

Esto nos suena y es obvio, pero algunas veces un equipo no presta atención a la identificación y estabilización de los paquetes de los que más depende, y termina experimentando más destrazo de versiones de lo que es necesario.

Este capítulo se fundamenta en la introducción que se hizo en el capítulo anterior de capas y paquetes, sugiriendo más heurísticas de grano fino para la organización de paquetes, para reducir este tipo de impacto de los cambios. El objetivo es crear un diseño de paquetes físicos robusto.

Uno sufre una organización de paquetes frágil, sensible a las dependencias, mucho más rápidamente en C++ que en Java, debido a las dependencias hiper-sensibles de compilación y enlace en C++; un cambio en una clase tiene un fuerte impacto en las dependencias transitivas, lo que nos lleva a la recompilación de muchas clases, y enlazar de

nuevo¹. Por tanto, estas sugerencias son útiles especialmente en proyectos C++, y menos para proyectos Java, Smalltalk, o C# (como ejemplos).

El útil trabajo de Robert Martin [Martin95], que ha tratado de abordar el diseño físico y el empaquetado de las aplicaciones C++, ha influido en algunas de las guías siguientes.

Diseño físico del código fuente en el Modelo de Implementación

Este asunto es un aspecto del **diseño físico** —el Modelo de Implementación del UP para el empaquetado del código fuente—.

Mientras estamos simplemente dibujando los diagramas del diseño de paquetes en una pizarra o herramienta CASE, podemos colocar de manera arbitraria los tipos en cualquier paquete funcionalmente cohesivo sin ningún impacto. Pero durante el diseño físico del código fuente —la organización de los tipos en unidades de versión físicas como “paquetes” de Java o C++— nuestras elecciones influirán en el nivel de impacto del desarrollador cuando tengan lugar los cambios en estos paquetes, si existen muchos desarrolladores compartiendo un código base común.

31.1. Guías para la organización de paquetes

Guía: Paquete de secciones verticales y horizontales funcionalmente cohesivas

El principio “intuitivo” básico es dividir en módulos en base a la cohesión funcional —se agrupan los tipos que están fuertemente relacionados en función de su participación en un objetivo común, servicio, colaboración, política y función—. Por ejemplo, todos los tipos en el paquete *FijarPrecios* de NuevaEra están relacionados con la política para fijar los precios a los productos. Las capas y los paquetes en el diseño de NuevaEra se organizan por grupos funcionales.

Normalmente es suficiente con conjeturas informales sobre la agrupación por funciones (“Creo que la clase *LineaDeVenta* pertenece a *Ventas*”) pero, además, otro indicio de agrupamiento funcional es un grupo de tipos con fuerte acoplamiento interno y débil acoplamiento entre grupos. Por ejemplo, el *Registro* está acoplado fuertemente con la *Venta*, que está fuertemente acoplada con *LineaDeVenta*.

El acoplamiento interno del paquete, o **cohesión relacional**, puede cuantificarse, aunque tal análisis formal raramente tiene una utilidad práctica. Para los curiosos, una medida es:

$$CR = \frac{\text{NumeroDeRelacionesInternas}}{\text{NumeroDeTipos}}$$

¹ En C++ los paquetes podrían realizarse como namespaces, pero es más probable que suponga la organización del código fuente en directorios físicos separados —uno por cada “paquete”—.

Donde el *NumeroDeRelacionesInternas* incluye las relaciones de atributos y parámetros, herencia, e implementaciones de interfaces entre los tipos del paquete.

Un paquete de 6 tipos con 12 relaciones internas tiene CR=2. Un paquete de 6 tipos con 3 relaciones entre los tipos tiene CR=0.5. Los números más altos dan a entender una mayor cohesión o relación en el paquete.

Nótese que esta medida es menos aplicable a los paquetes formados en su mayor parte por interfaces; es más útil para paquetes que contienen algunas clases de implementación.

Un valor CR muy bajo sugiere algunas de las siguientes cuestiones:

- El paquete contiene elementos no relacionados y no está bien factorizado.
- El paquete contiene elementos no relacionados y al diseñador no le importa deliberadamente. Esto es habitual con paquetes de utilidades o servicios dispares (ej. *java.util*), donde no es importante un valor alto o bajo de CR.
- Contiene una o más agrupaciones de subconjuntos de grupos con alto CR, pero el global no lo es.

Guía: Paquete de una familia de interfaces

Coloque una familia de *interfaces* relacionadas funcionalmente en un paquete separado —separado de las clases de implementación. Esto no es fundamental para el caso de una o dos interfaces relacionadas, sino cuando existe una familia de quizás tres o más interfaces. El paquete de la tecnología Java EJB *javax.ejb* es un ejemplo: es un paquete de al menos doce interfaces; las implementaciones están en paquetes separados.

Guía: Paquete por trabajo y por agrupaciones de clases inestables

El contexto para esta discusión es que los paquetes son normalmente la unidad básica del trabajo de desarrollo y de versiones. Es menos común trabajar y lanzar como versión únicamente una clase.

Suponga que 1) existe un paquete grande P1 con treinta clases, y 2) la tendencia en el trabajo es que un subconjunto concreto de diez clases (de C1 a C10) se modifica regularmente y se vuelve a lanzar.

En este caso, reagrupe P1 en P1-a y P1-b, donde P1-b contiene las diez clases sobre las que se trabaja frecuentemente.

De este modo, el paquete se ha reagrupado en subconjuntos más estables y menos estables, o de manera más general, en grupos relacionados con el trabajo. Esto es, si se trabaja de manera conjunta en la mayoría de los tipos de un paquete, entonces es conveniente agruparlos.

Idealmente, menos desarrolladores dependen de P1-b que de P1-a, y agrupando esta parte inestable en un paquete separado, no hay tantos desarrolladores que se vean

afectados por las nuevas versiones de P1-b como por los relanzamientos del paquete P1 original más grande.

Nótese que esta reagrupación se debe a una tendencia de trabajo que surge. Es difícil identificar de manera especulativa una buena estructura de paquetes en las primeras iteraciones. Evoluciona incrementalmente a lo largo de las iteraciones de la elaboración, y debería ser un objetivo de la fase de elaboración (porque es significativo para la arquitectura) estabilizar la mayoría de la estructura del paquete cuando se complete la fase.

Esta guía ilustra la estrategia básica: **reducir una dependencia extendida sobre paquetes inestables**.

Guía: Los más responsables son los más estables

Si los paquetes más responsables (de los que más se depende) son inestables, existe un mayor riesgo de extender el impacto de los cambios por las dependencias. Un caso extremo es, si un paquete de utilidades ampliamente utilizado como *com.foo.utilidades* cambia con frecuencia, podrían dejar de funcionar muchas cosas. Por tanto, la Figura 31.1 ilustra una estructura de dependencia apropiada.

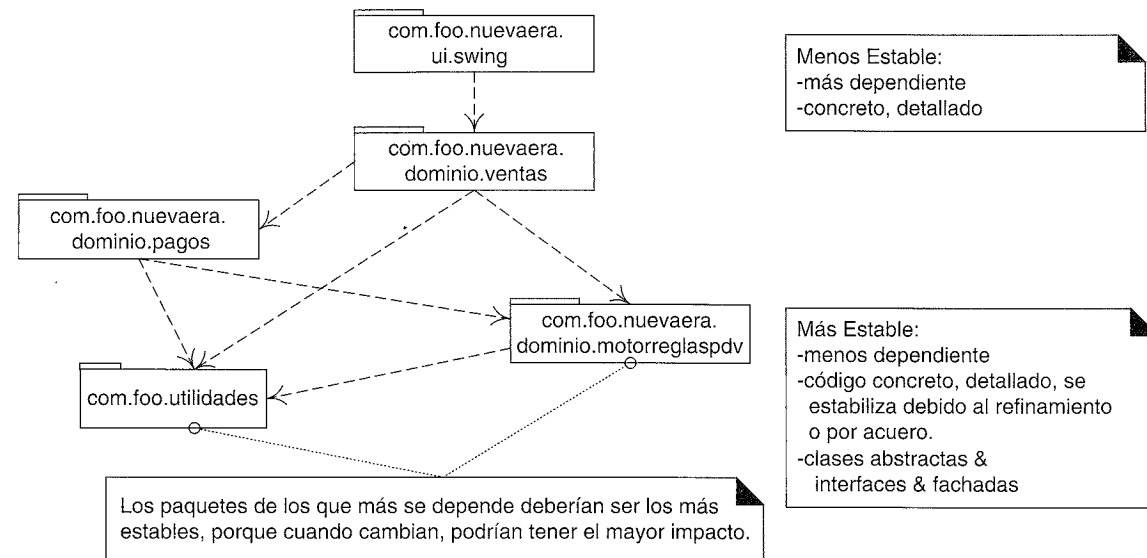


Figura 31.1. Los paquetes más responsables deberían ser más estables.

Visualmente, los paquetes inferiores en este diagrama deberían ser los más estables. Hay formas diferentes de incrementar la estabilidad en un paquete:

- Contiene sólo o en su mayor parte interfaces y clases abstractas.
 - Por ejemplo, *java.sql* contiene ocho interfaces y seis clases, y las clases son en su mayor parte tipos simples y estables como *Time* y *Date*.

- No depende de otros paquetes (es independiente), o depende de otros paquetes muy estables, o encapsula sus dependencias de manera que los dependientes no se ven afectados.
 - Por ejemplo, *com.foo.nuevaera.dominio.motorreglaspdv* oculta la implementación del motor de reglas detrás de un simple objeto fachada. Incluso si la implementación cambia, los paquetes dependientes no se ven afectados.
- Contiene código relativamente estable porque se implementó con mucho cuidado y se refinó antes de lanzar la versión.
 - Por ejemplo, *java.util*.
- Es obligatorio una planificación lenta de cambios.
 - Por ejemplo, *java.lang* el paquete central de las librerías de Java, simplemente no se le permite que cambie con frecuencia.

Guía: Separar los tipos independientes

Organice los tipos que se puedan utilizar independientemente o en contextos diferentes en paquetes separados. Sin una estimación cuidadosa, la agrupación según funcionalidades comunes podría no proporcionar el nivel adecuado de granularidad en la factorización de paquetes.

Por ejemplo, suponga que se ha definido un subsistema para los servicios de persistencia en un paquete *com.foo.servicios.persistencia*. En este paquete hay dos clases de utilidades/auxiliares muy generales *UtilidadesJDBC* y *OrdenesSQL*. Si hay utilidades generales para trabajar con JDBC (servicios Java para el acceso a una base de datos relacional), entonces se pueden utilizar de manera independiente del subsistema de persistencia, en cualquier ocasión que el desarrollador utilice JDBC. Por tanto, es mejor migrar estos tipos a un paquete separado, como *com.foo.utilidades.jdbc*. La Figura 31.2 ilustra.

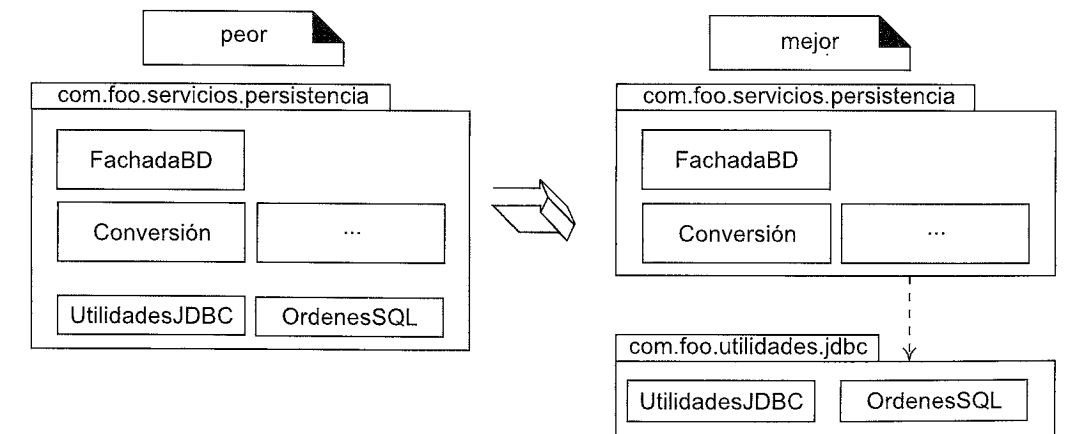


Figura 31.2. Separar los tipos independientes.

Guía: Utilice factorías para reducir la dependencia en paquetes concretos

Una forma de incrementar la estabilidad de los paquetes es reducir la dependencia de clases concretas de otros paquetes. La Figura 31.3 ilustra la situación “anterior”.

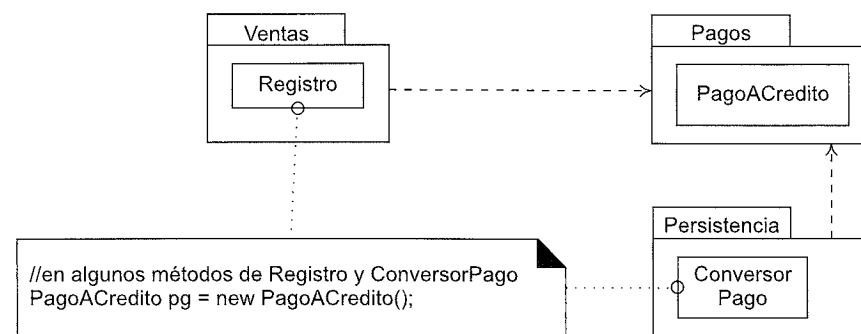


Figura 31.3. Acoplamiento directo de paquetes concretos debido a la creación.

Suponga que tanto el *Registro* como el *ConversorPago* (una clase que almacena/recupera los objetos pago en una base de datos relacional) crean instancias de *PagoACredito* del paquete *Pagos*. Un mecanismo para incrementar la estabilidad a largo plazo de los paquetes *Ventas* y *Persistencia* es detener la creación explícita de objetos de clases concretas definidas en otros paquetes (*PagoACredito* en *Pagos*).

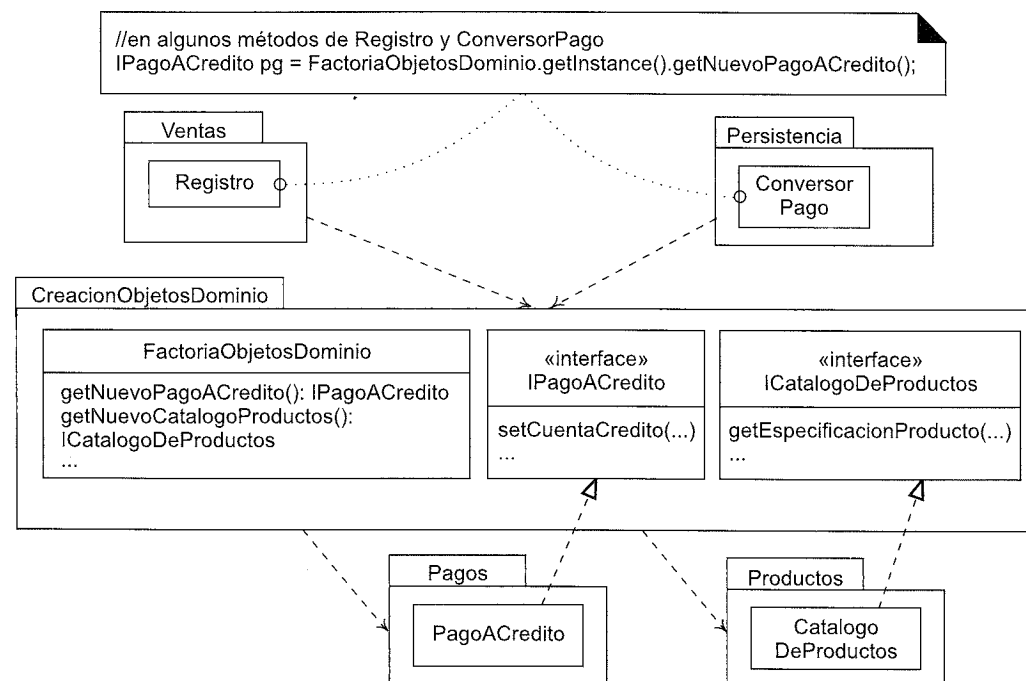


Figura 31.4. Reducción del acoplamiento con un paquete concreto utilizando un objeto factoría.

Podemos reducir el acoplamiento con este paquete concreto utilizando un objeto factoría que crea las instancias, pero cuyos métodos de creación devuelven objetos declarados en términos de interfaces en lugar de clases. Véase la Figura 31.4.

Patrón Factoría de Objetos del Dominio

El uso de factorías de objetos del dominio con interfaces para la creación de *todos* los objetos del dominio es un estilo de diseño común. He visto mencionarlo informalmente en la literatura sobre el diseño como el patrón Factoría de Objetos del Dominio, pero no conozco ninguna referencia en el que se describa formalmente como patrón.

Guía: Paquetes sin ciclos

Si un grupo de paquetes tiene un ciclo de dependencias entonces podrían necesitar que se les tratara como un paquete más grande en términos de una unidad de versión. Esto no es conveniente porque lanzando paquetes mayores (o paquetes agregados) incrementa la probabilidad de afectar a algo.

Hay dos soluciones:

1. Separar los tipos que participan en el ciclo en un paquete nuevo más pequeño.
2. Romper el ciclo con una interfaz.

Los pasos para romper el ciclo con una interfaz son:

1. Redefinir las clases de las que se depende en uno de los paquetes para implementar nuevas interfaces.
2. Definir las nuevas interfaces en un paquete nuevo.
3. Redefinir los tipos dependientes para que dependan de las interfaces del nuevo paquete, en lugar de las clases originales.

La Figura 31.5 ilustra esta estrategia.

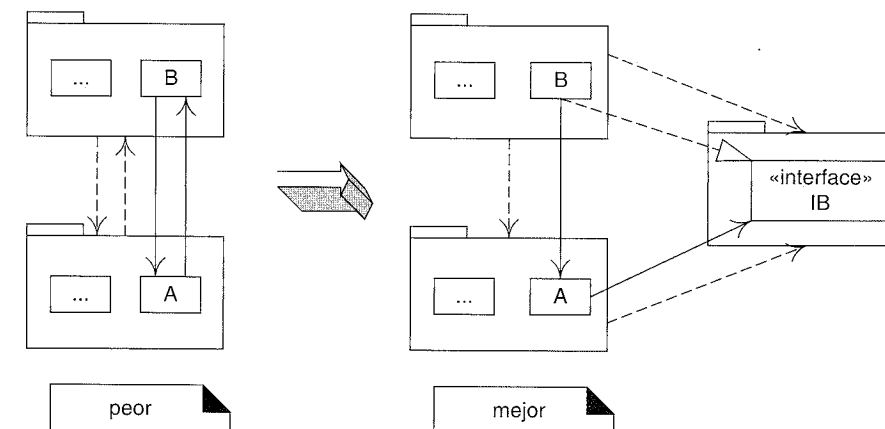


Figura 31.5. Rotura de un ciclo de dependencias.